



Django

- web 이란
- Python, Django 설치
- Django 구조

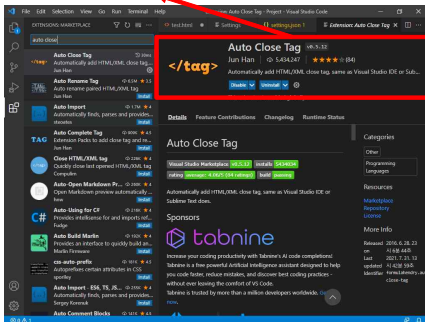
vscode 환경설정

Django html, html

통합단축키 설정

Vscode (Django,html 2개 모두 자동완성기능 사용) 1

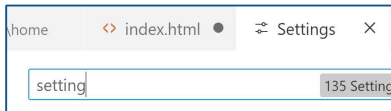
1. "Auto Close Tag" 확장팩 설치



2. settings.json 파일 수정

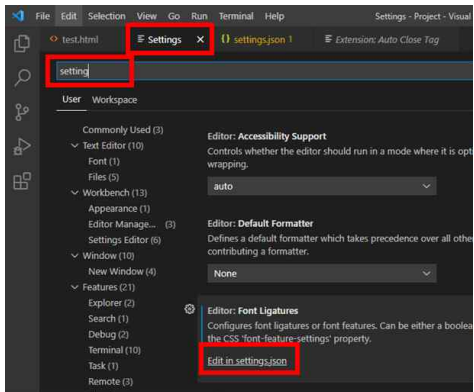
macOS : ⌘ + ,
Windows : Ctrl + , (실패: 누름)

3. setting 입력

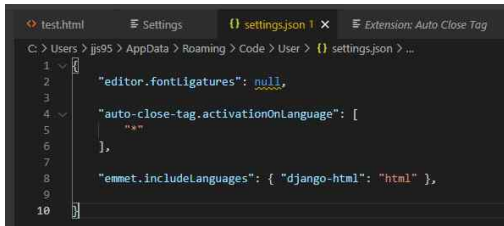


Vscode (Django,html 2개 모두 자동완성기능 사용) 1

4. Edit in settings.json 클릭



5. "auto-close-tag.activationOnLanguage" 수정



```
"editor.fontLigatures": False,  
"auto-close-tag.activationOnLanguage": [  
  "*" ],  
"emmet.includeLanguages": { "django-html": "html" },
```

1 네트워크



삼성전자 블로그
SAMSUNG TOMORROW

모든 서비스는 네트워크와 연결

1 네트워크



완전한 웹 프레임워크:

Django는 강력한 기능을 가진 완전한 웹 프레임워크

높은 수준의 추상화: Django는 많은 작업을 추상화하여 개발자가 더 쉽게 애플리케이션을 만들 수 있도록 도와줍니다. 이로 인해 빠른 개발이 가능

Admin 기능 제공: Django는 관리자 페이지를 생성하기 위한 기능을 내장하고 있어 데이터베이스 관리 및 CRUD 작업을 쉽게 수행

ORM (Object-Relational Mapping): Django의 ORM은 데이터베이스와의 상호 작용을 추상화하여 개발자가 SQL 쿼리 대신 파이썬 코드를 사용하여 데이터베이스 조작을 수행

보다 큰 프로젝트에 적합: Django는 대규모 및 복잡한 웹 애플리케이션에 적합한 구조를 가지고 있음. 자체적으로 많은 기능과 패키지를 제공하여 개발자의 작업을 줄여줍니다.



마이크로 웹 프레임워크:

Flask는 더 작고 가볍게 설계된 웹 프레임워크입니다. 필요한 기능만 선택적으로 추가하여 사용할 수 있으며, 더 많은 제어를 원하는 개발자에게 적합합니다. 최소한의 추상화: Flask는 개발자에게 높은 수준의 제어를 제공합니다. 이는 프로젝트가 복잡해질수록 더 유용할 수 있습니다.

확장성: Flask는 필요한 기능을 원하는 대로 추가하여 사용할 수 있기 때문에 작은 프로젝트부터 대규모 프로젝트까지 다양한 규모의 애플리케이션을 개발할 수 있습니다.

ORM 미포함: Flask는 ORM을 기본으로 제공하지 않습니다. 따라서 ORM 라이브러리(예: SQLAlchemy)를 별도로 선택하여 사용해야 함.

유연한 구조: Flask는 개발자가 애플리케이션의 구조와 기능을 더 유연하게 설계할 수 있도록 합니다. 이는 몇 가지 기본 구성 요소만 제공하고 필요한 기능은 확장 가능한 방식으로 추가할 수 있음을 의미합니다.

1

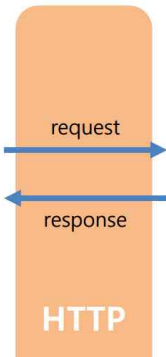
client와 server



device



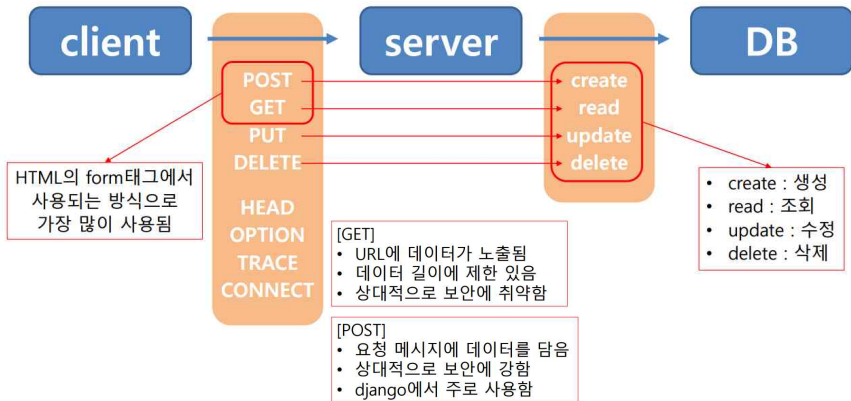
software

**client**

연결을 유지하기 위해서
쿠키 또는 세션 이용

**server**

1 HTTP 처리 방식



1 URL

URL 기본 형태

https://search.naver.com/search.naver?query=weather

프로토콜

도메인(호스트)

경로

쿼리

REST URL 형태

https://search.naver.com/search/today/weather

프로토콜

도메인(호스트)

URL 맵핑 (데이터)스트링

1 REST와 Django url 매핑

<https://search.naver.com/search/today/weather>

URL 매핑 (데이터)스트링

URLconf(urls.py)

search/<str:today>/weather	→	view.weather(request, today)
search/<str:sport>/movie	→	view.movie(request, sport)
search/<str:korea>/food	→	view.food(request, korea)
search/<str:map>/seoul	→	view.seoul(request, map)

함수 호출

1 우아한 URL

우아한 URL

정규표현식 Regular Expression을 추가적으로 사용할 수 있음

```
from django.urls import path
from . import views
urlpatterns = [
    path('articles/2003/', views.special_case_2003),
    path('articles/<int:year>/', views.year_archive),
    path('articles/<int:year>/<int:month>/', views.month_archive),
    path('articles/<int:year>/<int:month>/<slug:slug>/', views.article_detail),
]
```

1 서버 구성



[웹서버]

- 주로 정적인 데이터 요청 처리
- 동적인 데이터 요청 시 애플리케이션 서버에 전달

[애플리케이션서버]

- 주로 동적인 데이터 요청 처리
- Database 연동

1 HTTP프로토콜

메스드형	의미	CRUD와 매핑되는 역할
1xx	Informational (정보 제공)	임시적인 응답으로, 현재 클라이언트의 요청까지 처리되었으니 계속 진행하라는 의미입니다. HTTP 1.1 버전부터 추가되었습니다.
2xx	Success(성공)	클라이언트의 요청이 서버에서 성공적으로 처리되었다는 의미입니다.
3xx	Redirection (리다이렉션)	완전한 처리를 위해서 추가적인 동작을 필요로 하는 경우입니다. 주로 서버의 주소 또는 요청한 URI의 웹 문서가 이동되었으니, 그 주소로 다시 시도해라는 의미입니다.
4xx	Client Error (클라이언트 에러)	없는 페이지를 요청하는 것처럼 클라이언트의 요청 메시지 내용이 잘못된 경우입니다.
5xx	Server Error (서버 에러)	서버 측 사정에 의해서 메시지 처리에 문제가 발생한 경우입니다. 서버의 부하, DB 처리 과잉 오류, 서버에서 악발션이 발생하는 경우가 이에 해당합니다.

상태 코드	상태 텍스트	응답 문구	서버 측면에서의 의미
2xx	Success	성공	클라이언트가 요청한 동작을 수신하여 이해했고, 승낙했으며 성공적으로 처리했다.
200	OK	성공	서버가 요청을 성공적으로 처리했다.
201	Created	생성됨	요청이 처리되어서 새로운 리소스가 생성되었다. 응답 헤더 Location에 새로운 리소스의 절대 URI를 기록합니다.
202	Accepted	허용됨	요청은 접수했지만 처리가 완료되지 않았다. 클라이언트는 응답 헤더의 Location, Retry-After를 참고하여 다시 요청을 보냅니다.
3xx	Redirection	리다이렉션	클라이언트는 요청을 마치기 위해 추가적인 동작을 취해야 한다.
301	Moved Permanently	영구 이동	지정된 리소스가 새로운 URI로 이동했다. 이동할 곳의 새로운 URI는 응답 헤더 Location에 기록합니다.

상태 코드	상태 텍스트	응답 문구	서버 측면에서의 의미
303	See Other	다른 위치 보기	다른 위치로 요청하라. 요청에 대한 처리 결과를 응답 헤더 Location에 표시된 URI에서 GET으로 취득할 수 있습니다. 브라우저의 폼 요청을 POST로 처리하고 그 결과 화면으로 리다이렉션시킬 때 자주 사용하는 응답 코드입니다.
307	Temporary Redirect	임시 리다이렉션	일시로 리다이렉션 요청이 필요하다. 요청한 URI가 없으므로, 클라이언트는 메스드를 그대로 유지한 채 응답 헤더 Location에 표시된 다른 URI로 요청을 재송신할 필요가 있습니다. 클라이언트는 향후 요청 시점에 위치를 계속 사용해야 합니다. 302의 의미를 정확하게 재정의해서 HTTP/1.1의 307 응답으로 추가되었습니다.
4xx	Client Error	클라이언트 에러	클라이언트의 요청에 오류가 있다.
400	Bad Request	잘못된 요청	요청의 구문이 잘못되었다. 클라이언트가 모르는 4xx 계열의 응답 코드가 반환된 경우에도 클라이언트는 400과 동일하게 처리하도록 규정하고 있습니다.
401	Unauthorized	권한 없음	지정된 리소스에 대한 액세스 권한이 없다. 응답 헤더 WWW-Authenticate에 필요한 인증 방식을 지정합니다.
403	Forbidden	금지됨	지정된 리소스에 대한 액세스가 금지되었다. 401 인증 처리 이외의 사유로 리소스에 대한 액세스가 금지되었음을 의미합니다. 리소스의 존재 자체를 문제이고, 실은 경우는 404 응답 코드를 사용할 수 있습니다.
404	Not Found	찾을 수 없음	지정된 리소스를 찾을 수 없다.
5xx	Server Error	서버 에러	클라이언트의 요청은 유효한데, 서버가 처리에 실패했다.
500	Internal Server Error	내부 서버 오류	서버쪽에서 에러가 발생했다. 클라이언트가 모르는 5xx 계열의 응답 코드가 반환된 경우에도 클라이언트는 500과 동일하게 처리하도록 규정하고 있습니다.
502	Bad Gateway	불량 게이트웨이	게이트웨이 또는 프록시 역할을 하는 서버가 그 뒷단의 서버로부터 잘못된 응답을 받았다.
503	Service Unavailable	서비스 제공불가	현재 서버에서 서비스를 제공할 수 없다. 보통은 서버의 과부하나 서비스 점검 등 일시적인 상태입니다.

Djanago 설치

2 python 패키지 pip 업그레이드

Pip 업그레이드

pip : Python Install Package 의 약자로 SW패키지를 사용하기 위한 명령어

pip install --upgrade pip (리눅스)

python -m pip install --upgrade pip (윈도우)

Django 설치

pip install Django (Django 설치)

pip install Django --upgrade (Django 업그레이드)

2

Django 삭제

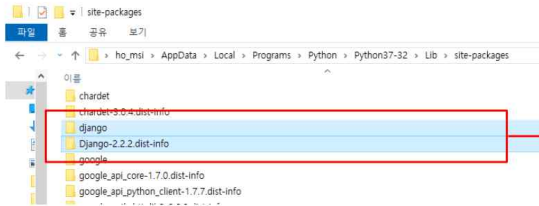
장고 설치 경로 이동 후 디렉토리 삭제

```
C:\WINDOWS\system32\cmd.exe
Python\python37-32\lib\site-packages (from Django) (2018.9)
C:\Users\who_msi>python -c "import django; print(django.__path__)"
['C:\Users\who_msi\AppData\Local\Programs\Python\Python37-32\lib\site-packages\django']
C:\Users\who_msi>
```

Django 설치 경로 확인

[위치확인]

python -c "import django; print(django.__path__)"



Django 디렉토리 삭제

가상환경 – Virtualenv

한 서버에서 여러개의 프로젝트 진행 시,
여러 프로젝트의 파이썬 라이브러리 버전간 충돌을 방지하기 위해
프로젝트마다 독립적인 환경을 제공

#해당 폴더 생성 후> 폴더 위치 이동

```
pip install virtualenv      #가상환경 설치
python -m venv my_env      #가상환경 생성 -현재 폴더에 my_env폴더 생성됨.
```

Django 를 설치, upgrade 설치를 해야 함.

#가상환경 활성화/가상환경 해제

```
venv\Scripts\activate.bat  #my_env\Scripts\ 폴더에서 activate.bat 실행
                           #(my_env) 표시가 나타남.
(my_env) venv\Scripts\deactivate  #deactivate 빠져나옴.
```

같은 환경에서
진행된다면
설정할 필요없
음

Project / Apps 생성

3 프로젝트 세팅

생성

Project 생성

```
django-admin startproject spjt
```

App 생성

```
python manage.py startapp students
```

project 세팅

setting.py

```
students.apps.StudentsConfig
```

urls.py

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path(' ', include('home.urls')),  
    path('students/', include('students.urls')),  
]
```

app 세팅

Students urls.py

```
from . import views  
app_name = 'students'  
urlpatterns = [  
    path('reg/', views.regStudent, name='reg'),  
]
```

Home urls.py

```
from . import views  
app_name = ''  
urlpatterns = [  
    path("", views.index, name="index"),  
]
```

views.py

```
def regStudent(request):  
    return render(request, 'students/registerStudent.html')
```

db 계정 생성

```
python manage.py migrate  
python manage.py makemigrations : db 반영할 내용 확인
```

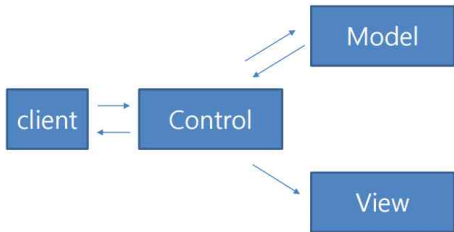
관리자 계정 생성

```
python manage.py createsuperuser
```

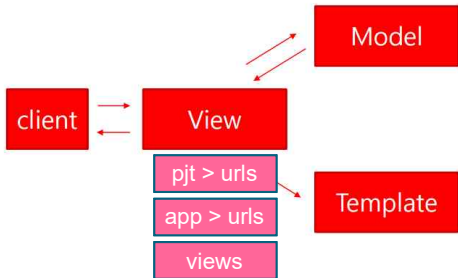
서버 구동

```
python -u manage.py runserver 0.0.0.0:8000
```

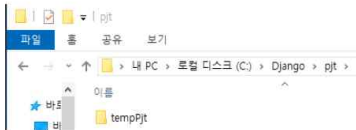
MVC 패턴



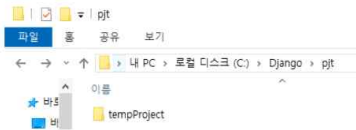
MVT 패턴



3 프로젝트 디렉토리 이름 변경



```
C:\Django\pjt\tempPjt>cd ../  
C:\Django\pjt>move tempPjt tempProject  
1개의 디렉터를 이동했습니다.  
C:\Django\pjt>
```



→ 디렉토리 이름 변경

이름을 변경해도 되고 하지 않아도 됨.

ren w01Pjt wPjt
- w01Pjt 이름을 wPjt 이름으로 변경



- ▼ w01Pjt
 - > students
 - ▼ w01Pjt
 - > __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py
 - wsgi.py
 - db.sqlite3
 - manage.py

세부세팅

setting.py

3 프로젝트 세팅

생성

Project 생성

```
django-admin startproject spjt
```

App 생성

```
python manage.py startapp students
```

project 세팅

setting.py

```
students.apps.StudentsConfig
```

urls.py

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path(' ', include('home.urls')),  
    path('students/', include('students.urls')),  
]
```

app 세팅

Students urls.py

```
from . import views  
app_name = 'students'  
urlpatterns = [  
    path('reg/', views.regStudent, name='reg'),  
]
```

Home urls.py

```
from . import views  
app_name = ''  
urlpatterns = [  
    path("", views.index, name="index"),  
]
```

views.py

```
def regStudent(request):  
    return render(request, 'students/registerStudent.html')
```

db 계정 생성

```
python manage.py migrate  
python manage.py makemigrations : db 반영할 내용 확인
```

관리자 계정 생성

```
python manage.py createsuperuser
```

서버 구동

```
python -u manage.py runserver 0.0.0.0:8000
```

```
import os
```

```
setting.py
```

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'students.apps.StudentsConfig',
    'students',
    'home',
]
```

* fullName을 다 적거나
app_name을 적어도 됨.

```
# 한글,한국시간 설정
```

```
LANGUAGE_CODE = 'ko-kr'
```

```
TIME_ZONE = 'Asia/Seoul'
```

```
STATIC_URL = 'static/'
```

```
# static파일 위치 설정 - 현재root 모든 static폴더를 가져옴
```

```
# 정적파일 : css,js,image
```

```
STATICFILES_DIRS = (
    os.path.join(BASE_DIR, 'static'),
)
```

- 개발모드와 운영모드 설정

```
DEBUG = True
```

```
DEBUG = True #개발모드
DEBUG = False #운영모드
```

```
ALLOWED_HOSTS = [localhost, '127.0.0.1']
```

```
#개발모드
```

```
- 입력하지 않아도 자동으로 localhost로 정의
```

```
#운영모드
```

```
- 서버 IP주소 입력
```

- * 실서버 운영시 : False , 서버 ip를 등록해야 서비스 됨.

```
urls.py
```

```
from django.urls import path,include
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    path(' ',include('students.urls')),
```

```
    path('students/',include('students.urls')),
```

```
]
```

3

app > views.py

프로젝트 > app > urls.py 설정 (app안 urls.py 생성후 세팅)
프로젝트 > views.py

students app > urls.py

urls.py

```
from django.urls import path
from . import views
```

```
app_name = 'students'
urlpatterns = [
    path("", views.index, name='index'),
    path('reg/', views.regStudent, name='reg'),
]
```

views.py

```
from django.http import HttpResponse
```

```
def index(request):
    return render(request, 'students/index.html')
```

```
def regStudent(request):
    return
    render(request, 'students/registerStudent.html')
```

```
def home(request):
    return HttpResponse('페이지 호출')
```

[참조] 페이지 이동 함수

```
return redirect('index')          ### url 입력
# return redirect(reverse('index')) ### app_name
# return HttpResponseRedirect(reverse('index')) from django.http import HttpResponseRedirect
```

1. HttpResponseRedirect : HttpResponseRedirect(data, content_type)
- Json,html까지도 보낼수 있음

2. HttpResponseRedirect : HttpResponseRedirect(url)
- 지정된 url페이지로 redirect.

3. render : render(request, template_name, context=None, content_type=None, status=None, using=None)
- context값을 함께 넘겨줄 수 있음
예) return render(request, 'index.html',context)

4. redirect : redirect (to, permanent=False, *args, **kwargs) – redirect 데이터 전달방법 뒤에 설명.
- Render로 context에 값을 전달해서 location.href='/' 로 이동시켜야 함.

예) return redirect(reverse('board:read',args=(id,))) ## url내부로 데이터 전달할때 사용, 튜플 1개 일때 뒤에 쉼표 넣어야 함.

```
return redirect( reverse( 'students:view',args=(name,) ) )
return redirect('students:view',name)#내부적으로 resolve_url() 사용
return redirect(f'/students/{name}/view')
```

return redirect('index') # redirect('앱이름:앱이름') -> redirect('앱이름') : 없을경우 1개만 씀

return redirect('/blog/detail/1/') #상대 url

return redirect('http://example.com/blog/detail/1') #절대 url

[참조] redirect로 값 전달을 하려면

```
# 메인페이지 호출
def index(request):
    error = request.GET.get('error')
    context = {"error":error}
    return render(request,'index.html',context)
```

Home > views.py

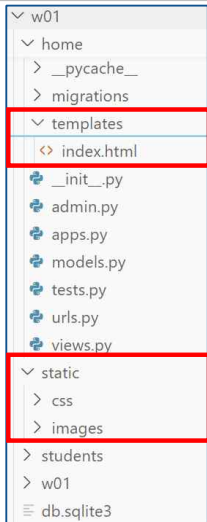
```
url = reverse('index')
return redirect(f'{url}?error=1')
```

member > views.py

```
<h2>메인페이지</h2>
<h2>아이디 :{{error}}</h2>
```

Index.html

3 Html파일 저장폴더 - Templates (*.html)



1. templates 파일은 직접 생성을 해야 함.

Templates 폴더 생성
- 카테고리 별 폴더 생성 가능

Database 생성

4 ORM(Object Relational Mapping)이란

ORM (Object Relational Mapping)

객체(Object)와 DB의 테이블을 자동으로 연결(Mapping)시켜
RDB 테이블을 객체 지향적으로 사용하게 해주는 기술

다만, ORM만으로는 SQL의 모든 부분을 다루기가 어려움

백엔드 개발 시 SQL 쿼리에 대한 지식과 경험이 바탕이 되어야
더 효과적인 백엔드 개발이 가능하다

4 기본 사용자 및 그룹 테이블 생성

1. 사용자 및 그룹 테이블 생성

`python manage.py migrate`

2. 데이터베이스 변경사항 반영

`python manage.py makemigrations`

`python manage.py migrate`

3. 관리자 계정생성

`python manage.py createsuperuser`

Username :

Email address :

Password :

Password(again) :

--- 패스워드 사용 여부 확인 [y/N] :

4. 서버구동

`python manage.py runserver 0.0.0.0:8000`

`python manage.py runserver` # 포트설정 없이 사용

가능

Django administration

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#) [Change](#)

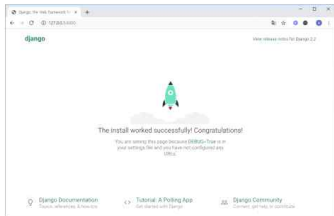
Users [+ Add](#) [Change](#)

STUDENTS

Students [+ Add](#) [Change](#)

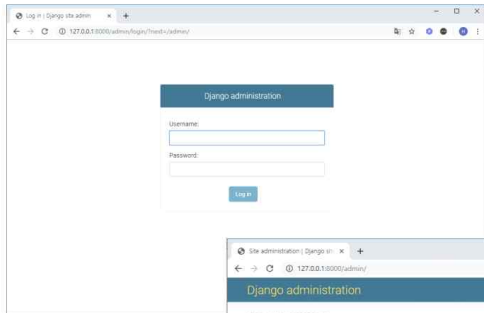
*** 서버 화면창에서 print 출력 할때
`python -u manage.py runserver`**

4 관리자 계정 생성 및 서버 구동



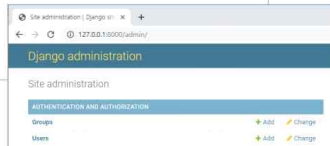
client 화면

[웹사이트 주소]
http://127.0.0.1:8000/



admin 화면

[웹사이트 주소]
http://127.0.0.1:8000/admin



models.py에 테이블 클래스 정의 후 admin.py에 등록

```
from django.db import models

class Student(models.Model):
    s_name = models.CharField(max_length=100)
    s_major = models.CharField(max_length=100)
    s_age = models.IntegerField(default=0)
    s_grade = models.IntegerField(default=0)
    s_gender = models.CharField(max_length=30)

    def __str__(self):
        return self.s_name
```

student 테이블 생성을 위한 Student 클래스 정의

student table					
id(PK)	s_name	s_major	s_age	s_grade	s_gender

```
from django.contrib import admin
from students.models import Student

admin.site.register(Student)
```

admin.py에 Student 등록

```
C:\Django\pjt\tempProject>python manage.py makemigrations
Migrations for 'students':
  students\migrations\0001_initial.py
    - Create model Student

C:\Django\pjt\tempProject>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, students
Running migrations:
  Applying students.0001_initial... OK

C:\Django\pjt\tempProject>
```

DB 변경사항 반영

4 테이블 생성

Member
id
pw
name
nickname
tel
zipcode
address1
address2
gender
hobby
createdate
update date

Freeboard
f_no
id
f_title
f_content
f_group
f_step
f_indent
f_hit
f_createdate
f_updatedate
f_file

select member.id,nickname from member,freeboard where member.id = freeboard.id

4 models.py – DB 데이터 Field Type

AutoField

#auto increment 되는 IntegerField
id = models.AutoField(primary_key=True)

BooleanField

기본으로 null=False라서 true/false 가짐
default값을 설정하지 않으면 None값이 나오기 때문에 주의

CharField

문자열을 저장하는 필드로 max_length로 최대 길이를 제한할수 있음.

DateField

datetime.date를 저장하는 필드
auto_now=True를 인자로 추가하면
Model.save로 저장할 때마다 현재 시간으로 갱신

DateTimeField : DateField 와 동일

자동 id

각 모델마다 pk 자동 생성. 이름은 id
pk가 설정되어 있으면 자동생성 되지 않음

id = models.AutoField(primary_key=True)

id 번호는 자동으로 등록되므로 출력가능함
{{ s.id }}
1,2,3,... 번호가 나타남.

board = Board.objects.get(pk=id)
path('<int:id>/view/',views.view,name='view')
qs = Student.objects.get(pk=id)

4 models.py – Field Type

IntegerField FloatField 실수 표현
32bit integer를 표현

TextField
CharField보다 큰 텍스트 설정
max_length 인자로 widget까지는 제어할 수 있음
model, database까지 제어되지는 않음.

ForeignKey

owner = models.ForeignKey(User, on_delete=models.CASCADE)

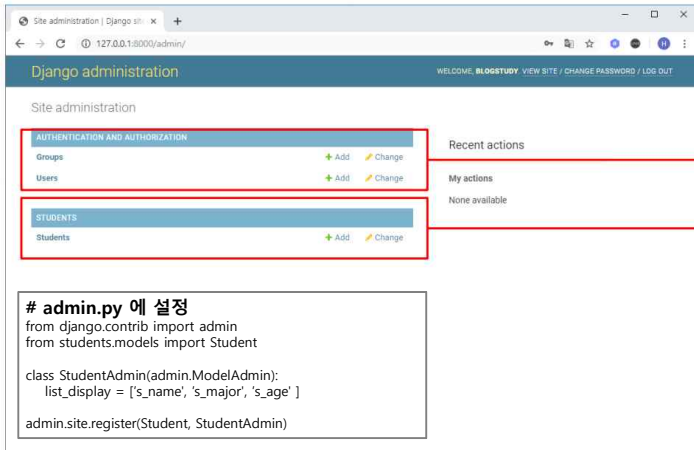
ON DELETE CASCADE는 ForeignKey를 포함한 객체를 삭제.

외래 키를 지정하는 것을 통해
다대일 관계를 정의.
1명의 user는 여러 개의 상품을 가질수 있음
(다대일 관계)
다대일 관계에서는 2개 위치인수 필요

예) 1개의 게시글에는 여러 개의 댓글 가능
- 1개의 게시글을 user가 삭제될때
해당되는 댓글을 모두 삭제

4

테이블 생성



The screenshot shows the Django administration interface. The top navigation bar includes the title 'Django administration' and links for 'WELCOME, BLOGSTUDY', 'VIEW SITE', 'CHANGE PASSWORD', and 'LOG OUT'. The main content area is titled 'Site administration' and contains two sections: 'AUTHENTICATION AND AUTHORIZATION' and 'STUDENTS'. The 'AUTHENTICATION AND AUTHORIZATION' section lists 'Groups' and 'Users', each with 'Add' and 'Change' links. The 'STUDENTS' section lists 'Students' with 'Add' and 'Change' links. To the right of these sections, there are links for 'Recent actions' and 'My actions', with the text 'None available' below them. Red boxes highlight the 'AUTHENTICATION AND AUTHORIZATION' and 'STUDENTS' sections. Red arrows point from these boxes to Korean text labels on the right: '기본 사용자 및 그룹 테이블' (Basic user and group table) and 'Student 테이블' (Student table).

```
# admin.py 에 설정
from django.contrib import admin
from students.models import Student

class StudentAdmin(admin.ModelAdmin):
    list_display = ['s_name', 's_major', 's_age']

admin.site.register(Student, StudentAdmin)
```

기본 사용자 및 그룹 테이블

Student 테이블

4 models - admin.py 설정

students > admin.py : 기본형태

```
from django.contrib import admin
from students.models import Student
```

```
admin.site.register(Student) # 기본 modelAdmin에 등록
```

students > admin.py : 여러컬럼 출력형태

```
from django.contrib import admin
from students.models import Student
```

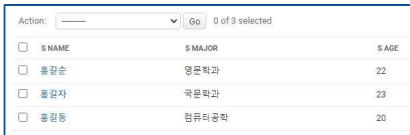
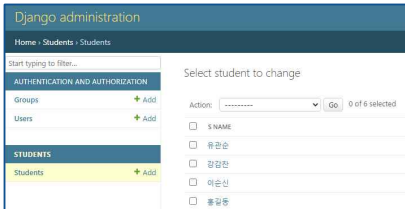
```
class StudentAdmin(admin.ModelAdmin):
    list_display = ['s_name', 's_major', 's_age']
```

```
admin.site.register(Student, StudentAdmin)
```

장식자(decorator) 형태로 등록이 가능

```
@ admin.register(Student)
```

```
class StudentAdmin(admin.ModelAdmin):
    list_display = ['s_name', 's_major', 's_age']
```



레코드 다루기

DB 명령어

4 레코드 다루기 - create, object.all()

장고 shell모드 실행 Python manage.py shell

```
C:\WDjango\ojit\tempProject>python manage.py shell
Python 3.7.2 (tags/v3.7.2:0a0030402, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
(InteractiveConsole)
>>>
```

레코드 추가(create) - 데이터 생성 Student.object.create(s_name='Hong',s_major='computer' ...)

```
>>> from students.models import Student
>>> qs = Student(s_name='HongGilDong', s_major='computer', s_age=21, s_grade=2, s_gender='M')
>>> qs.save()
>>>
```

S name:

S major:

S age:

S grade:

S gender:

pip install ipython
- In [3] : 형태로 나타남.

Shell 형태가 바뀜

```
In [3]: qs = Member.objects.all()

In [4]: print(qs)
<QuerySet [<Member: aaa>, <Member: bbb>, <Member: ccc>, <Member: ddd>]>
```

4 레코드 다루기

1. 조회

```
Student.objects.filter(s_age=20) # s_age가 20인 데이터
Student.objects.get(s_age=20)   # s_age가 20인 데이터
Student.objects.all()           # 전체 데이터
```

2. 생성

```
Student.objects.create(
    s_name=name, s_major=major, s_age=age, s_grade=grade, s_
    gender=gender, s_hobby=hobby)
```

```
qs =
Student(s_name=name, s_major=major, s_age=age, s_gender
=gender, s_hobby=hobby)
qs.save()
```

3. 수정

```
qs = Student.objects.get(s_no=s_no)
qs.s_major = major
qs.save()
```

```
Student.objects.filter(s_no=sno).update(
    s_major=major, s_name=name, s_age=age )
```

4. 삭제

```
Student.objects.get(s_no=s_no).delete() # 해당삭제
Student.objects.all().delete() # 전체삭제
# qs = Student.objects.get(s_no=s_no)
# qs.delete()
```

5. null 값 검색 objects.filter(bio__isnull=True)

```
Student.objects.filter(s_age__=True)
Student.objects.filter(s_age__=False).values('s_name','s_age')
```

6. exists() : 파일 있는지 확인

```
Student.objects.filter(s_age=20).exists()
False / True
```

4 레코드 다루기

6. __contains / icontains : 대소문자(X)

```
Student.objects.filter(s_name__contains= ' 홍')
```

7. __startswith , __endswith

```
Student.objects.filter(s_name__startswith= ' 홍')
```

8. 비교 gt,gte,lt,lte

gt - Greater than
gte - Greater than or equal to
lt - Less than
lte - Less than or equal to

9. 특정컬럼 선택 values

```
Student.objects.values('s_name','member__name')  
// membe객체 name을 출력 : _ _ 언더바 2개 하부컬럼
```

10. in : 2개 데이터 검색

```
Student.objects.filter(no__in=[1,2])
```

11. aggregate() 그룹함수 : 집계 연산 실행 - Sum(),Avg(),Max(),Min()

```
User.objects.aggregate(Sum('age')) # 따옴표 넣어야 함.  
{'age__sum': 75}  
User.objects.aggregate(Avg('age'))  
{'age__avg': 25.0}
```

```
# 최대값 구하기  
from django.db.models import Max  
smax = Student.objects.aggregate(id=Max('id'))  
no = Fboard.objects.aggregate(b_no=Max('b_no'))
```

12. annotate() : 컬럼별 합계

```
User.objects.values('city').annotate(Sum('age'))
```

4 레코드 다루기 - filter()

레코드 읽기(read) - 데이터 필터(filter)

```
>>> qs = Student.objects.filter(s_age__lt=22)
>>> qs
<QuerySet [<Student: HongGilDong>]>
>>> qs = Student.objects.filter(s_age__gt=22)
>>> qs
<QuerySet [<Student: HongGilJaSoon>]>
>>> qs = Student.objects.filter(s_age__lte=22)
>>> qs
<QuerySet [<Student: HongGilDong>, <Student: HongGilJa>]>
>>> qs = Student.objects.filter(s_age__gte=22)
>>> qs
<QuerySet [<Student: HongGilJa>, <Student: HongGilJaSoon>]>
>>>
```

__lt	~보다 작다
__lte	~보다 작거나 같다
__gt	~보다 크다
__gte	~보다 크거나 같다
__isnull	~ null인 자료 검색
__contains	특정 문자열을 포함하는 자료 검색
__startswith	특정 문자열로 시작하는 자료 검색
__endwith	특정 문자열로 끝나는 자료 검색

objects.get() : 데이터가 없을때 에러

- DoesNotExist 메시지 띄움.

objects.filter() : 데이터가 없을때 빈 쿼리셋

4 레코드 다루기

13. Order by

```
Student.objects.order_by('s_age')
Student.objects.order_by('-s_age')
```

```
Student.objects.all().order_by('age','-sno')
```

14. Q 객체는 **AND, OR 및 NOT 연산**에 사용

```
from django.db.models import F, Q, Sum, Count
```

```
User.objects.filter(Q(city='city1') & Q(age=25))
```

```
User.objects.filter(Q(city='city1') |
Q(age=25)).values('id','city','age')
```

```
User.objects.filter(Q(city='city1') & ~Q(age=20))
```

and로 만 검색이 됨. Or 검색이 안됨.

or 검색은 Q함수 사용

```
Student.objects.filter(name__contains='순', age=23)
```

15. F() Expressions

Django QuerySet API에서 F()

DB의 모델 필드 값을 직접 참조

```
from django.db.models import F, Q, Sum, Count
```

```
Fboard.objects.filter(b_group=b_group,b_step__gt=b_step)
.update(b_step=F('b_step')+1)
```

```
User.objects.annotate(new_age=F('age')*2).values('id','age',
'new_age')
```

```
User.objects.filter(id__lt=F('age')).values('id','age')
```

id, pw가 일치하지 않을때 예외 처리

try:

```
member = Member.objects.get(m_id=id,m_pw=pw)
```

except Member.DoesNotExist:

```
member = None
```

```
from django.db import models
```

models.py

```
class Tag(models.Model):
    name = models.CharField(max_length=30)
```

```
def __str__(self):
    return self.name
```

```
class Post(models.Model):
    title = models.CharField(max_length=100)
    tags = models.ManyToManyField(Tag)
```

```
def __str__(self):
    return self.title
```

Posts: + 추가

Tags: + 추가

인증 및 권한

그룹: + 추가

사용자(들): + 추가

역선: 실행 0개가 4개 중에 선택됨.

TITLE TAGS

☐ Title, tags 컬럼

☐ 웹개발도서 파이썬

☐ 파이썬활용인공지능도서 인공지능, 파이썬

☐ 디잡고도서 디잡고

☐ 파이썬기초도서 파이썬

4 posts

샘플 포스트 생성

```
post1 = Post.objects.create(title='Python Basics')
post2 = Post.objects.create(title='Django Tutorial')
post3 = Post.objects.create(title='AI with Python')
post4 = Post.objects.create(title='Web Development')
```

포스트에 태그 추가 (다대다 관계 설정)

```
post1.tags.add(python)
post2.tags.add(django)
post3.tags.add(ai, python)
post4.tags.add(django, python)
```

Posts: + 추가

Tags: + 추가

인증 및 권한

그룹: + 추가

사용자(들): + 추가

변경할 post 선택

역선: 실행 0개가 4개 중에 선택됨.

TITLE TAGS

☐ Name컬럼

☐ 웹개발도서 디잡고, 파이썬

☐ 파이썬활용인공지능도서

☐ 디잡고도서

☐ 파이썬기초도서

4 posts

샘플 태그 생성

```
python = Tag.objects.create(name='Python')
django = Tag.objects.create(name='Django')
ai = Tag.objects.create(name='AI')
```

admin.py

```
from django.contrib import admin
from .models import Tag, Post
```

```
class PostAdmin(admin.ModelAdmin):
    list_display = ('title', 'list_tags')
    filter_horizontal = ('tags',)
```

```
def list_tags(self, obj):
    return ", ".join(tag.name for tag in obj.tags.all())
list_tags.short_description = 'Tags'
```

관리자 등록

```
admin.site.register(Post, PostAdmin)
admin.site.register(Tag)
```

post 추가

Title:

Tags:

이용 가능한 tags

Choose tags by selecting them and then select the "Choose" arrow button.

Q: 필터

Python
Django
AI

Choose all tags

선택된 tags

Remove tags by selecting them and then select the "Remove" arrow button.

Q: 필터

Remove all tags

하나 이상을 선택하려면 "Control" 키를 누른 채로 선택해주세요. Mac의 경우에는 "Command" 키를 눌러주세요.

저장 저장 및 다른 이름으로 추가 저장 및 편집 계속

4

다대다 설정 - 검색쿼리

1. 'Python' 태그가 있는 포스트 검색

```
Post.objects.filter(tags__name='Python')
```

검색쿼리

Python Basics
AI with Python
Web Development

검색결과

2. 'Python'과 'Django' 태그 둘 다 있는 포스트 검색 (AND 조건)

```
Post.objects.filter(tags__name='Python').filter(tags__name='Django').distinct()
```

검색쿼리

Web Development

검색결과

3. 'Python' 또는 'Django' 태그 하나라도 있는 포스트 검색 (OR 조건)

```
from django.db.models import Q

Post.objects.filter(
    Q(tags__name='Python') | Q(tags__name='Django')
).distinct()
```

검색쿼리

Python Basics
Django Tutorial
AI with Python
Web Development

검색결과

4. 특정 태그가 달린 포스트 (역방향 검색)

```
tag = Tag.objects.get(name='AI')
tag.post_set.all()
```

검색쿼리

AI with Python

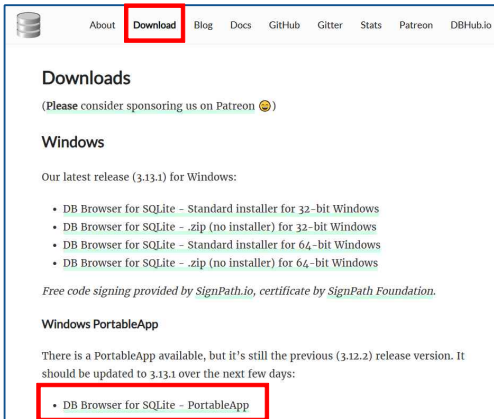
검색결과

SQLite Browser

(프로그램)

DB Browser for SQLite 다운로드

<https://sqlitebrowser.org/dl/>



The screenshot shows the 'Downloads' section of the DB Browser for SQLite website. The 'Download' link in the top navigation bar is highlighted with a red box. The page content includes a 'Downloads' heading, a note about sponsoring on Patreon, a 'Windows' section with a list of download links for 32-bit and 64-bit Windows, a note about code signing, a 'Windows PortableApp' section, and a list of download links for the PortableApp. The 'PortableApp' link in the bottom list is also highlighted with a red box.

 [About](#) **[Download](#)** [Blog](#) [Docs](#) [GitHub](#) [Gitter](#) [Stats](#) [Patreon](#) [DBHub.io](#)

Downloads

(Please consider sponsoring us on Patreon 😊)

Windows

Our latest release (3.13.1) for Windows:

- [DB Browser for SQLite - Standard installer for 32-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 32-bit Windows](#)
- [DB Browser for SQLite - Standard installer for 64-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 64-bit Windows](#)

Free code signing provided by [SignPath.io](#), certificate by [SignPath Foundation](#).

Windows PortableApp

There is a PortableApp available, but it's still the previous (3.12.2) release version. It should be updated to 3.13.1 over the next few days:

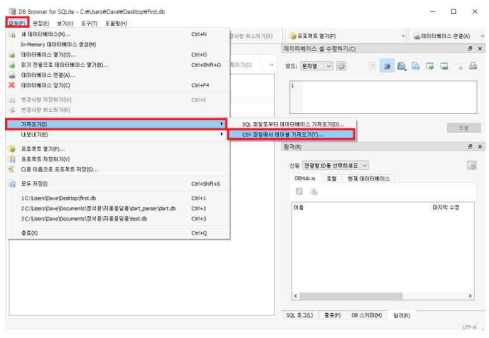
- [DB Browser for SQLite - PortableApp](#)

CSV 파일 불러오기

1. 프로그램 클릭

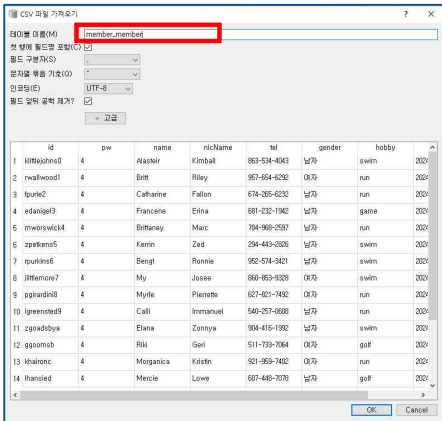


2. csv 파일 불러오기

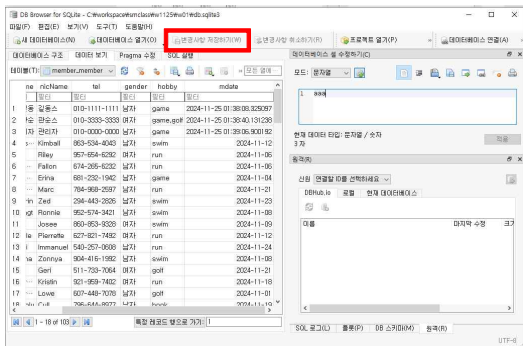


csv 파일 불러오기

3. 테이블 이름은 table명과 동일해야 함.



4. 변경사항 저장하기 클릭



migrate 되돌리기

1. python manage.py showmigrations

- 현재 migrations이 어떻게 진행되었는지 보여줌

```
home
(no migrations)
sessions
[X] 0001_initial
students
[X] 0001_initial
stuscore
[X] 0001_initial
[ ] 0002_rename_name_stuscore_name1
```

2. python manage.py migrate stuscore 0001_initial

- 특정 번호로 migration을 roll back해줌

```
PS C:\workspace\django\w0526\w05> python manage.py migrate stuscore
0001_initial
Operations to perform:
  Target specific migration: 0001_initial, from stuscore
Running migrations:
  No migrations to apply.
```

3. roll back을 한후 , 데이터 수정을 함.

- 다시 변경하고 하는 데이터를 찾아, 수정을 함.

예) 컬럼이름 변경, 컬럼타입 변경 등

4. python manage.py migrate

- 변경을 했다고 하면 지정된 형태 저장

migrate 에러 메시지

1. 필드추가 시 에러 메시지 - 다른 필드데이터 처리를 어떻게 할지에 대한 형태

It is impossible to add a non-nullable field 'tel' to students without specifying a default. This is because the database needs something to populate existing rows.

Please select a fix:

- 1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
- 2) Quit and manually define a default value in models.py.

1 번 선택 : 빈공백 처리를 어떤 컬럼으로 처리를 할지를 선택하면
그 값으로 처리를 해주겠다는 것임. 번호를 입력 1

2 번 선택 : models.py 에서 수정처리
공백처리 null처리, default 처리를 해야 함.
name = models.CharField(null=True, default= '')

Html Template Language

html사용

1. templates 설정

w1128\w01
 > board
 > comment
 > home
 > media
 > member
 > static
 > templates
 <> subbase.html
 w01
 > __pycache__
 __init__.py
 asgi.py
 settings.py
 urls.py

* 메인과 서브페이지 차이가 있으면, 별도로 구분

* 전체 templates 설정, 각 app별 templates 존재

템플릿 경로

TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')

base.html

```

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [TEMPLATE_DIR],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
  
```

5 Template Language

- Template 태그 : {% 문법 %} – for,if문,함수
- Template 변수 : {{ 변수 }}
- Template 필터 : {{ 변수|옵션 }}

5 Template Language - Template 태그 : {% 문법 %}

- `{{forloop.counter}}` : 1부터 반복횟수만큼 count
- `{{forloop.counter0}}` : 0부터 반복횟수만큼 count
- `{{forloop.revcounter}}` : 전체 길이로 시작해서 1까지 count
- `{{forloop.revcounter0}}` : 전체 길이로 시작해서 0까지 count

```
<body>
```

html

```
<h1>Template Language!</h1>
```

```
<p>{{value}}</p>
```

```
{% for i in value %}
```

```
<div>번호 : {{forloop.counter}} 번째  
data</div>
```

```
<div>제목 : {{i.title}}</div>
```

```
<div>추천수 : {{i.likeCount}}</div>
```

```
<div>조회수 : {{i.viewCount}}</div>
```

```
<div>내용 : {{i.contents}}</div>
```

```
<br />
```

```
{% endfor %}
```

```
</body>
```

Template Language!

<QuerySet [<Notice: test1>, <Notice: test2>, <Notice: test3>, <Notice: test4>, <Notice: tes

1 번째 data

제목 : test1

추천수 : 11

조회수 : 14

내용 : test1 내용입니다.

2 번째 data

제목 : test2

추천수 : 21

조회수 : 54

내용 : test2 내용입니다.

3 번째 data

제목 : test3

추천수 : 3

조회수 : 31

내용 : test3 내용입니다.

5 Template Language - Template 태그 : {% 문법 %}

- html 상에서 프로그래밍 로직을 이용할 수 있는 태그
- load static, for, if 등
- and, or, not, and not, > >=, < <= ,in, not in 등의 비교연산이 가능
- {% csrf_token %} : CSRF 공격 방지를 위한 태그
장고 내부적으로 CSRF 토큰값의 유효성을 검증
form태그의 첫 줄에 입력
- block content, extends
템플릿 확장에 활용하는 태그

5 Template Language - Template 태그 : {% 문법 %}

```
{% for i in value %} .. {% endfor %}  
{% if %} .. {% elif %} .. {% else %} .. {% endif %}
```

```
{%load static%}  
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <title>Document</title>  
  </head>  
  <body>  
    <h1>Template Language!</h1>  
    <p>{{value}}</p>  
    {% for i in value %}  
    <div>{{forloop.counter}} 번째 data</div>  
    <div>제목 : {{i.title}}</div>  
    <div>추천수 : {{i.likeCount}}</div>  
    <div>조회수 : {{i.viewCount}}</div>  
    <div>내용 : {{i.contents}}</div>
```

html

```
{% if i.viewCount > 10 %}  
<p>조회수가 10이상 입니다.</p>  
{% elif i.viewCount > 5 %}  
<p>조회수가 5이상 10미만 입니다.</p>  
{% else %}  
<p>조회수가 5미만 입니다.</p>  
{% endif %}  
<br />  
{% endfor %}  
</body>  
</html>
```

5 Template Language – for문

For문 선언 : range(10) html에서는 사용이 안됨.

```
def my_view(request):  
    return render(request, "test.html", {"nums": range(1, 11)})
```

Views.py

```
{% for n in nums %}  
    <p>{{ n }}번 반복</p>  
{% endfor %}
```

html

5 Template Language - Template 태그 : {% 문법 %}

변수선언 : {% with 변수명 = '값' %}

```
{%load static%}
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Document</title>
  </head>
  <body>
    <h1>Template Language!</h1>
    {% with value='hello Django'%} 👉 "hello Django"를 value라는 변수에 담았어요:)
    <h4>{{value}}</h4>
    {% endwith %}
  </body>
</html>
```

html

5 Template 변수: {{ 변수 }}

- view.py에서 변수에 데이터를 넣어 템플릿으로 전달
- html 파일 안에서 해당 변수 사용 (dict형, list형, 단일변수 모두 전달 가능)

```
<body>
<h1>Template Language!</h1>
<div>I 출력 : {{l}}</div>
<div>I의 0번째 값은 : {{l.0}}</div> # 리스트타입 출력
<div>I의 1번째 값은 : {{l.1}}</div>
<div>I의 2번째 값은 : {{l.2}}</div>
<p>
<div>d 출력 : {{d}}</div>
<div>d의 key(name) 값은 : {{d.name}}</div> # dic타입 출력
<div>d의 key(age) 값은 : {{d.age}}</div>
<div>d의 key(location) 값은 : {{d.location}}</div>
<div>d의 key(isman) 값은 : {{d.isman}}</div>
</p>
</body>
```

html

```
# views.py
from django.shortcuts import render, redirect
from django.http import HttpResponse,
JsonResponse
def sample(request):
    d = {'name':'jaewon', 'age':99, 'location':'seoul',
        'isman':True} # dic타입
    l = [100,200,300] # 리스트타입
    return render(request, 'sample.html', {'d':d, 'l':l})
```

views.py

5 Template 필터 : {{ 변수|옵션 }}

- 템플릿 변수의 값을 변환시키기 위한 옵션으로 사용 (템플릿 변수에 파이프(|)를 붙인 뒤 사용)
- 주로 문자열이나 리스트 등의 길이구하기, 슬라이싱, 합치기 등에 활용
- 문자포함여부확인 : {% if 템플릿변수 | contains:"game" %}
- 길이 구하기 : {{ 템플릿변수 | length }}
- 리스트를 문자열로 합치기 : {{ 변수|join:"옵션" }}
- 소문자 & 대문자로 변환 : {{ 변수|lower }} / {{ 변수|upper }}
- 리스트에서 random하게 추출 : {{ 리스트형 변수|random }}
- 문자열로 슬라이싱 : {{ 변수|slice:"n:n" }}
- Django Templates에서 lorem 사용하는 방법은 아래와 같음
{{ lorem [count][method][random] }}

5 Template 필터 : {{ 변수|옵션 }}

html

```
{%load static%}
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Document</title>
  </head>
  <body>
    <h1>Template Language!</h1>
    {% with value='hello Django'%}
    <h4>{{value}}</h4>
    {% endwith %}
    {% lorem %}    <!-- 📖 lorem 생성 -->
        {% lorem 3 p%} <!-- 📖 lorem 문장(p)을 3개 생성 -->
    {% lorem 2 w random %} {% lorem 5 w random %} <!-- 📖 lorem 단어(w)을 5개 랜덤 생성 -->
  </body>
</html>
```

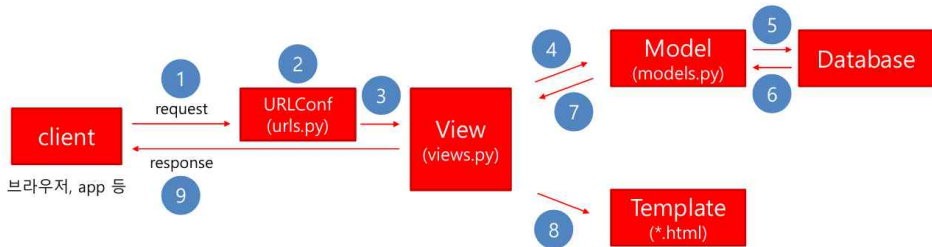
학생관리 페이지

프로젝트 진행

6

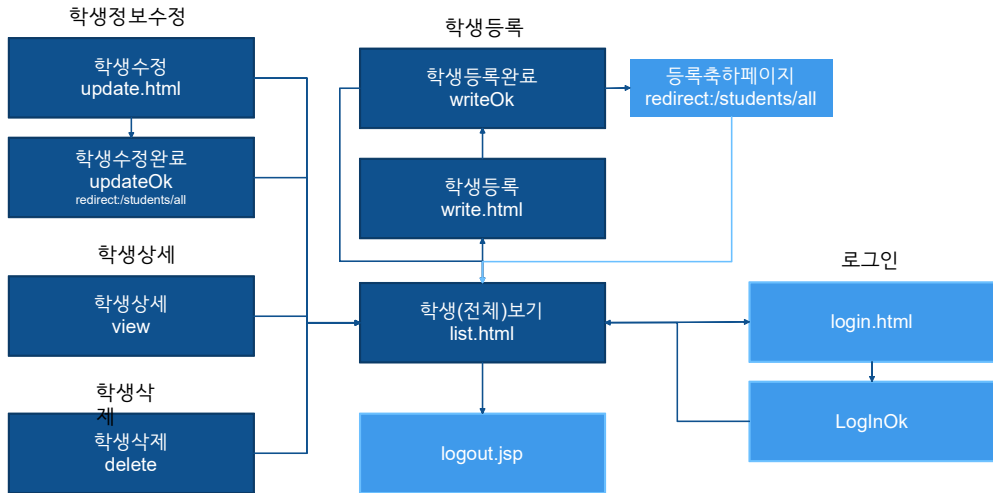
웹 애플리케이션 설계

Django framework flow



6

웹 애플리케이션 설계



6

프로젝트 세팅

생성

Project 생성

```
django-admin startproject spjt
```

App 생성

```
python manage.py startapp students
```

project 세팅

setting.py

```
students.apps.StudentsConfig
```

urls.py

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path(' ', include('home.urls')),
    path('students/', include('students.urls')),
]
```

app 세팅

Students urls.py

```
from . import views
app_name = 'students'
urlpatterns = [
    path('reg/', views.regStudent, name='reg'),
]
```

Home urls.py

```
from . import views
app_name = ''
urlpatterns = [
    path("", views.index, name="index"),
]
```

views.py

```
def regStudent(request):
    return render(request, 'students/registerStudent.html')
```

db 계정 생성

```
python manage.py migrate
python manage.py makemigrations : db반영할 내용 확인
```

관리자 계정 생성

```
python manage.py createsuperuser
```

서버 구동

```
python -u manage.py runserver 0.0.0.0:8000
```

import os

setting.py

```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'students.apps.StudentsConfig',
    'students',
    'home',
]

```

한글,한국시간 설정

LANGUAGE_CODE = 'ko-kr'

TIME_ZONE = 'Asia/Seoul'

STATIC_URL = 'static/'

static파일 위치 설정 - 현재root 모든 static폴더를 가져옴

정적파일 : css,js,image

```

STATICFILES_DIRS = (
    os.path.join(BASE_DIR,'static'),
)

```

* fullName을 다 적거나
app_name을 적어도 됨.

urls.py

from django.urls import path,include

urlpatterns = [

path('admin/', admin.site.urls),

path(' ',include('students.urls')),

path('students/',include('students.urls')),

]

6 데이터베이스 만들기(models.py)

models.py

```
from django.db import models

class Student(models.Model):
    # 번호가 순차적으로 증가
    no = models.AutoField(primary_key=True) # sequence - 자동증가
    name = models.CharField(max_length=100) # varchar2
    major = models.CharField(max_length=100)
    # 숫자타입
    grade = models.IntegerField(default=0) # number
    age = models.IntegerField(default=0)
    gender = models.CharField(max_length=30, blank=True) # 추가-100개
    hobby = models.CharField(max_length=100, blank=True)
    sdate = models.DateTimeField(auto_now=True) # date - sysdate
    memo = models.TextField(blank=True) # clob

    # 관리자페이지
    def __str__(self):
        return f"{self.no},{self.name},{self.sdate}"
```

```
s_name : 이름(char)
s_major : 전공(char)
s_age : 나이(int)
s_grade : 학년(int)
s_gender : 성구분(char)
```

admin.py

```
from django.contrib import admin
from students.models import Student
```

관리자페이지에서 컬럼부분 추가해서 보여줌.

```
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'major', 'age']
```

어드민 관리자 페이지 등록

```
admin.site.register(Student, StudentAdmin)
# admin.site.register(Students)
```

날짜 자동입력

```
# from datetime import datetime
models.DateTimeField(default=datetime.now(),
blank=True)
```

```
from django.contrib import admin
from board.models import Board
from django import forms
```

```
class BoardForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Board
```

```
        fields = '__all__'
```

```
        widgets = {
```

```
            'btitle': forms.TextInput(attrs={'placeholder': '제목을 입력하세
```

```
요.'}),
        }
```

```
# 관리자페이지에서 컬럼부분 추가해서 보여줌.
```

```
class BoardAdmin(admin.ModelAdmin):
```

```
    form = BoardForm
```

```
    list_display = ['id', 'btitle', 'bdate']
```

```
admin.site.register(Board, BoardAdmin)
```

Admin.py

The screenshot shows the Django Admin interface for adding a new board. On the left is a sidebar with navigation links: 'BOARD', 'Boards' (with a '+ 추가' button), '인원 및 권한', '그룹' (with a '+ 추가' button), and '사용자(들)' (with a '+ 추가' button). The main area is titled 'board 추가' and contains a form with fields: 'Id:' (with a text input), 'Btitle:' (with a text input containing the placeholder '제목을 입력하세요.'), and 'Bcontent:' (with a text area). A red box highlights the 'Btitle' field's placeholder, and a red line points from it to a red text box at the bottom.

* 입력창에 placeholder 를 넣을수 있음.

6 데이터베이스 만들기(models.py)

models.py

```
from django.db import models

class Student(models.Model):
    # 번호가 순차적으로 증가
    no = models.AutoField(primary_key=True) # sequence - 자동증가
    name = models.CharField(max_length=100) # varchar2
    major = models.CharField(max_length=100)
    # 숫자타입
    grade = models.IntegerField(default=0) # number
    age = models.IntegerField(default=0)
    gender = models.CharField(max_length=30, blank=True) # 추가-100개
    hobby = models.CharField(max_length=100, blank=True)
    sdate = models.DateTimeField(auto_now=True) # date - sysdate
    memo = models.TextField(blank=True) # clob

    # 관리자페이지
    def __str__(self):
        return f"{self.no},{self.name},{self.sdate}"
```

```
s_name : 이름(char)
s_major : 전공(char)
s_age : 나이(int)
s_grade : 학년(int)
s_gender : 성구분(char)
```

admin.py

```
from django.contrib import admin
from students.models import Student
```

관리자페이지에서 컬럼부분 추가해서 보여줌.

```
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'major', 'age']
```

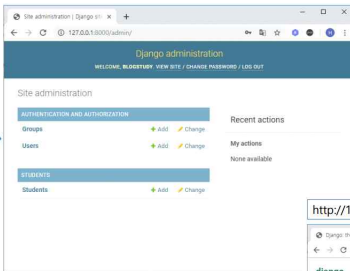
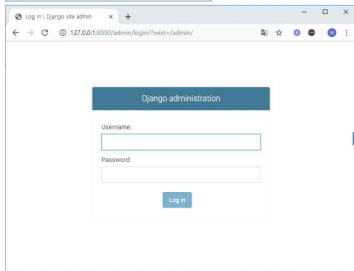
어드민 관리자 페이지 등록

```
admin.site.register(Student, StudentAdmin)
# admin.site.register(Students)
```

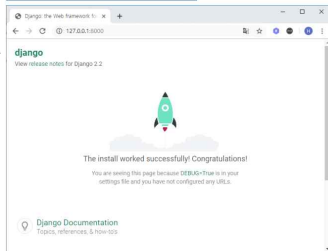
날짜 자동입력

```
# from datetime import datetime
models.DateTimeField(default=datetime.now(),
blank=True)
```

http://127.0.0.1:8000/admin/



http://127.0.0.1:8000



6 학생정보리스트, 학생정보등록 페이지

학생정보리스트

번호	이름	학과	날짜
1	홍길동2	컴퓨터공학과2	2025년 5월 29일 12:55 오후
2	유관순	국문학과	2025년 5월 29일 11:04 오전
3	이순신	역사학과	2025년 5월 29일 11:10 오전
5	김구	역사학과	2025년 5월 29일 11:10 오전
6	김유신	기계공학과	2025년 5월 29일 12:00 오후

홈으로

학생등록

학생정보등록

이름

학과

학년

나이

성별

☐ 남자 ☐ 여자

취미

☐ 게임 ☐ 골프 ☐ 수영 ☐ 조깅 ☐ 독서

저장

메인이동

6 학생정보 상세보기, 학생정보수정

학생정보 상세보기

번호	1
이름	홍길동2
학과	컴퓨터공학과2
학년	12
나이	202
성별	남자
취미	수영,조깅
메모	학생을 등록합니다.

학생리스트

수정

삭제

order_by(' ') 오름차순

order_by('- ') 내림차순

학생정보수정

번호 1

이름 홍길동2

학과 컴퓨터공학과2

학년 12

나이 202

성별

☒ 남자 ☐ 여자

취미

☐ 게임 ☐ 골프 ☒ 수영 ☒ 조깅 ☐ 독서

수정

취소

__lt ~보다 작다

__lte ~보다 작거나 같다

__gt ~보다 크다

__gte ~보다 크거나 같다

__isnull ~ null인 자료 검색

__contains 특정 문자열을 포함하는 자료 검색

__startswith 특정 문자열로 시작하는 자료 검색

__endwith 특정 문자열로 끝나는 자료 검색

6 학생 등록 페이지 - students.write

프로젝트 파일 > students > urls.py
프로젝트 파일 > students > views.py

```
from django.urls import path, include
from . import views
```

urls.py

```
# app_name는 / 붙이지 않음
```

```
app_name = 'students'
```

```
urlpatterns = [
```

```
    # path / 붙임.
```

```
    path('list/', views.list, name='list'),
```

```
    path('write/', views.write, name='write'),
```

```
    path('view/<int:no>/', views.view, name='view'),
```

```
    path('update/<int:no>/', views.update, name='update'),
```

```
    path('delete/<int:no>/', views.delete, name='delete'),
```

```
    # 학생정보삭제
```

```
]
```

```
from django.shortcuts import render, redirect
from students.models import Student
```

views.py

```
# 학생정보저장
```

```
def write(request):
```

```
    if request.method == 'GET': # 학생정보등록페이지 열기
```

```
        return render(request, 'students/write.html')
```

```
    elif request.method == 'POST': # 학생정보저장
```

```
        name = request.POST.get('name')
```

```
        major = request.POST.get('major')
```

```
        grade = request.POST.get('grade')
```

```
        age = request.POST.get('age')
```

```
        gender = request.POST.get('gender')
```

```
        # 취미 - 리스트
```

```
        hobby = request.POST.getlist('hobby') # 자동으로 형변환 됨.
```

```
        # 리스트타입 -> str타입으로 변경
```

```
        hobby = ','.join(hobby) # 'game,golf,swim'
```

```
        print("저장정보 이름 : ", name)
```

```
        print("저장정보 학과 : ", major)
```

```
        print("저장정보 hobby : ", hobby) # ['game', 'golf', 'swim']
```

```
        Student(name=name, major=major, grade=grade, age=age, gender=gender, hobby=hobby, memo='등록합니다.').save()
```

```
        return redirect('/students/list/')
```

6 학생 등록 페이지 - write.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h2>학생정보등록</h2>
  <form action="/students/write/" method="post" name="frm">
    {% csrf_token %}
    <label>이름</label>
    <input type="text" name="name"><br>
    <label>학과</label>
    <input type="text" name="major"><br>
    <label>학년</label>
    <input type="text" name="grade"><br>
    <label>나이</label>
    <input type="text" name="age"><br>
```

Write.html

등록 버튼 클릭시 action 페이지 이동
csrf_token : form 데이터를 암호화해서 전송

```
<label>성별</label><br>
<input type="radio" name="gender" value="남자" id="man">
<label for="man">남자</label>
<input type="radio" name="gender" value="여자" id="woman">
<label for="woman">여자</label>
<br>
<label>취미</label><br>
<input type="checkbox" name="hobby" value="게임" id="game">
<label for="game">게임</label>
<input type="checkbox" name="hobby" value="골프" id="golf">
<label for="golf">골프</label>
<input type="checkbox" name="hobby" value="수영" id="swim">
<label for="swim">수영</label>
<input type="checkbox" name="hobby" value="조깅" id="run">
<label for="run">조깅</label>
<input type="checkbox" name="hobby" value="독서" id="book">
<label for="book">독서</label>
<br>
```

```
<input type="submit" value="저장">
<!-- button특성 : type지정하지 않으면 form action주소를 호출
```

-->

```
<a href="/students/list/"><button type="button">메인이동
</button></a>
</form>
</body>
```

6 학생 리스트 페이지 – students:list

```
from django.urls import path,include
from . import views
```

urls.py

```
# app_name는 / 붙이지 않음
app_name = 'students'
urlpatterns = [
    # path / 붙임.
```

```
    path('list/', views.list, name='list'),
    path('write/', views.write, name='write'),
```

```
    path('view/<int:no>/', views.view, name='view'),
    path('update/<int:no>/', views.update, name='update'),
```

```
    path('delete/<int:no>/', views.delete, name='delete'),
    # 학생정보삭제
```

```
]
```

학생정보리스트

```
def list(request):
```

```
    qs = Student.objects.all()
```

```
    context = {'list':qs}
```

```
    # render 폴더앞에 / 안붙임.
```

```
    return render(request,'students/list.html',context)
```

views.py

리스트화면

학생정보리스트

번호	이름	학과	날짜
1	홍길동2	컴퓨터공학과2	2025년 5월 29일 12:55 오후
2	유관순	국문학과	2025년 5월 29일 11:04 오전
3	이순신	역사학과	2025년 5월 29일 11:10 오전
5	김구	역사학과	2025년 5월 29일 11:10 오전
6	김유신	기계공학과	2025년 5월 29일 12:00 오후

홈으로

학생등록

6 학생 리스트 페이지 - list.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <link rel='stylesheet' href='{% static "css/style.css" %}'>
  <style>

  </style>
</head>
<body>
  <h2>학생정보리스트</h2>
  <table>
    <tr>
      <th>번호</th>
      <th>이름</th>
      <th>학과</th>
      <th>날짜</th>
    </tr>
```

list.html

```
{# forloop.counter #}
{% for s in list %}
<tr>
  <td>{{s.no}}</td>
  <td>
    <a href='{% url "student:view" s.sno %}'> {{s.name}} </a>
    <a href='/students/view/{{s.no}}/'>{{s.name}}</a>
  </td>
  <td>{{s.major}}</td>
  <td>{{s.sdate}}</td>
</tr>
{% endfor %}
</table>
<br>
<div>
  <a href='/><button>홈</button></a>
  <a href='/students/write/'><button>학생등록</button></a>
</div>
</body>
</html>
```

{% if student_list %}

{% endif %}

**For문 앞에 데이터 있는지
확인**

삭제 적용 : 자바스크립트(제이쿼리) 구현

```
{% if student_list %}
```

```
<ol>
```

```
{% for s in student_list %}
```

```
<li>
```

```
<a href="{% url 'students:stuDet' s.s_name %}" >{{s.s_name}}</a>
```

```
<a href="{% url 'students:stuMod' s.s_name %}" >수정</a>
```

```
{# <a href="{% url 'students:stuDel' s.s_name %}" >삭제</a> #}
```

```
<a href="#" class="delete" data-uri="{% url 'students:stuDel' s.s_name %}" >삭제</a>
```

```
</li>
```

```
{% endfor %}
```

```
</ol>
```

```
{% else %}
```

```
<p>Student : 0</p>
```

```
{% endif %}
```

html

자바스크립트

```
<script src="http://code.jquery.com/jquery-latest.min.js">
```

```
</script>
```

```
<script>
```

```
$(function(){
```

```
$(".delete").on('click', function() {
```

```
if(confirm("정말로 삭제하시겠습니까?")) {
```

```
location.href = $(this).data('uri');
```

```
// location.href = "../" + name + "/stuDel/";
```

```
}
```

```
});
```

```
});
```

```
</script>
```

저장 2가지 방법

1. Create 명령어

```
Student.objects.create(s_name=name,s_major=major,s_age=age,s_grade=grade,s_gender=gender )
```

2. save() 명령어

```
qs = Student(s_name=name,s_major=major,s_age=age,s_grade=grade,s_gender=gender)  
qs.save()
```

글을 생성하고, 방금 생성한 글을 stu 변수에 저장한다.

```
stu = qs.save()
```

데이터 베이스 트랜잭션

```
from django.db import transaction

with transaction.atomic():
    qs = Student.objects.get(s_no=s_no)
    qs.s_major = major
    qs.s_name = name
    qs.s_age = age
    qs.save()
```

- 에러시 자동 롤백기능

[참조] 페이지 이동 함수

```
return redirect('index')          ### url 입력
# return redirect(reverse('index'))  ### app_name
# return HttpResponseRedirect(reverse('index'))
```

1. HttpResponseRedirect : HttpResponseRedirect(data, content_type)
- Json,html까지도 보낼수 있음

2. HttpResponseRedirect : HttpResponseRedirect(url)
- 지정된 url페이지로 redirect.

3. render : render(request, template_name, context=None, content_type=None, status=None, using=None)
- context값을 함께 넘겨줄 수 있음
예) return render(request, 'index.html',context)

4. redirect : redirect(to, permanent=False, *args, **kwargs)
링크만 넘겨줌.(redirect는 url로 이동)
예) return redirect(reverse('board:read',args=(id,))) ## 튜플 1개 일때 뒤에 쉼표 넣어야 함.
return redirect(reverse('students:view',args=(name,)))
return redirect('students:view',name)#내부적으로 resolve_url() 사용
return redirect(f'/students/{name}/view')

return redirect('/blog/detail/1/') #상대 url
return redirect('http://example.com/blog/detail/1') #절대 url

6 학생 정보 보기 페이지 – students:view

```
from django.urls import path,include
from . import views
```

urls.py

```
# app_name는 / 붙이지 않음
app_name = 'students'
```

```
urlpatterns = [
    # path / 붙임.
    path('list/', views.list, name='list'),
    path('write/', views.write, name='write'),
```

```
    path('view/<int:no>/', views.view, name='view'),
    path('update/<int:no>/', views.update, name='update'),
    #학생정보수정페이지 열기
```

```
    path('delete/<int:no>/', views.delete, name='delete'),
    # 학생정보삭제
```

```
]
```

html 파일에서 데이터를 전달 받음

```
<str : name >   # 문자일때
<int : b_no>    # 숫자일때
```

```
# 학생정보 상세보기
def view(request,no):
```

```
    try:
```

```
        qs = Student.objects.get(no=no)
```

```
    except:
```

```
        qs = None
```

```
    print('전달 no :',no)
```

```
    context = {'stu':qs}
```

```
    return render(request,'students/view.html',context)
```

views.py

학생정보 상세보기

번호	1
이름	홍길동2
학과	컴퓨터공학과2
학년	12
나이	202
성별	남자
취미	수영,조깅
메모	학생을 등록합니다.

상세보기화면

학생리스트

수정

삭제

6 학생 정보 보기 페이지 - view.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    h2{text-align:center;}
    table,th,td{border:1px solid black; border-collapse:collapse;}
    table{width:800px; margin:20px auto;}
    th,td{height:40px; text-align:center;}
    div{width:500px; margin:0 auto;}
    button{width:150px; height:50px; font-size:20px;}
  </style>
</head>
<body>
  <h2>학생 정보 상세 보기</h2>
  <table>
    {% if stu != None %}
    <tr>
      <th>번호</th>
      <td>{{stu.no}}</td>
    </tr>
    <tr>
      <th>이름</th>
      <td>{{stu.name}}</td>
    </tr>
    <tr>
      <th>학과</th>
      <td>{{stu.major}}</td>
    </tr>
    <tr>
      <th>학년</th>
      <td>{{stu.grade}}</td>
    </tr>
```

list.html

```
<tr>
  <th>나이</th>
  <td>{{stu.age}}</td>
</tr>
<tr>
  <th>성별</th>
  <td>{{stu.gender}}</td>
</tr>
<tr>
  <th>취미</th>
  <td>{{stu.hobby}}</td>
</tr>
<tr>
  <th>메모</th>
  <td>{{stu.memo}}</td>
</tr>
{% else %}
  <tr>
    <td>찾고자 하는 데이터가 없습니다. </td>
  </tr>
{% endif %}
</table>
<br>
<div>
  <a href="/students/list/"><button>학생 리스트</button></a>
  <a href="/students/update/{{stu.no}}/"><button>수정</button></a>
  <button onclick="deleteBtn()">삭제</button>
</div>
<script>
  function deleteBtn(){
    if(confirm('{{stu.name}} 학생을 삭제하시겠습니까?')){
      location.href="/students/delete/{{stu.no}}/";
    }
  }
</script>
</body>
</html>
```

6 학생 정보 수정 페이지 : students:update

```
from django.urls import path,include
from . import views
```

urls.py

```
# app_name는 / 붙이지 않음
app_name = 'students'
urlpatterns = [
    # path / 붙임.
    path('list/', views.list, name='list'),
    path('write/', views.write, name='write'),

    path('view/<int:no>/', views.view, name='view'),
    path('update/<int:no>/', views.update, name='update'),

    path('delete/<int:no>/', views.delete, name='delete'),
    # 학생정보삭제
]
```

학생정보수정페이지 열기

```
def update(request,no):
```

```
    if request.method == 'GET':
```

```
        qs = Student.objects.get(no=no) # set타입 1개
        context = {'stu':qs}
        # qs = Student.objects.filter(no=no) # 데이터 타입 - 리스트타입
        # context = {'stu':qs[0]}
        return render(request,'students/update.html',context)
```

```
    elif request.method == 'POST': # 학생정보수정완료
```

```
        no = request.POST.get('no')
        qs = Student.objects.get(no=no) # 데이터 검색
        # 데이터 수정
        qs.name = request.POST.get('name')
        qs.major = request.POST.get('major')
        qs.grade = request.POST.get('grade')
        qs.age = request.POST.get('age')
        qs.gender = request.POST.get('gender')
        hobby = request.POST.getlist('hobby')
        hobby = ','.join(hobby)
        qs.hobby = hobby
        qs.save()
```

```
    return redirect(f'/students/view/{no}/')
```

views.py

6 학생 정보 수정 페이지 : update.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h2>학생 정보 수정</h2>
  <form action="/students/updateOk/" method="post" name="frm">
    {% csrf_token %}
    <input type="hidden" name="no" value="{{stu.no}}">
    <label>번호</label>
    {{stu.no}}<br>
    <label>이름</label>
    <input type="text" name="name" value="{{stu.name}}"><br>
    <label>학과</label>
    <input type="text" name="major" value="{{stu.major}}"><br>
    <label>학년</label>
    <input type="text" name="grade" value="{{stu.grade}}"><br>
    <label>나이</label>
    <input type="text" name="age" value="{{stu.age}}"><br>
```

update.html

```
<input type="checkbox"
name="hobby" value="게임" id="game"
{%if '게임' in stu.hobby%}checked{%endif%}>
```

```
<label>성별</label><br>
<input type="radio" name="gender" value='남자' id='man' {%if '남자' in
stu.gender%}checked{%endif%}>
<label for='man'>남자</label>
<input type="radio" name="gender" value='여자' id='woman' {%if '여자' in
stu.gender%}checked{%endif%}>
<label for='woman'>여자</label>
<br>
```

```
<label>취미</label><br>
<input type="checkbox" name="hobby" value='게임' id='game' {%if '게임' in stu.hobby%}checked{%endif%}>
<label for='game'>게임</label>
<input type="checkbox" name="hobby" value='골프' id='golf' {%if '골프' in
stu.hobby%}checked{%endif%}>
<label for='golf'>골프</label>
<input type="checkbox" name="hobby" value='수영' id='swim' {%if '수영' in
stu.hobby%}checked{%endif%}>
<label for='swim'>수영</label>
<input type="checkbox" name="hobby" value='조깅' id='run' {%if '조깅' in
stu.hobby%}checked{%endif%}>
<label for='run'>조깅</label>
<input type="checkbox" name="hobby" value='독서' id='book' {%if '독서' in
stu.hobby%}checked{%endif%}>
<label for='book'>독서</label>
<br>
```

```
<input type="submit" value='수정'>
<!-- button특성 : type지정하지 않으면 form action주소로 호출 -->
<a href="/students/view/{{stu.no}}/"><button type="button">취소</button></a>
</form>
</body>
</html>
```

6 학생 정보 삭제 페이지 : students:delete

```
from django.urls import path, include
from . import views
```

urls.py

```
# app_name는 / 붙이지 않음
app_name = 'students'
urlpatterns = [
    # path / 붙임.
    path('list/', views.list, name='list'),
    path('write/', views.write, name='write'),

    path('view/<int:no>/', views.view, name='view'),
    path('update/<int:no>/', views.update, name='update'),

    path('delete/<int:no>/', views.delete, name='delete'),
    # 학생정보삭제
]
```

views.py

학생정보삭제

def delete(request, no):

Student.objects.get(no=no).delete()

return redirect('/students/list/') # url -> name

return redirect('students:list') # app_name, path name

* name으로 데이터 검색 후
1개의 데이터를 삭제

6 학생 정보 삭제 페이지 : delete.html

```
<div>
  <a href='/students/list/'><button>학생리스트</button></a>
  <a href='/students/update/{{stu.no}}/'><button>수정</button></a>
  <button onclick='deleteBtn()'>삭제</button>
  <!--
  <a href="#" class="delete" data-uri="{% url 'students:stuDel' s.s_name %}">삭제 </a>
  -->
</div>
```

delete.html

```
<script>
  function deleteBtn(){
    if(confirm('{{stu.name}} 학생을 삭제하시겠습니까?')){
      location.href='/students/delete/{{stu.no}}/';
    }
  }
</script>
```

자바스크립트

```
<script src="http://code.jquery.com/jquery-latest.min.js">
</script>
<script>
  $(function(){
    $(".delete").on('click', function() {
      if(confirm("정말로 삭제하시겠습니까?")) {
        location.href = $(this).data('uri');
      }
    });
  });
</script>
```

자바스크립트

[참조] html -> views.py 데이터 전달방법

1. form 으로 전달
2. 파라미터 전달
3. 간단 URL 전달

1

form 전달

html

```
<form action="{% url 'students:regCon' %}" method="post">
  {% csrf_token %}
  name:<input type="text" name="name"> <br>
  <input type="submit" value="등록">
  <input type="button" onclick="cancel()" value="취소"> <br>
</form>
```

FROM 데이터 전달 : POST방식
request.POST.get('name')

2

파라미터 전달

```
<a href="students/boardList/?page=3">3페이지 호출</a>
```

파라미터 데이터 전달
request.GET.get('page') # 파라미터는 이렇게 받음.

3

간단 URL : page변수에 3이 들어 있음.

```
<a href="{% url 'students:boardList' page %}">변수넘김</a>
```

간단 URL 데이터 전달
def boardList(request,page): # 매개변수로만 받을수 있음.
print(page)
(x) request.GET.get('page') 이렇게 안됨.

[참조] checkbox

checkbox

```
user_hobby = request.POST.get('hobby')  
list_item = request.POST.getlist('hobby')
```

```
print("type : ",type(user_hobby))  
print("hobby : ",user_hobby)  
print("list_item type : ",type(list_item))  
print("list_item : ",list_item)
```

```
## form.html  
<input type="checkbox" name="hobby" value="game"  
{% if 'game' in stu.s_hobby %} checked {% endif %}>  
게임<br>
```

타입 : str (마지막 checkbox 1개만 출력)

타입 : list (선택된 모든 checkbox 값 출력)

```
hobby = request.POST.getlist("hobby")  
# hobby 리스트를 바로 저장 가능  
# 문자열로 자동변환  
qs = Student(name=name,age=age,grade=grade,gender=gender,hobby=hobby)
```

```
## 리스트를 문자열로 변경  
## join : list -> str
```

```
".join(list_item)  
' '.join(list_item)
```

```
## 문자열을 리스트로 변경  
## split : str -> list  
list = str.split(',')
```

static

이미지, css, js

7 [참조] css, js, images 연결

css, js, images 파일 연결 방법 : 프로젝트 파일 > setting.py 설정
자동import : vs-code ctrl + .

```
import os    ##추가시켜줌.
```

```
# setting.py > static 파일 설정  
STATIC_URL = 'static/'
```

```
# 이 부분을 추가
```

```
STATICFILES_DIRS = (  
    os.path.join(BASE_DIR, 'static'),  
  
    # BASE_DIR / 'static',  
    # os.path.join(BASE_DIR, 'students', 'static'),  
)
```

root > static 폴더를 생성하면 인식됨.
* 서버 구동시, 모든 apps에 있는 static폴더를 읽어옴.

7 [참조] css, js, images 연결

```
## html 에서 css, image 코드
{% load static %}
<link href="{% static 'css/style.css' %}" rel="stylesheet">


# url링크를 확인한후 현재폴더에서 링크 시킴

```

디버그 모드가 false변경 시
웹 서버가 대신 처리, 웹서버는 위치를 알지 못함.
STATIC_ROOT 경로에 모아 관리해야 함. (명령어 필요)

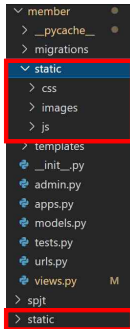
실서버 반영

```
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles/')
```

개발모드

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [ os.path.join(BASE_DIR, 'static'), ]
```



Root 디렉토리 static 과
Member apps static
모두 static 으로 통합되어 관리 됨.

어느 곳에 있어도 같은 폴더로 인식

프로젝트 폴더 > static 폴더 생성

- css
- images
- js

[참조] 주석

한줄 설명문 처리

[HTML]

<!-- 이곳에 작성... -->

[Javascript]

// 이곳에 작성...

[Django Template]

{# 이곳에 작성... #}

여러 줄 설명문 처리

[HTML]

<!--
이곳에 작성
-->

[Javascript]

/*
이곳에 작성...
*/

[Django Template]

{% comment %}
이곳에 작성...
{% endcomment %}

[참조]게시판 리스트

```
<h1>게시판 리스트</h1>
<h2>관리자 : {{ master }}, test변수 : {{ test }}</h2>
<table>
  <tr>
    <th>번호</th><th>제목</th><th>내용</th>
    <th>등록일</th><th>등록자</th><th>조회</th>
  </tr>
  {% if boardList %}
    {% for board in boardList %}
      <tr>
        <td>{{board.b_no }}</td>
        <td>{{board.b_title }}</td>
        <td>{{board.b_content }}</td>
        <td>{{board.b_date|date:'Y-m-d H:i' }}</td>
        <td>{{board.b_userid }}</td>
        <td>{{board.b_hit }}</td>
      </tr>
    {% endfor %}
  {% else %}
    <tr>
      <td colspan="5">데이터가 없습니다.</td>
    </tr>
  {% endif %}
</table>
<br>
<button><a href="{% url 'board:boardWrite' %}">글쓰기</a></button>
```

dictionary 에 추가로 데이터를 보낼때
- {{ }} 데이터변수 출력 방법

날짜 형식 출력
- board.b_date|date:'Y-m-d H:i'

[천단위 표시]

Settings.py

```
INSTALLED_APPS = [
    'django.contrib.humanize',
]
```

List.html : 천단위 표시

```
{% load humanize %}
{{d.audiAcc|intcomma}}
```

1,027,538

로그인 페이지

8 웹 애플리케이션 설계 - 게시판



8

Member 생성

```
## db구조  
from django.db import models
```

Member > models.py

```
class Member(models.Model):  
    id = models.CharField(max_length=50,primary_key=True)  
    pw = models.CharField(max_length=100)  
    name = models.CharField(max_length=50)  
    nickname = models.CharField(max_length=50)  
    tel = models.CharField(max_length=50,default='010-1111-1111')  
    gender = models.CharField(max_length=10,default='남자')  
    hobby = models.CharField(max_length=50,default='game')  
    mdate = models.DateTimeField(auto_now=True)  
  
def __str__(self):  
    return f"{self.id},{self.name},{self.nickname}"
```

```
from django.contrib import admin  
from member.models import Member
```

admin.py

```
@admin.register(Member)  
class MemberAdmin(admin.ModelAdmin):  
    list_display = ['id','name','nickname','mdate']  
  
# 관리자 페이지 컬럼 보이기  
# admin.site.register(Member,MemberAdmin)
```

브라우저가 서버로부터 받은 쿠키(사용자 정보)를 사용자의 웹 브라우저에 저장함.
 웹 브라우저가 종료되면 삭제되지만, 만료 날짜를 정해둘 시 만료일에 삭제 됨.

	쿠키(Cookie)	세션(Session)
저장 위치	클라이언트(사용자 PC)	서버
저장 형식	Text 형식	Object
만료 기준	브라우저 종료 여부 관계 없이, 만료시점까지	원칙적으로 브라우저 종료시 삭제, 장시간 켜둘 시 지정된 기간 뒤 삭제
리소스	클라이언트(사용자 PC) 사용	서버 리소스
속도	쿠키 > 세션	
보안	쿠키 < 세션	

8 쿠키(Cookie)

쿠키구성요소

- 이름 : 각각의 쿠키에 대한 식별자
- 값 : 쿠키의 이름과 관련된 값
- 유효시간 : 쿠키의 유지시간
- 도메인 : 쿠키를 전송할 도메인
- 경로 : 쿠키를 전송할 도메인

용량제한

- 한 도메인 당 20개, 모든 도메인에 대해 300개 제한, 하나의 쿠키는 4KB
- 클라이언트도 모르게 접속되는 사이트에 의해서 설정될 수 있기 때문에,
쿠키로 인한 문제가 발생하는 것을 막고자 제한을 걸어 둠
- 하나의 도메인에서 쿠키가 20개를 초과하면 가장 적게 사용된 쿠키부터 삭제

예시

- 아이디 기억하기 기능
- 쇼핑물의 장바구니 (비 로그인시, 장바구니에 담았던 품목을 로그인시

그대로 유지하기 위해서 사용)

사용 예

- 아이디 기억하기 기능
- 쇼핑물의 장바구니 (비 로그인시, 장바구니에 담았던 품목을 로그인시

그대로 유지하기 위해서 사용)

8 쿠키(Cookie)-html: 읽어오기

<h2>로그인</h2>

html

```
<form action="{% url 'students:login' %}" method='post' name='lfrm'>
  {% csrf_token %}
  <label>아이디</label>
  <input type="text" name='id' id='id' value="{{cook_id}}"> <br>
  <label>비밀번호</label>
  <input type="text" name='pw' id='pw'> <br>
  <input type='submit' value="로그인"> <br>

  <input type="checkbox" name='cook_id' id='cook_id' value='1'
    {% if cook_id %}checked{%endif%} >
  <label>아이디 기억하기</label> <br>

</form>
```

```
<div>쿠키정보 : {{c_info}}</div> # contex에서 넘어온 데이터
<div>쿠키정보 : {{ request.COOKIES }}</div> # html에서 직접 호출
```

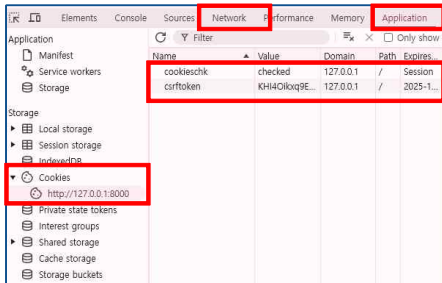
views.py에서
context 에 cook_id 를 넘겨줌.

8 쿠키(Cookie) - views.py : 저장, 읽어오기

```
from django.http import HttpResponse
from django.utils import timezone
```

views.py

```
## login
def login(request):
    if request.method == "GET":
        print(request.COOKIE)
        cook_id = request.COOKIE.get("cook_id", "")
        # 찾는 데이터가 있으면 값, 없으면 빈공백으로 리턴
        c_info = request.COOKIE
        context = {"c_info": c_info, "cook_id": cook_id} # 쿠키모든 정보보기
        response = render(request, 'students/login.html', context)
        if not request.COOKIE.get('cookieschk'):
            response.set_cookie("cookieschk", timezone.now())
        return response
    else:
        id = request.POST.get('id')
        pw = request.POST.get('pw')
        cook_id = request.POST.get('cook_id')
        print(id, pw, cook_id)
        print(request.COOKIE)
        response = redirect('students:login')
        if cook_id is not None:
            response.set_cookie('cook_id', id, max_age=60)
        else:
            response.delete_cookie("cook_id")
        return response
```



한글 디코딩

```
c_pld = request.COOKIE.get('c_pld', "")
c_pld = base64.b64decode(c_pld).decode()
```

한글 인코딩

```
c_pld = base64.b64encode(pld.encode()).decode()
```

8 쿠키(Cookie)

쿠키 검색

```
def checkCookie(request):
    mycookie_value = request.COOKIES.get('mycookie') # 쿠키읽기
    if mycookie_value == 'cookievalue':
        # Cookie exists and has the expected value
    else:
        # Cookie is not set or has a different value
```

쿠키 저장

```
def set_cookie(request):
    # response = render(request,'students/login.html')
    response = HttpResponse("Cookie Set!")
    # 쿠키설정
    response.set_cookie('mycookie', 'cookie_value')
    response.set_cookie('mycookie', 'cookie_value', max_age=60 )
    return response
```

쿠키 삭제

```
def deleteCookie(request):
    # response = render(request,'students/login.html')
    response = HttpResponse("Cookie Deleted!")
    response.delete_cookie('mycookie')
    return response
```

`set_cookie(name, value, max_age = None)`

- name : 쿠키의 이름(필수)
- value : 쿠키에 저장하려는 값(필수)
- max_age : 쿠키의 경과 시간(초).

지정하지 않으면 브라우저가 닫힐 때까지 쿠키가 존재(선택)

`max_age=60*60*24*365`
`# 60초 * 60분 * 24시간 * 365일`

일정 시간동안 같은 사용자, 브라우저로부터 들어오는 일련의 요구를 하나의 상태로 보고 그 상태를 일정하게 유지시키는 기술
쿠키와의 차이점은 사용자 정보를 브라우저에 저장하지 않고 서버에서 관리

만료시점

- 서버에서는 클라이언트를 구분하기 위해서 세션 ID 를 부여
웹 브라우저가 서버에 접속해서 브라우저를 종료할 때까지 인증상태 유지

장점

- 클라이언트의 정보를 서버에 저장하기 때문에 쿠키보다 보안에 우수

단점

- 쿠키보다 보안 면에서 우수하지만
사용자가 많아질 경우 서버 메모리를 많이 차지하게 됨

```
# session 저장
request.session['session_id'] = user_id
```

```
# session 삭제
del request.session['session_id'] # 일부삭제
request.session.clear()           # 모두삭제
```

```
- Html : session 읽기
{% if request.session.session_id %}
  <h1>로그인 상태 :
  {{ request.session.session_id }} </h1>
```

8 Login 설정

로그인 설정

views.py

```
def login(request):
    if request.method == 'GET':
        return render(request, 'login.html')

    elif request.method == 'POST':
        user_id = request.POST['id']
        user_pw = request.POST['pw']
        res_data = {}
        if not (user_id and user_pw):
            res_data['error'] = '모든 값을 입력해 주세요.'
        else:
            member = Member.objects.get(user_id=user_id)
            if member:
                user_id = member.user_id
                request.session['session_id'] = user_id
                # 세션에서 user를 key로하고 값을 할당 받은 user_id로 설정
                # session_id = cleaned_data.get('session_id')
                return redirect('/')
                # '/' 홈으로 이동
            else:
                res_data['error'] = '잘못된 아이디 또는 비밀번호 입니다.'
    return render(request, 'login.html', res_data)
```

로그인 설정

views.py

```
def logout(request):
    if request.session.get('session_id'):
        request.session.clear() # 모든 섹션 삭제
        # del request.session['session_id'] ## 일부 섹션만 삭제
    return redirect('/')
```

html

html

```
<h1>로그인</h1>
{% if request.session.session_id %}
<h1>로그인 상태 : {{ request.session.session_id }} </h1>
{% else %}
<h1>로그인 하셔야 합니다. </h1>
{% endif %}
```

id, pw가 일치하지 않을때 예외 처리

```
try:
    member = Member.objects.get(m_id=id,m_pw=pw)
except Member.DoesNotExist:
    member = None
```

<https://www.bootstrapcdn.com/>

BootstrapCDN

The recommended CDN for [Bootstrap](#), [Font Awesome](#), [Bootswatch](#) and [Bootstrap Icons](#)

☆ Star 1,354

🔗 Follow @jsdelivr

💖 Sponsor

Quick Start

Get your production ready CDN links below. You can also automate your upgrades by pulling the latest versions from our API.

v5.3.3

CSS

<https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css>

Click to copy

HTML

[<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">](https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css)

▼ REACTWORKSPACE

> re03

▼ re04

> node_modules

▼ public

★ favicon.ico

<> index.html

<> index.html U X JS App.js U JS Navbar.js U

re04 > public > <> index.html > html > head > link

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
6     <meta name="viewport" content="width=device-width, initial-scale=1" />
7     <meta name="theme-color" content="#000000" />
8     <meta
9       name="description"
10      content="Web site created using create-react-app"
11    />
12    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
```

<https://getbootstrap.com/docs/5.3/components/navbar/#how-it-works>

Components

- Accordion
- Alerts
- Badge
- Breadcrumb
- Buttons
- Button group
- Card
- Carousel
- Close button
- Collapse
- Dropdowns
- List group
- Modal
- Navbar**
- Navs & tabs
- Offcanvas

Nav

Navbar navigation links build on our `.nav` options with their own modifier class and require the use of [toggle classes](#) for proper responsive styling. **Navigation in navbars will also grow to occupy as much horizontal space as possible** to keep your navbar contents securely aligned.

Add the `.active` class on `.nav-link` to indicate the current page.

Please note that you should also add the `aria-current` attribute on the active `.nav-link`.

Navbar Home Features Pricing Disabled

HTML

```
<nav class="navbar navbar-expand-lg bg-body-tertiary">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">Navbar</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarN
      <span class="navbar-toggler-icon"></span>
```

Bootstrap 클래스 속성 사용하기

1. 부트스트랩 클래스명 총정리

1.1. 종단점 클래스명

1.2. width / height

1.3. 텍스트 색상

1.4. 텍스트 정렬

1.5. 텍스트 준비

1.6. 텍스트 대소문자

1.7. 폰트 크기

1.8. 폰트 속성

1.9. 텍스트 행 사이 간격 (행간)

1.10. margin / padding

텍스트 색상

텍스트 정렬

텍스트 줄바꿈

텍스트 대소문자

폰트 크기

텍스트 행 사이 간격 (행간)

margin / padding

별첨

Bootstrap 클래스 속성 사용하기

☞ 목차

1. 부트스트랩 클래스명 총정리

1.1. 중단점 클래스명

1.2. width / height

1.3. 텍스트 색상

1.4. 텍스트 정렬

1.5. 텍스트 줄바꿈

1.6. 텍스트 대소문자

1.7. 폰트 크기

1.8. 폰트 속성

1.9. 텍스트 행 사이 간격 (행간)

1.10. margin / padding

중앙 정렬

```
1 <div class="mx-auto" style="width: 200px;">
2   Centered element
3 </div>
```

border

```
1 // 테두리 방향
2 .border-top, end, bottom, start
3
4 // 테두리 색상
5 .border-primary, white, ...
6
7 // 테두리 두께
8 .border-1 ~ 6
9
10 // 테두리 둥근 모서리 형태
11 .rounded-top, circle, pill, ...
12
13 // 테두리 둥근 모서리 크기
14 .rounded-0 ~ 3
```

background

```
1 // 배경색
2 .bg-primary, white, ...
3
4 // 배경 그라데이션
5 .bg-gradient
```

display

```
1 // xs크기에서만 적용
2 .d-{value}
3 .d-inline-block
4 .d-grid
5
6 // 중단점 명시
7 .d-{breakpoint}-{value}
8 .d-lg-block
9
10 /* value 값 */
11 none
12 inline
13 inline-block
14 block
15 grid
```

position

```
1 // position : 속성
2 position-static
3 position-relative
4 position-absolute
5 position-fixed
6 position-sticky
```

top, left, right, bottom

```
1 top-0, 50, 100
2 start-0, 50, 100
```

flex

```
1 // display: flex;
2 .d-flex
3 .d-inline-flex
4
5 // 반응형
6 .d-sm-inline-flex
7 .d-xxl-inline-flex
```

방향

```
1 // flex-direction
2 .flex-row
3 .flex-row-reverse
4 .flex-column
5 .flex-column-reverse
6
7 // 반응형
8 .flex-sm-row
9 .flex-md-column
10 ...
```

계층형 게시판

```
## db구조
from django.db import models
from member.models import Member
```

Fboard > models.py

#Foreign-key 를 등록하면 Member 객체를 저장시켜야 함.

```
class Board(models.Model):
    bno = models.AutoField(primary_key=True)
    id = models.CharField(max_length=100)

    member = models.ForeignKey(Member, on_delete=models.DO_NOTHING, null=True)

    btitle = models.CharField(max_length=1000)
    bcontent = models.TextField()
    bgroup = models.IntegerField(null=True)
    bstep = models.IntegerField(default=0)
    bident = models.IntegerField(default=0)
    bhit = models.IntegerField(default=0)
    bdate = models.DateTimeField(auto_now=True)
```

```
def __str__(self):
    return self.btitle
```

Member > models.py

```
from django.db import models
```

```
class Member(models.Model):
    id = models.CharField(max_length=50, primary_key=True)
    pw = models.CharField(max_length=100)
    name = models.CharField(max_length=50)
    nickname = models.CharField(max_length=50)
    tel = models.CharField(max_length=50, default='010-1111-1111')
    gender = models.CharField(max_length=10, default='남자')
    hobby = models.CharField(max_length=50, default='game')
    mdate = models.DateTimeField(auto_now=True)
```

```
def __str__(self):
    return f'{self.id},{self.name},{self.nickname}'
```

* ForeignKey 생성

Member

```
class Member(models.Model):
    id = models.AutoField(primary_key=True)
```

<https://eveningdev.tistory.com/22?category=1182329>

https://velog.io/@brighten_the_way/Django%EC%99%80-Reverse-relations%EA%B3%BC-Relatedname

9 Foreign-key on_delete 설정방법

```
Member = models.ForeignKey(Member, on_delete=models.DO_NOTHING, null=True)
```

on_delete	의미
models.CASCADE	외래키를 포함하는 행도 함께 삭제
models.PROTECT	해당 요소가 함께 삭제되지 않도록 오류 발생 (ProtectedError)
models.SET_NULL	외래키 값을 NULL 값으로 변경 (null=True일 때 사용 가능)
models.SET(func)	외래키 값을 func 행동 수행 (func는 함수나 메서드 등을 의미)
models.DO_NOTHING	아무 행동을 하지 않음, 외래키에 없어도 입력 진행

9 글쓰기 저장 - Foreign-key 저장

views.py

```
from django.db.models import Max
# 글쓰기페이지, 글쓰기저장
def bwrite(request):
    if request.method == "GET":
        return render(request,'bwrite.html')
    else:
        id = request.POST.get("id")
        btitle = request.POST.get("btitle")
        bcontent = request.POST.get("bcontent")
```

```
# no = Board.objects.aggregate(max_bno = Max('bno'))
# no['max_bno']+1
# 오라클 sequence.nextval, sequence.currval
# bno,bstep,bindent,bhit,bdate - 자동
# id,btitle,bcontent,bgroup - 입력
```

방법2 : Max 후

```
# DB저장
qs = Board.objects.create(id=id,btitle=btitle,bcontent=bcontent)
qs.bgroup = qs.bno
qs.save()
```

방법1 : create 후

```
# 1회 노출
messages.success(request,message='게시글이 저장되었습니다.')

# return redirect('board:blist')
return render(request,'bwrite.html',{'w_msg':'1'})
```

F0 객체는 모델의 필드 혹은 어노테이트된 열의 값을 나타낸다. 실제로 데이터베이스에서 Python 메모리로 가져오지 않고, 모델 필드 값을 참조하고 이를 데이터베이스에서 사용하여 작업할 수 있다.

```
Fboard.objects.filter(b_group=b_group,b_step_gt=b_step).update(b_step=
F("b_step")+1)
```

* Foreign-key로 등록되어 있을때, 저장
`id = request.session.get('session_id')`
`member = Member.objects.get(id = id)`

* 최대값 가져오기
bno 수동으로 1씩증가해서 저장시켜줌.

게시글 저장을 먼저 한 후
bno데이터 읽어와 bgoup 다시 저장

* 가장 나중에 저장된 인덱스 가져오기
`last_helpdesk = helpdesk.objects.all().order_by('id').last()`

<https://ohgym.tistory.com/80?category=731677>

<https://pythong.com/so/python/1816041>

<https://wagin9.tistory.com/entry/Django-orm-max-value-latest-index>

<https://lqez.github.io/blog/django-queryset-basic.html>

9 게시판리스트

views.py

```
# 게시판리스트
def blist(request):
    qs = Board.objects.all().order_by("-bgroup", "bstep")
    context = {"blist":qs}
    return render(request, 'blist.html', context)
```

list.html

blist.html파일에서 출력

```
{% for board in blist %}
<tr>
  <td><span class='table-notice'>{{board.b_no}}</span></td>
  <td class='table-title' id='plnput'>
    <a href='{% url 'board:bview' board.b_no %}'>{{board.b_title}}</a>
  </td>
  <td>{{board.b_date|date:'Y-m-d H:i'}}</td>
  <td>{{board.member.m_id}}</td>
  <td>{{board.b_hit}}</td>
</tr>
{% endfor %}
```

{{m.cdate|date:'Y-m-d H:i'}}
Y : 년도 4자리 수
y : 년도 2자리

* board.member : return 함수의 값이 출력.
* board.member.m_id : id값이 출력

```
qs =
Board.objects.annotate(comment_count=Count('comment')).order_b
y('-bgroup', 'bstep', '-comment_count')
```

```
from django.db.models import Count
* 게시글리스트, 하단댓글개수 포함
* blist + comment_count
```

9 게시판 상세페이지 - 조회수 1증가

```
from django.db.models import F
```

```
# 글상세보기
```

```
def bview(request,bno):
```

```
    print("게시글 번호 : ",bno)
```

```
    # 조회수 1증가 - get : save()
```

```
    # qs = Board.objects.get(bno=bno)
```

```
    # qs.bhit += 1
```

```
    # qs.save()
```

```
    # context = {'board':qs}
```

```
    # 조회수 1증가 - filter : update()
```

```
    qs = Board.objects.filter(bno=bno)
```

```
    qs.update(bhit=F('bhit') + 1)
```

```
    context = {'board':qs[0]}
```

```
    return render(request,'bview.html',context)
```

views.py

F() 객체는 모델의 필드 혹은 어노테이트된 열의 값을 나타낸다. 실제로 데이터베이스에서 Python 메모리로 가져오지 않고, 모델 필드 값을 참조하고 이를 데이터베이스에서 사용하여 작업할 수 있다.

```
Fboard.objects.filter(b_group=b_group,b_step__gt=b_step).update(b_step=F('b_step')+1)
```

__gt 보다 작은

```
reboard = Fboard.objects.filter(b_group=b_group,b_step__gt=b_step)
reboard.update(b_step=F('b_step')+1)
```

```
from django.urls import path,include
from . import views
```

urls.py

```
app_name = "board"
```

```
urlpatterns = [
```

```
    path('blist/', views.blist, name='blist'),
```

```
    path('bwrite/', views.bwrite, name='bwrite'),
```

```
    path('bview/<str:bno>/', views.bview, name='bview'),
```

```
]
```

9

게시판 조회수 증가-중복방지가 안됨.

쿠키를 이용해서 조회수 중복 증가 막기

```
def view(request, no=0):  
    if no == 0: return HttpResponseRedirect('list')  
  
    board = Board.objects.filter(id=no)  
    board.update(hit=F('hit')+1)  
    context = {  
        'board':board[0]  
    }  
    return render(request, 'board/view.html', context)
```

이 코드의 경우 F5를 누르면 계속해서 조회수가 +1씩 증가

게시판 상세페이지 - 조회수 방지 (자바스크립트 사용)

```
import datetime
from django.db.models import F
```

views.py

```
# 게시글 상세보기
# 쿠키 저장형태 : cookie_bhit:aaa 101|102|97|90
def view(request, bno):
    # 조회수 1증가 - filter : update()
    qs = Customer.objects.filter(bno=bno)
    context = {'customer': qs[0]}
    response = render(request, 'customer/view.html', context)
```

```
# [1] 로그인 확인 - get() : 예러발생하지 않음. 쿠키이름
cookie_bhit:aaa
if request.session.get('session_id') is None:
    cookie_name = 'cookie_bhit'
else:
    cookie_name = f'cookie_bhit:{request.session["session_id"]}'
```

```
# 시간설정 - 오늘 23:59까지 설정
tomorrow = datetime.datetime.replace(datetime.datetime.now(),
hour=23, minute=59, second=0)
# 현재날짜 에 시간만 변경 - 현재날짜 23:59:00 변경됨.
```

```
# 쿠키 시간설정 형태로 맞춤.
expires = datetime.datetime.strftime(tomorrow,
"%a, %d %b %Y %H:%M:%S GMT")
```

views.py

```
# 쿠키가 존재하면
if request.COOKIE.get(cookie_name) is not None:
    cookies = request.COOKIE.get(cookie_name)
    print("cookies : ", cookies)
    cookies_list = cookies.split('|')
    print(cookies_list)
    # 쿠키리스트에 번호가 존재하지 않으면 1증가, 그렇지 않으면 증가없음.
    if str(bno) not in cookies_list:
        response.set_cookie(cookie_name, cookies + f'|{bno}', expires=expires)
        qs.update(bhit=F('bhit') + 1)
        return response
# [4] cookie_bhit 쿠키가 없으면
else:
    response.set_cookie(cookie_name, bno, expires=expires)
    qs.update(bhit=F('bhit') + 1)
    return response

return response
```

```
set_cookie(name, value,
max_age)
```

```
저장형태
1|5|7
cookies_bhit:aaa 1|5|7
```

9 게시판 상세페이지 – 이전글, 다음글

```
from django.db.models import F
from django.db.models import Q
```

views.py

```
# 조회수 1증가 - filter : update()
qs = Board.objects.filter(bno=bno)
```

```
# 이전글 가져오기
prev_board =
Board.objects.filter(Q(bgroup__lt=qs[0].bgroup,bstep__lte=qs[0].bstep)
|Q(bgroup=qs[0].bgroup,bstep__gt=qs[0].bstep)).order_by("-
bgroup","bstep").first()
```

다음글 가져오기 레코드 읽기(read) – 데이터 필터(filter)

```
next_board =
Board.objects.filter(Q(b
p)|Q(bgroup=qs[0].bgr
-bstep)).first()
```

```
context = {'board':qs[0]
"next_board":next_board
response = render(re
```

```
>>> qs = Student.objects.filter(s_age__lt=22)
>>> qs
<QuerySet [(<Student: HongGilDong>)]>
>>> qs = Student.objects.filter(s_age__gt=22)
>>> qs
<QuerySet [(<Student: HongGilJaSoon>)]>
>>> qs = Student.objects.filter(s_age__lte=22)
>>> qs
<QuerySet [(<Student: HongGilDong>, <Student: HongGilJa>)]>
>>> qs = Student.objects.filter(s_age__gte=22)
>>> qs
<QuerySet [(<Student: HongGilJa>, <Student: HongGilJaSoon>)]>
>>>
```

```
prev_board =
Board.objects.filter(Q(bgroup__lt=qs[0].bgroup,bstep__lte=qs[0].bstep) |
Q(bgroup=qs[0].bgroup,bstep__gt=qs[0].bstep)).order_by("-
bgroup","bstep").first()
```

이전글 가져오기

Q(bgroup__lt=qs[0].bgroup,bstep__lte=qs[0].bstep)

bgroup이 현재글보다 작으면, bstep 은 작거나 같은 것 중 첫번째

또는

Q(bgroup=qs[0].bgroup,bstep__gt=qs[0].bstep)

1째

__lt ~보다 작다

__lte ~보다 작거나 같다

__gt ~보다 크다

__gte ~보다 크거나 같다

__isnull ~ null인 자료 검색

__contains 특정 문자열을 포함하는 자료 검색

__startswith 특정 문자열로 시작하는 자료 검색

__endswith 특정 문자열로 끝나는 자료 검색

9

게시판 삭제

```
from django.conf import settings
from django.conf.urls.static import static
import os
```

```
# 게시글 삭제
```

```
def bdelete(request,bno):
    qs = Board.objects.get(bno=bno)
```

```
# 파일삭제
```

```
media_root = settings.MEDIA_ROOT
remove_file = media_root+"/"+str(qs.bfile)
if os.path.isfile(remove_file):
    os.remove(remove_file)
```

```
# db삭제
```

```
qs.delete()
context = {"dmsg":bno}
return render(request,'blist.html',context)
```

views.py

views.py

```
def bmodify(request,bno):
    print("게시글 번호 : ",bno)

    if request.method == "GET":
        qs = Board.objects.filter(bno=bno)
        context = {"board":qs[0]}
        return render(request,'bmodify.html',context)

    else: #post
        bno = request.POST.get("bno")
        btitle = request.POST.get("btitle")
        bcontent = request.POST.get("bcontent")
        qs = Board.objects.get(bno=bno)
        qs.btitle = btitle
        qs.bcontent = bcontent
        qs.save()
        # return redirect("board:blist")
        return render(request,'bmodify.html',{'u_msg':bno})
```

9 게시판 수정 - update.html

update.html

```
<form name="modify" method="post"
action='/board/update/{{board.bno}}' >
```

```
{% csrf_token %}
<table>
<input type="hidden" name="bno" value="{{board.bno}}">
<colgroup>
  <col width="15%">
  <col width="85%">
</colgroup>
<tr>
  <th>작성자</th>
  <td>
    <input type="text" name="name" value="{{board.id}}"
    readonly>
  </td>
</tr>
<tr>
  <th>제목</th>
  <td>
    <input type="text" name="btitle" value="{{board.btitle}}">
  </td>
</tr>
```

```
<form action="{% url 'board:bmodify' board.bno %}" name="modify"
method="post">
```

```
<tr>
  <th>내용</th>
  <td>
    <textarea name="bcontent" cols="50"
    rows="10">{{board.bcontent}}
  </td>
</tr>
<tr>
  <th>이미지 표시</th>
  <td>
    <input type="file" name="file" id="file">
  </td>
</tr>
</table>
<hr>
<div class="button-wrapper">
  <button type="submit" class="write">수정완료</button>
  <button type="button" class="cancel"
  onclick="javascript:location.back()">취소</button>
</div>
</form>
```

9 게시판 : 답글쓰기 저장

답글쓰기 저장
def breplyOk(request):

부모의 정보

b_group = int(request.POST.get('b_group'))

b_step = int(request.POST.get('b_step'))

b_indent = int(request.POST.get('b_indent'))

답글쓰기 내용

id = request.POST.get('id')

id = request.session.get('session_id')

member = Member.objects.get(id = id)

member = Member.objects.get(m_id=id)

title = request.POST.get('title')

content = request.POST.get('content')

img = request.FILES.get('img', '')

DB저장

qs = Board.objects.create(id=id, btitle=btitle, bcontent=bcontent
, bgroup = b_group)

qs =

Board.objects.create(member=member, btitle=btitle, bcontent=bcontent
, bgroup = b_group)

qs.save()

views.py

from django.db.models import F, Q

부모의 b_group속한 게시글 b_step을 +1 , __gt 보다 작은
reboard = Fboard.objects.filter(b_group=b_group, b_step__gt=b_step)
reboard.update(b_step=F('b_step')+1)

부모의 bgroup, 부모의 bstep+1, 부모의 b_indent+1

qs =

Fboard(b_no=b_no, member=member, b_title=title, b_content=content, b_group=b_group, b_step=b_step+1, b_indent=b_indent+1, b_img=img)

qs.save()

Fboard.objects.create(b_id=id, b_title=title, b_content=content)

return redirect('board:blist')

Q 객체는 AND, OR 및 NOT 연산에 사용

User.objects.filter(Q(city='city1') & Q(age=25))

9 게시판 : 답글쓰기 후리스트

blist.html

```
# html 페이지
<!-- 내용부분 -->
<tbody id="tbody">
  {% if blist %}
    {% for board in blist %}
      <tr>
        <td><span class='table-notice'>{{board.b_no}}</span></td>
        <td class='table-title' id='plnput'>
          <a href='{% url 'board:bview' board.b_no %}'>
            {% if board.b_indent > 0 %}
              {% for i in " "|rjust:board.b_indent %}
                ▶
              {% endfor %}
              {% for i in b_indent %}
                ▶ <!-- 반복 결과를 출력 -->
              {% endfor %}
              {{board.b_title}}
            {% else %}
              {{board.b_title}}
            {% endif %}
          </td>
        <td>{{board.b_date|date:'Y-m-d H:i'}}</td>
        <td>{{board.member}}</td>
        <td>{{board.b_hit}}</td>
      </tr>
    {% endfor %}
  {% else %}
    <tr>
      <td colspan='5'>데이터가 없습니다.</td>
    </tr>
  {% endif %}
```

```
{% for i in " "|rjust:board.b_indent %}
- board.b_indent 를 문자열로 취환해서 반복문으로 해결
- ""|4 -> " " 문자열을 4칸만듬 그래서 4번실행
```

```
</a>
</td>
<td>{{board.b_date|date:'Y-m-d H:i'}}</td>
<td>{{board.member}}</td>
<td>{{board.b_hit}}</td>
</tr>
{% endfor %}
{% else %}
<tr>
  <td colspan='5'>데이터가 없습니다.</td>
</tr>
{% endif %}
```

```
* views.py
context = { 'b_indent': range(board['b_indent']) }
```

```
* html
{% for i in b_indent %}
  {{ i }} <!-- 반복 결과를 출력 -->
{% endfor %}
```

9

게시판 만들기 : 답글추가 - reply 4

```
# html 페이지
<h1>답글달기</h1>
<hr>
<form action="{% url 'board:breplyOk' %}" name="reply"
method="post" enctype="multipart/form-data">
    {% csrf_token %}
    <table>
        <input type="hidden" name="b_no" value="{{board.b_no}}">
        <input type="hidden" name="b_group" value="{{board.b_group}}">
        <input type="hidden" name="b_step" value="{{board.b_step}}">
        <input type="hidden" name="b_indent"
value="{{board.b_indent}}">
```

breply.html

File 업로드

10 File 업로드 1

```
## file업로드에서 ImageField 항목 사용하려면 Pillow 인스톨 해야 함.  
## FileField 는 설치할 필요 없음.  
pip install Pillow
```

```
## 파일 업로드 할 위치  
#사용자가 업로드한 파일 관리  
MEDIA_URL = '/media/'  
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')  
# MEDIA_ROOT = r'c:/aaa' # 로컬 드라이브 설정
```

setting.py

```
## 프로젝트 > urls.py
```

project > urls.py

```
from django.conf import settings  
from django.conf.urls.static import static
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path("", include('board.urls')),  
]
```

```
# 파일업로드시 url구성 , urlpatterns 에 추가로 설정이 들어감.  
urlpatterns += static(settings.MEDIA_URL,  
    document_root=settings.MEDIA_ROOT)
```

```
## html
```

Write.html

```
<form action="{% url 'board:bWriteOk' %}" method="post"  
enctype="multipart/form-data">  
    {% csrf_token %}  
    작성자:<input type="text" name="b_userId"> <br>  
    제목:<input type="text" name="b_title"> <br>  
    내용:<input type="text" name="b_content"> <br>  
    파일:<input type="file" name="b_img"> <br>  
    <input type="submit" value="가입완료">
```

```
b_img = models.ImageField(blank=True,upload_to="fupload")
```

models.py

```
* request확인가능 : print(request.POST)  
파일 확인가능 : print(request.FILES)
```

request.POST.get()으로는 파일이 절대 전달되지 않음
반드시 request.FILES.get()을 사용해야 함

10 File 업로드 1

models.py의 models.ImageField()에 upload_to 옵션 추가
pip install pillow

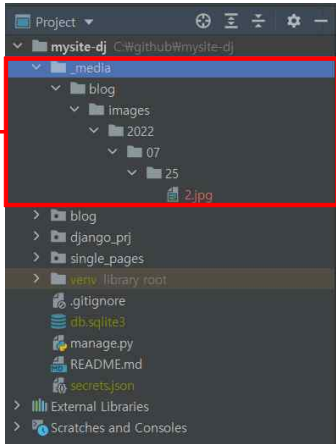
- upload_to 옵션 : media 폴더 내에 어느 경로로 저장할지 설정
 - 사용자가 이미지를 업로드할 때 해당 경로의 폴더들이 자동으로 생성된다. (media폴더도 생성해줄 필요 없다.)
- 예) b_img = models.ImageField(upload_to="reviewBoard/images", blank=True)

이미지 업로드 날짜에 따라 디렉토리에 저장 (strftime 으로 포매팅)

```
photo = models.ImageField(upload_to="blog/images/%Y/%m/%d",  
null=True)
```

- 사용자가 업로드한 이름이 기존 파일 이름과 겹치면 자동으로 변경

models.py



10 File 업로드 2

models.py에서 이미지 등록할수 있는 공간 필요

Models.py

```
b_hit = models.IntegerField(default=1)
b_img = models.ImageField(null=True,upload_to='board') # 항목추가
```

```
## db업데이트 해야 함.
python manage.py makemigrations
python manage.py migrate
```

```
# FileField 클래스 상속 ImageField
## ImageField -> FileField 변경가능
b_img = models.FileField
(null = True, upload='board')
```

views.py > 등록페이지 메소드
save해야 파일이 저장됨.

views.py

```
b_img = request.FILES.get('b_img','') ## 항목추가 / 파일이 없으면 ' '입력
qs = FreeBoard(b_title=b_title,b_content=b_content,b_userId=b_userId,b_img=b_img)
qs.save()
```

view.html

boardView.html

```
<th>사진</th>
{% if board.b_img %}
    <td></td>
{% else %}
    <td>이미지가 없습니다.</td>
{% endif %}
```

1. 경로

```

```

2. 경로

```

```

3. 파일이름

```

```

- url 은 media 폴더 부터 시작하는 상대 경로
- path 는 절대경로

10 File 삭제

views.py

```
from django.conf import settings
from django.conf.urls.static import static
import os
```

게시물 삭제

```
def bdelete(request,bno):
    qs = Board.objects.get(bno=bno)
```

파일삭제

```
media_root = settings.MEDIA_ROOT
remove_file = media_root+"/"+str(qs.bfile)
if os.path.isfile(remove_file):
    os.remove(remove_file)
```

db삭제

```
qs.delete()
context = {"dmsg":bno}
return render(request,'blist.html',context)
```

10 models.py class 1개 사용

models.py

```
class Board(models.Model):
    bno = models.AutoField(primary_key=True) #기 본키 등록
    id = models.CharField(max_length=100) #작성자
    btitle = models.CharField(max_length=1000) #제목
    bcontent = models.TextField() #내용
    # 답글달기
    bgroup = models.IntegerField(default=0) #답글달기 묶음
    bstep = models.IntegerField(default=0) #답글달기 순서
    bindent = models.IntegerField(default=0) #들여쓰기
    # -----
    bhit = models.IntegerField(default=0) #조회수
    bfile = models.ImageField(null=True, blank=True, upload_to='board')
    # FileField : 모든파일 업로드 가능
    ntchk = models.IntegerField(default=0)
    bdate = models.DateTimeField(auto_now=True) # 현재 날짜시간 자동등록

    def __str__(self):
        return f'{self.bno},{self.btitle},{self.bgroup}'
```

10 models.py class 2개 사용

models.py

```
class Board(models.Model):
    bno = models.AutoField(primary_key=True) #기본키 등록
    id = models.CharField(max_length=100) #작성자
    btitle = models.CharField(max_length=1000) #제목
    bcontent = models.TextField() #내용
    # 답글달기
    bgroup = models.IntegerField(default=0) #답글달기 묶음
    bstep = models.IntegerField(default=0) #답글달기 순서
    bindent = models.IntegerField(default=0) #들여쓰기
    # -----
    bhit = models.IntegerField(default=0) #조회수

    # FileField : 모든파일 업로드 가능
    ntchk = models.IntegerField(default=0)
    bdate = models.DateTimeField(auto_now=True) # 현재 날짜시간자동등록

    def __str__(self):
        return f'{self.bno},{self.btitle},{self.bgroup}'
```

models.py

```
class BoardFile(models.Model):
    board = models.ForeignKey(Board, on_delete=models.CASCADE,
        related_name='boardFiles')
    bfile = models.ImageField(null=True, blank=True, upload_to='board')
    bfileDate = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.bfile}'
```

admin.py

```
from django.contrib import admin
from .models import Board, BoardFile

admin.site.register(Board)
admin.site.register(BoardFile)
```

10 form.html - js : 파일업로드 후 바로 이미지 확인

이미지 업로드 시 바로 이미지 확인하는 스크립트

```
function readURL(input) {  
  if (input.files && input.files[0]) {  
    var reader = new FileReader();  
    reader.onload = function(e) {  
      document.getElementById('preview').src = e.target.result;  
    };  
    reader.readAsDataURL(input.files[0]);  
  } else {  
    document.getElementById('preview').src = "";  
  }  
}
```

script

```
<input type="file" onchange="readURL(this);">  
<img id="preview" style="width:100px;" />
```

html

 할시 이미지 크기 조정 가능

- * 파일 업로드시 이미지를 바로 보여지는 html은 없음.
- * 자바스크립트를 이용해서 보여질 수 있음

이미지 표시

파일 선택 _temp_16557127733930view.jpg

샘플이미지

업로드



작성완료

취소

Ajax - 1개 파일 업로드 방법

```

<script src="http://code.jquery.com/jquery-latest.min.js"></script>
<script>
    function writeCheck(){
        alert("파일업로드 진행");

        $.ajax({
            url: "/write",
            type: "post",
            enctype: "multipart/form-data",
            data: new FormData($('#writeForm')[0]),
            processData: false,
            contentType: false,
            cache: false,
            success: function(data){
                alert("데이터업로드성공");
                location.href = "/list";
            },
            error: function(){
                alert("에러");
            }
        });
    }
</script>
<button type="button" onclick="writeCheck()" class="write">작성완료</button>

```

Controller 부분

```

@RequestMapping("/board/write")
@ResponseBody
public String write(BoardDto boardDto, @RequestPart MultipartFile file) {
    boardService.boardWrite(boardDto, file);

    return "redirect:/board/list";
}

```

* processData : query string 이라는 형태로 전달

ex) http://example.com/board/content_view?btitle='제목'&bcontent='내용이 들어갑니다.'
 -> 이렇게 들어가면 파일 전송이 안됨.

* contentType : default 값이
 "application/x-www-form-urlencoded; charset=UTF-8"
 -> 이렇게 전달되면 파일 전송이 안됨.

* cache : 남아 있는 데이터 정보 정리 (false: 새롭게 데이터를 가져옴)

10 파일업로드 - Ajax 파일업로드 설정

Ajax - 다중 파일 업로드 방법

```
<script src="http://code.jquery.com/jquery-latest.min.js"></script>
function uploadFile(){
    var form = $('#writeForm')[0];
    var formData = new FormData(form);
    formData.append("files", $("#FILE1")[0].files[0]);
    formData.append("files", $("#FILE2")[0].files[0]);

    $.ajax({
        url: '/file_insert',
        processData: false,
        contentType: false,
        data: formData,
        type: 'POST',
        success: function(result){
            alert("업로드 성공!!");
            alert("name:"+result);
        }
    });
}
</script>
```

```
<button type="button" onclick="writeCheck()" class="write">작성완료</button>
```

Controller 부분

```
@RequestMapping("/board/write")
@ResponseBody
public String write(BoardDto boardDto, @RequestPart MultipartFile file) {
    boardService.boardWrite(boardDto, file);

    return "redirect:/board/list";
}
```

* processData : query string 이라는 형태로 전달

ex) http://example.com/board/content_view?btitle='제목'&bcontent='내용이 들어갑니다.'
-> 이렇게 들어가면 파일 전송이 안됨.

* contentType : default 값이
"application/x-www-form-urlencoded; charset=UTF-8"
-> 이렇게 전달되면 파일 전송이 안됨.

* cache : 남아 있는 데이터 정보 정리 (false:새롭게 데이터를 가져옴)

10 파일업로드 - Ajax 파일업로드 설정

수동으로 form 추가방법

script

```
const formData = new FormData();

// 텍스트 데이터 추가
formData.append('title', $('#title').val());

// 파일 추가 (1개만 업로드하는 경우)
const fileInput = $('#uploadFile')[0];
if (fileInput.files.length > 0) {
    formData.append('uploadFile', fileInput.files[0]);
} else {
    alert("파일을 선택해주세요.");
    return;
}
```

html

```
<input type="text" id="title" placeholder="제목">
<input type="file" id="uploadFile">
<button type="button" id="submitBtn">업로드
</button>

<script src="https://code.jquery.com/jquery-
3.6.0.min.js"></script>
```

10 파일업로드 - Ajax 파일업로드 설정

Form태그가 없을때 - 수동으로 form 추가방법

```
const formData = new FormData();
```

```
// 텍스트 데이터 추가  
formData.append('title', $('#title').val());
```

```
// 파일 추가 (1개만 업로드하는 경우)  
const fileInput = $('#uploadFile')[0];  
if (fileInput.files.length > 0) {  
    formData.append('uploadFile', fileInput.files[0]);  
} else {  
    alert("파일을 선택해주세요.");  
    return;  
}
```

```
// AJAX 요청  
$.ajax({  
    url: './write',  
    type: 'post',  
    data: formData,  
    processData: false,  
    contentType: false,  
    cache: false,  
    success: function(data) {  
        alert("업로드 성공");  
        location.href = './list';  
    },  
    error: function() {  
        alert("에러 발생");  
    }  
});
```

script

html

```
<input type="text" id="title" placeholder="제목">  
<input type="file" id="uploadFile">  
<button type="button" id="submitBtn">업로드  
</button>
```

```
<script src="https://code.jquery.com/jquery-  
3.6.0.min.js"></script>
```

게시판 (페이지 처리)

views.py

```
from django.http import HttpResponseRedirect
from django.shortcuts import render
from django.utils import timezone
from django.urls import reverse
from django.core.paginator import Paginator
```

```
from .models import Board
```

```
def boardList(request):
    qs = Board.objects.all().order_by("-b_no")
    paginator = Paginator(qs, 5)
    page = int(request.GET.get('page', 1))
    boardList = paginator.get_page(page)
    context = {'board_list': boardList }
    return render( request, 'board/index.html', context )
```

```
* Board.objects.all().order_by("-b_no")
- board테이블 번호역순으로 list 가져오기

* paginator = Paginator(qs, 5)
- qs에서 5개의 list만 출력시킴.

* page = int(request.GET.get('page', 1))
- boardList로 GET요청이 들어오면 page 파라미터를 읽는다. 없는 경우 1로 세팅.
- 파라미터 : href="?page=1"

* paginator.get_page(page)
- class 'django.core.paginator.Page' 를 리턴
- 페이징 처리에 필요한 데이터 있음.
( 총 페이징 수, 현재 페이지 등)
```

11 게시판 (페이지 처리) 2

list.html

```
<h2>현재 페이지 : {{nowpage}}</h2>
```

```
<ul class="page-num">
```

```
<!-- 이전페이지 정리 -->
```

```
{% if blist.has_previous %}
```

```
<a href="?nowpage=1"><li class="first"></li> </a>
```

```
<a href="?nowpage={{blist.previous_page_number }}"><li  
class="prev"></li> </a>
```

```
{% else %}
```

```
<li class="first"></li>
```

```
<li class="prev"></li>
```

```
{% endif %}
```

```
<!-- 순차적 넘버링 -->
```

```
{% for page in blist.paginator.page_range %}
```

```
{% if page != nowpage %}
```

```
<a href="?nowpage={{page}}"><li class="num  
off"><div>{{page}}</div> </li> </a>
```

```
{% else %}
```

```
<li class="num on"><div>{{page}}</div> </li>
```

```
{% endif %}
```

```
{% endfor %}
```

- * **boardList.has_previous** - 이전 페이지가 있는지 확인
- * **boardList.has_next** - 이후 전페이지가 있는지 확인

* **boardList.previous_page_number**

- 이전 페이지 번호

* **boardList.next_page_number**

- 이후 페이지 번호

* **boardList.paginator.page_range**

- 모든 페이지를 순차적으로 넘겨줌.

* **boardList.number|add:-2 and page <=**

boardList.number|add:2

- 현재 페이지를 중심으로 앞뒤 2개의 페이지 수가 보여줌

* **boardList.number|add:-3 and page <=**

boardList.number|add:3

- 현재 페이지를 중심으로 앞뒤 3개의 페이지는
.. 보여줌

11 게시판 (페이지 처리) 2

list.html

```
{% for p in page_range %}
  {% if p >= list.number|add:"-2" and p <= list.number|add:"2" %}
    {% if p == list.number %}
      <strong>{{ p }}</strong>
    {% else %}
      <a href="?page={{ p }}">{{ p }}</a>
    {% endif %}
  {% endif %}
{% endfor %}
```

11 게시판 (페이지 처리) 2

<!-- 이후 페이지 정리 -->

list.html

```
{% if blist.has_next %}
    <a href="?nowpage={{ blist.next_page_number }}"> <li
class="next"> </li> </a>
    <a href="?nowpage={{ blist.paginator.num_pages }}"> <li
class="last"> </li> </a>
{% else %}
    <li class="next"> </li>
    <li class="last"> </li>
{% endif %}

</ul>
```

```
from . import views
from django.urls import path, include
```

urls.py

```
app_name = 'board'
urlpatterns = [
    path('breply/<int:bno>/', views.breply, name='breply' ), # 답변달기
    path('bdelete/<int:bno>/', views.bdelete, name='bdelete' ), # 글삭제
    path('bmodify/<int:bno>/', views.bmodify, name='bmodify' ), # 글수정
    path('bview/<int:bno>/', views.bview, name='bview' ), # 글상세보기
    path('bwrite/', views.bwrite, name='bwrite' ), # 글쓰기
    path('blist/', views.blist, name='blist' ), # 게시판리스트
]
```

views.py

```
# 게시판리스트
def blist(request):
    # 현재페이지 받음, 없을때 1 고정
    nowpage = int(request.GET.get('nowpage', 1))
    # page = int(request.GET.get('page', 1))

    qs = Board.objects.all().order_by("-bgroup", "bstep")
    paginator = Paginator(qs, 5)
    blist = paginator.get_page(nowpage)

    context = {'blist': blist, 'nowpage': nowpage}
    return render(request, 'blist.html', context)
```

11 게시판 (페이지 처리) 3

메소드	설명
count()	총 객체 수
num_pages()	총 페이지 수
page(n)	n 번째 페이지 반환
page_range()	(1부터 시작하는)페이지 리스트 반환
get_page()	n번 페이지 가져오기
has_next()	다음 페이지의 유무를 boolean 으로 반환
has_previous()	이전 페이지의 유무를 boolean 으로 반환
previous_page_number()	이전 페이지 번호 반환

11 레코드 함수 사용

레코드 읽기(read) – 데이터 필터(filter)

```
>>> qs = Student.objects.filter(s_age__lt=22)
>>> qs
<QuerySet [<Student: HongGilDong>]>
>>> qs = Student.objects.filter(s_age__gt=22)
>>> qs
<QuerySet [<Student: HongGilJaSoon>]>
>>> qs = Student.objects.filter(s_age__lte=22)
>>> qs
<QuerySet [<Student: HongGilDong>, <Student: HongGilJa>]>
>>> qs = Student.objects.filter(s_age__gte=22)
>>> qs
<QuerySet [<Student: HongGilJa>, <Student: HongGilJaSoon>]>
>>>
```

Lookup	설명
contains	부분 문자열 포함 (대소문자 구분)
icontains	부분 문자열 포함 (대소문자 무시)
startswith	특정 문자열로 시작
istartswith	특정 문자열로 시작 (대소문자 무시)
endswith	특정 문자열로 끝남
iendswith	특정 문자열로 끝남 (대소문자 무시)

from django.db.models import F, Q, Sum, Count

__lt ~보다 작다

__lte ~보다 작거나 같다

__gt ~보다 크다

__gte ~보다 크거나 같다

__isnull ~ null인 자료 검색

__contains 특정 문자열을 포함하는 자료 검색

__startswith 특정 문자열로 시작하는 자료 검색

__endswith 특정 문자열로 끝나는 자료 검색

Model.objects.filter(필드__contains="부분문자열")

상단메뉴 공동사용

(top,footer)

12 templates 설정

1. templates 설정

▼ w1128\w01

- > board
- > comment
- > home
- > media
- > member
- > static
- ▼ templates
 - <> subbase.html
- ▼ w01
 - > __pycache__
 - 🔗 __init__.py
 - 🔗 asgi.py
 - 🔗 settings.py
 - 🔗 urls.py

* 메인과 서브페이지 차이가 있으면, 별도로 구분

* 전체 templates 설정, 각 app별 templates 존재

템플릿 경로

TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')

Setting.py

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [TEMPLATE_DIR],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

12 Base.html (상단메뉴 공동사용) 1

1. 공동부분 파일 생성

- blockhtml
 - > __pycache__
 - > migrations
 - templates
 - <> admin_account.html
 - <> admin_login.html
 - <> base.html
 - <> event_list.html
 - <> event_read.html
 - <> idCheck.html
 - <> join01_terms.html
 - <> join02_info_input.html
 - <> join03_success.html
 - <> login.html
 - <> main.html
 - <> modifying_member_info.html
 - <> notice_list.html

```
{% extends "base.html" %}  
{% extends "base2.html" %}
```

불가능

12 Base.html (상단메뉴 공동사용) 1

1. 공동부분 파일 생성

▼ blockhtml

- > __pycache__
- > migrations
- ▼ templates
 - <> admin_account.html
 - <> admin_login.html
 - <> base.html
 - <> event_list.html
 - <> event_read.html
 - <> idCheck.html
 - <> join01_terms.html
 - <> join02_info_input.html
 - <> join03_success.html
 - <> login.html
 - <> main.html
 - <> modifying_member_info.html
 - <> notice_list.html

* base.html 공통으로 적용될 html

* notice_list.html nav, footer (상단부분과 하단부분) 에 적용시킴

```
{% load static %}
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link
    href="https://fonts.googleapis.com/css?family=Noto+Sans+KR:400,500,700,900&display=swap&subset=korean" rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.12.1/css/all.min.css">
  {% block css-block %}
  {% endblock css-block %}
</head>
<body>
  <header>
    <ul>
      <li>회원가입</li> <span>|</span>
      <li>로그인</li> <span>|</span>
      <li>고객행복센터</li> <span>|</span>
      <li>배송지역검색</li> <span>|</span>
      <li>기프트카드 등록</li>
    </ul>
  </header>
```

base.html

```
<meta name="csrf-token" content="{{ csrf_token }}">
```

```
{% block css-block %}
{% endblock css-block %}
```

```
{% extends 'base.html' %}
{% load static %}
```

notice_list.html

```
{% block css-block %}
  <link rel="stylesheet" href="{% static 'css/css2/style.css' %}">
  <link rel="stylesheet" href="{% static 'css/css2/notice_list.css' %}">
{% endblock css-block %}
```

12 html 중복 (상단메뉴 공동사용) 1

1. 공동부분 파일 생성

```
▼ blockhtml
> __pycache__
> migrations
▼ templates
  <> admin_account.html
  <> admin_login.html
  <> base.html
  <> event_list.html
  <> event_read.html
  <> idCheck.html
  <> join01_terms.html
  <> join02_info_input.html
  <> join03_success.html
  <> login.html
  <> main.html
  <> modifying_member_info.html
  <> notice_list.html
```

```
<nav>
  <div class="logo"> </div>

  <div id="search">
    <div class="search"> </div> <br>
    <span>메뉴찾기 </span>
  </div>

  <div id="cart">
    <div class="cart"> </div> <br>
    <span>장바구니 </span>
  </div>

  <div class="nav-menu">
    <ul>
      <li>COOKIT소개 </li>
      <li>COOKIT 메뉴 </li>
      <li>리뷰 </li>
      <li>이벤트 </li>
      <li>MY쿡킷 </li>
    </ul>
  </div>
</nav>
<section>
  <!-- block 들어가는 부분 -->
  {% block content-block %}
  {% endblock content-block %}
  <!-- block 끝나는 부분 -->
</section>
```

base.html

```
{% extends 'base.html' %}
{% load static %}
```

```
{% block css-block %}
```

```
  <link rel="stylesheet" href="{% static 'css/css2/style.css' %}">
  <link rel="stylesheet" href="{% static
'css/css2/notice_list.css' %}">
{% endblock css-block %}
```

```
{% block content-block %}
```

```
<h1>NOTICE</h1>
<div class="wrapper">
  <form action="/search" name="search" method="post">
    <select name="category" id="category">
      <option value="0">전체 </option>
      <option value="title">제목 </option>
      <option value="content">내용 </option>
    </select>
```

```
.....
<ul class="page-num">
  <li class="first"> </li>
  <li class="prev"> </li>
  <li class="num">
    <div>1 </div>
  </li>
  <li class="next"> </li>
  <li class="last"> </li>
</ul>
```

```
  <div class="write"> 쓰기 </div>
{% endblock content-block %}
```

notice_list.html

Messages Framework

13 Messages

- 메시지를 하나의 HttpRequest 인스턴스에 임시로 저장하고 바로 다음 request에 표시됨
- 1회 노출되고 해당 페이지를 reload하면 사라짐

```
from django.contrib import messages
```

views.py

```
#축약된 방법
```

```
messages.info(request, 'Three credits remain in your account.')
```

```
messages.success(request, message="성공 메시지")
```

```
messages.error( request, f"에러 메세지" )
```

```
{% if messages %}
```

```
<div>
```

```
    {% for message in messages %}
```

```
        <h1>결과 : {{message.message}}</h1>
```

```
        <h1>결과 : {{message}}</h1>
```

```
    {% endfor %}
```

```
</div>
```

```
{% endif %}
```

html

하단댓글달기

14 게시글 하단댓글 달기 - DB설계

하단 댓글 클래스

```
## 게시글 > 댓글저장  
## post 1개의 게시글, primary key – board_no임.
```

views.py

```
class Comment(models.Model):  
    cno = models.AutoField(primary_key=True)  
    board = models.ForeignKey(Board,on_delete=models.CASCADE) # 부모삭제시, 자식삭제  
    member = models.ForeignKey(Member,on_delete=models.DO_NOTHING)  
    cpw = models.CharField(max_length=10,null=True,blank=True)  
    ccontent = models.TextField(blank=True)  
    cdate = models.DateTimeField(auto_now=True)
```

* ForeignKey로 등록하기 위해서는
우선 Board 클래스에
bno가 primary key로 등록이 되어 있어야 함.

14 제이쿼리 - 버튼 클릭시 data 넘기기

하단댓글 리스트 - ajax 사용

html

```
<button id="myButton" data-id="{{request.session.session_id}}">Click me</button>

<script>
$(function(){
    let c_token = '{{csrf_token}}'; //바로 받을수 있음.
    $(".memberbtn").click(function() {
        var name = $(this).data("id"); //해당 cno값 가져오기
        console.log("id: " + id);
    })
});
});
</script>
```

14 게시글 하단댓글 리스트 - board / view.html , urls.py

하단댓글 리스트 가져오기

```
<!-- 출력부분 -->
{% if comment %}
{% for c in comment %}
    <ul id="{{c.cno}}">
        <li class="name">{{c.member.name}} <span>[{{c.cdate|date:' Y-m-d
H:i:s '}}]</span></li>
        <li class="txt">{{c.ccontent}}</li>
        <li class="btn">
            <a href="#" class="rebtn">수정</a>
            <a href="#" class="rebtn">삭제</a>
        </li>
    </ul>
{% endfor %}
{% else %}
    <ul>
        <li class="name"></li>
        <li class="txt">댓글이 없습니다.</li>
        <li class="btn"></li>
    </ul>
{% endif %}
```

board : view.html

{{ comment.count }}

총 11 개의 댓글이 달려있습니다.

비밀번호

등록

* 비밀번호를 입력하시면 댓글이 비밀글로 등록됩니다.

aaa [2024-07-22 12:40:35.0]

test

수정

삭제

bbb [2024-07-17 12:49:21.0]

* 비밀글입니다.

aaa [2024-07-17 12:48:24.0]

비밀글을 등록합니다.

수정

삭제

board : urls.py

```
app_name="board"
urlpatterns = [
    path('blist/', views.blist,name="blist"),
    path('bview/<int:bno>/', views.bview,name="bview"),
]
```

14 게시글:하단댓글 포맷 변환

objects.get() 데이터 가져오기

```
from django.http import JsonResponse
from django.forms.models import model_to_dict
```

```
comment = Comment.objects.get(cno=cno)
comment_dict = model_to_dict(comment)
```

```
확인 : {comment: {...}, result: '성공'}
  comment:
    board: 105
    ccontent: "test3"
    cno: 30
    cpw: "1111"
    member: "aaa"
  ▶ [[Prototype]]: Object
  result: "성공"
```

objects.filter() 데이터 가져오기

```
qs =
Comment.objects.create(board=board,cpw=cpw,ccontent=ccontent,me
mber=member)
```

```
c_qs = list(Comment.objects.filter(cno=qs.cno).values())
context = {'comment':c_qs[0],'result':'성공'}
```

```
확인 : {comment: {...}, result: '성공'}
  comment:
    board_id: 105
    ccontent: "7777"
    cdate: "2025-12-18T05:47:48.390Z"
    cno: 31
    cpw: "1111"
    member_id: "aaa"
  ▶ [[Prototype]]: Object
  result: "성공"
```

Serializers.serialize() 변환

```
qs =
Comment.objects.create(board=board,member=member,cpw=cpw,cc
ontent=ccontent)
```

```
# Json 직렬화 작업 - 객체를 Json타입으로 변경
qs = serializers.serialize('json', [qs])
```

```
// 예: 문자열로 받아온 경우
const jsonData = '{"model": "board.board", "pk": 30, "fields": {"id": "aaa", "btitle": "444",
"bcontent": "444", "bgroup": 30, "bstep": 0, "bindent": 0, "bhit": 0, "bfile": "", "ntchk": 0,
"bdate": "2025-06-12T05:59:57.837Z"}}';
```

```
// 문자열로 넘어온 데이터를 Json타입으로 변경
let jsonData = JSON.parse(data.board);
console.log(jsonData[0].pk);
console.log(jsonData[0].fields.btitle);
```

14 게시글 하단댓글 쓰기 – comment / views.py

views.py

```
from django.shortcuts import render
from django.views.decorators.csrf import csrf_exempt
from django.http import JsonResponse, HttpResponse
from django.core import serializers # json타입
from member.models import Member
from board.models import Board
from comment.models import Comment
from django.forms.models import model_to_dict
```

하단댓글쓰기저장

def cwrite(request):

데이터가져오기

```
id = request.session['session_id']
member = Member.objects.get(id=id)
bno = request.POST.get("bno",1)
board = Board.objects.get(bno=bno)
cpw = request.POST.get("pw","")
ccontent = request.POST.get("ccontent","")
print(cpw,ccontent) ## 성공
```

날짜타입, Filefeild 타입 때문에 에러 나기도 함

def list(request) : ### 리스트 이름을 list 하지 말것.
List타입변경과 겹쳐 에러가 남.

views.py

DB저장 - 1개 파일

```
qs =
Comment.objects.create(board=board,member=member,cpw=cpw,ccntent=ccontent)
```

// json 타입변환 – QuerySet List타입이어야 list() 로 변환됨.

```
list_qs = list(Comment.objects.filter(cno=qs.cno).values())
print(list_qs)
context = {"comment":list_qs,"result":"success"}
return JsonResponse(context)
```

db저장 – 전체 파일 QuerySet List타입

```
qs = Board.objects.all()
l_qs = list(qs.values())
# l_qs = serializers.serialize('json', [qs])
```

14 게시글 하단댓글 읽기 : 방법2 - comment / views.py

views.py

```
from django.shortcuts import render
from comment.models import Comment
from board.models import Board
from member.models import Member
from django.http import JsonResponse, HttpResponse
# 전송할때 Json타입으로 변경해서 전송
from django.core import serializers
# Json타입으로 전달된 데이터를 파이썬데이터(set)로 변경
from django.core.serializers.json import DjangoJSONEncoder
```

방법 1

```
def list(request):
    qs = Board.objects.all()
    data = serializers.serialize('json', qs)
    return HttpResponse(data, content_type='application/json')
```

방법 2

```
# Json타입변경 - QuerySet List타입은 list타입으로 바로 변경
qs = Comment.objects.filter(cno=1)
print(qs)
l_qs = serializers.serialize('json', qs) # json타입으로 변환
context = {'result': 'success', 'list': l_qs}
return JsonResponse(context)
```

def list(request) : ### 리스트 이름을 list 하지 말것.
List타입변경과 겹쳐 에러가 남.

Html(Ajax)

HttpResponse 타입으로 넘어오면 배열 인식 됨.

```
console.log("확인 : ", data);
console.log("확인2 : ", list[0]); // 이제 제대로 출력됨
console.log("확인3 : ", data[0].fields);
```

Html(Ajax)

JsonResponse 로 리턴시 문자열이므로 JSON 변환해야 함.

```
console.log("확인 : ", data);
let list = JSON.parse(data.list); // 문자열 -> JS 객체
console.log("확인2 : ", list[0]); // 이제 제대로 출력됨
```

14 view 하단 댓글리스트 등록 1 - view.html : json데이터 받아오기

하단댓글리스트 등록

출 11 개의 댓글이 달려있습니다.

비밀번호

등록

* 비밀번호를 입력하시면 댓글이 비밀번호로 등록 됩니다.

aaa [2024-07-22 12:40:35.0]
test

수정 삭제

bbb [2024-07-17 12:49:21.0]
* 비밀번호입니다.

aaa [2024-07-17 12:48:24.0]
비밀글을 등록합니다.

수정 삭제

```
//keyCode함수
$(".replyTxt").keyup(function(e){
    console.log(e.keyCode);
    if(e.keyCode == 13) // 엔터키 입력시 버튼이벤트 호출
        $(this).closest("ul").find(".replyBtn").click();
});
```

```
//등록버튼 클릭
$(".replyBtn").click(function(){
    //alert("등록버튼 클릭");
    if("{{request.session.session_id}}" == ""){
        alert("로그인을 하셔야 댓글작성이 가능합니다.");
        //location.href="/member/login";
        $(".replyType").val("");
        return false;
    }
    let bno = "{{ board.bno }}";
    let id = '{{sessionid}}';
    let cpw = $(".replynum").val();
    let ccontent = $(".replyType").val();
    console.log("bno : "+bno);
    console.log("id : "+id);
    console.log("cpw : "+cpw);
    console.log("ccontent : "+ccontent);
```

view.html

메소드	설명
.html()	선택된 요소 안의 내용을 지우고 콘텐츠를 추가한다.
.append()	선택된 요소의 마지막에 새로운 요소나 콘텐츠를 추가한다.
.prepend()	선택된 요소의 첫번째에 새로운 요소나 콘텐츠를 추가한다.
.appendTo()	선택된 요소를 해당 요소의 마지막에 추가한다.
.prependTo()	선택된 요소를 해당 요소의 첫번째에 추가한다.

다음페이지 계속 >

14 view 하단 댓글리스트 등록2

```
//ajax
$.ajax({
  url: '/comment/cwrite/',
  type: "post",
  headers: { "X-CSRFToken": '{{csrf_token}}' },
  data: {"pw": pw, "ccontent": ccontent, "bno": bno},
  success: function(data){
    console.log("data : "+data)
    console.log("cno : "+data.comment[0].cno);
    console.log("member_id : "+ data.comment[0].member_id);
    console.log("board_id : "+ data.comment[0].board_id);
    console.log("id : "+ data.comment[0].ccontent);
    console.log("id : "+ data.comment[0].cdate);
```

view.html

```
if(data.result == "success"){
  alert("하단댓글이 저장되었습니다.")
  //하단댓글을 추가하는 코드 입력
  var datahtml = '';
  datahtml += '<ul id="'+ data.comment[0].cno +'>';
  datahtml += '<li class="name">{{board.member.name}}<span> [ '+
data.comment[0].cdate + ' ]</span></li>';
  datahtml += '<li class="txt">'+ data.comment[0].ccontent
+'</li>';
  datahtml += '<li class="btn">';
  datahtml += '<a href="#" class="rebtn">수정</a>&nbsp;&nbsp;&nbsp;';
  datahtml += '<a href="#" class="rebtn">삭제</a>';
  datahtml += '</li>';
  datahtml += '</ul>';
  $(".replyBox").prepend(datahtml);
```

//입력한 글 지우기

```
$(".replynum").val("")
$(".replyType").val("")
}else{
  alert("저장하는데 에러가 났습니다. 다시 입력하세요.")
}
},
error: function(){
  alert("실패");
}
})//ajax
})//click
});//jquery
```

<!-- 날짜 포맷함수 -->

```
<script type="text/javascript"
src="https://cdnjs.cloudflare.com/ajax/libs/moment.js/2.24.0/moment.min.js"></script>
```

* 자바스크립트 날짜 포맷함수 2024-01-01

aaa [2024-07-22 12:40:35.0]

test

수정

삭제

[+moment(data.cdate).format("YYYY-MM-DD HH:mm:ss")+];



[참고] 자바스크립트 백틱 사용

view.html

```
var datahtml = "";
datahtml += `
  <ul id="${data.comment[0].cno}">
    <li class="name">{{board.member.name}}<span> [ ${data.comment[0].cdate} ]</span></li>
    <li class="txt">${data.comment[0].ccontent}</li>
    <li class="btn">
      <a href="#" class="rebtn">수정</a>&nbsp;
      <a href="#" class="rebtn">삭제</a>
    </li>
  </ul>
`;

$(".replyBox").prepend(datahtml);
```

자바스크립트 : 백틱(`) 을 사용, 백틱 안에 \${} 를 사용하여 변수,표현식을 넣어주면 됨

14 view 하단 댓글리스트 - 삭제

하단댓글 리스트 삭제



* jquery 사용시

- .on 명령어와 .click 명령어
- .on 명령어는 동적인 제어가능, .click은 정적인 제어만 가능
- 댓글쓰기를 한 이후,
수정, 삭제를 하면 첫번째 페이지 호출 후 추가 된 소스이기에
동적으로 추가 .on 명령어를 사용해야 함

```
$(document).on("click", ".dBtn", function(){  
  
})
```

* jquery 사용시

\$(this)를 사용하면 매개변수 값을 넘겨줄 필요 없음

- \$(this) : 현재 위치
- closest : 가까이에 있는 위치값 찾을
- attr : 속성값 찾을
- children : 자식에서 찾을.

view.html

```
// 삭제버튼 클릭  
$(document).on("click", ".dBtn", function(){  
    console.log("ui cno : "+$(this).closest('ul').attr('id'));  
    if(!confirm("댓글을 삭제하시겠습니까?")){  
        return false;  
    }  
    let cno = $(this).closest('ul').attr('id');  
    $.ajax({  
        url: "/customer/commentBDelete",  
        type: "post",  
        headers: { "X-CSRFToken": '{{csrf_token}}' },  
        data: {"cno": cno},  
        dataType: "text",  
        success: function(data){  
            alert("댓글이 삭제되었습니다.");  
            $("#"+cno).remove();  
        }  
    });  
    //총개수 -1감소  
    $("#comment_number").text( Number($("#comment_number").text())-1 );  
}  
  
error: function(){  
    alert("실패");  
}  
}); //ajax  
}); //dBtn
```

- 특정 선택자를 가진 요소 제거 : .remove()
- 선택한 요소의 내용만 지울 때 : .empty()
- 선택한 요소의 속성(attribute) 제거 : .removeAttr()
- 선택한 요소의 클래스(class)의 값 제거 : .removeClass()

14 view 하단 댓글리스트 수정버튼 클릭

하단댓글 리스트 수정버튼 클릭 - 화면 변경

총 11 개의 댓글이 달려있습니다. 비밀번호 등록

* 비밀번호를 입력하시면 댓글이 비밀글로 등록 됩니다.

aaa [2024-07-22 12:40:35.0]
test

1 수정 삭제

총 11 개의 댓글이 달려있습니다. 비밀번호 등록

* 비밀번호를 입력하시면 댓글이 비밀글로 등록 됩니다.

aaa [2024-07-22 12:40:35.0]
test

완료 취소

view.html

// 수정버튼 클릭

```
$(document).on("click", ".uBtn", function(){
    alert("댓글 수정을 진행합니다.");
    console.log("ui cno : "+$(this).closest('ul').attr('id'));
    cno = $(this).closest('ul').attr('id');
    id = '${sessionId}';
    cdate = $(this).closest('ul').children('.name').children('span').text();
    ccontent = $(this).closest('ul').children('.txt').text();
    console.log("cno : "+cno);
    console.log("id : "+id);
    console.log("cdate : "+cdate);
    console.log("ccontent : "+ccontent);
    let htmlData = "";
    htmlData += '<li class="name">'+id+'<span>'+cdate+'</span></li>';
    htmlData += '<li class="txt"><textarea';
    htmlData += 'class="replyType">'+ccontent+'</textarea></li>';
    htmlData += '<li class="btn">';
    htmlData += '<a class="rebtn updateBtn">완료</a> ';
    htmlData += '<a class="rebtn cancelBtn">취소</a>';
    htmlData += '</li>';
    $("#"+cno).html(htmlData);
}); //uBtn
```

* jquery 사용시

\$(this)를 사용하면 매개변수 값을 넘겨줄 필요 없음

- \$(this) : 현재 위치
- closest : 가까이에 있는 위치값 찾을
- attr : 속성값 찾을
- children : 자식에서 찾을

14 view 하단 댓글리스트 수정버튼 취소

하단댓글 리스트 수정버튼 취소 - 화면변경

총 11 개의 댓글이 달려있습니다. 비밀번호

* 비밀번호를 입력하시면 댓글이 비밀글로 등록 됩니다.

aaa [2024-07-22 12:40:35.0]

test

총 11 개의 댓글이 달려있습니다. 비밀번호

* 비밀번호를 입력하시면 댓글이 비밀글로 등록 됩니다.

aaa [2024-07-22 12:40:35.0]

test

// 취소버튼 클릭

```
$(document).on("click",".cancelBtn",function(){
    alert("취소버튼을 클릭하셨습니다.");
    //데이터 확인
    console.log("ui cno : "+$(this).closest('ul').attr('id'));
    console.log("update cno : "+cno);
    console.log("update id : "+id);
    console.log("update cdate : "+cdate);
    console.log("update ccontent : "+ccontent);
    //데이터 넣기
    let htmlData = "";
    htmlData += '<li class="name">'+id+' <span>'+cdate+'</span></li>';
    htmlData += '<li class="txt">'+ccontent+'</li>';
    htmlData += '<li class="btn">';
    htmlData += '<a class="rebtn uBtn">수정 </a>&nbsp;&nbsp;&';
    htmlData += '<a class="rebtn dBtn">삭제 </a>';
    htmlData += '</li>';
    $("#"+cno).html(htmlData);
});//cancelBtn
```

view.html

14 view 하단 댓글리스트 수정완료 버튼

하단댓글 리스트 수정완료 버튼클릭

총 11 개의 댓글이 달려있습니다.

※ 비밀번호를 입력하시면 댓글이 비밀글로 등록 됩니다.

aaa [2024-07-22 12:40:35.0]

1

* DB 저장후 저장된 데이터 1개 가져오기

view.html

```
// 수정완료버튼 클릭
$(document).on("click", ".updateBtn", function(){
    alert("댓글 수정을 하였습니다.");
    //데이터 확인
    console.log("ui cno : "+$(this).closest('ul').attr('id'));
    ccontent = $(this).closest('ul').children('.txt').children('.replyType').val();
    console.log("update cno : "+cno);
    console.log("update id : "+id);
    console.log("update ccontent : "+ccontent);
    //수정한 내용을 controller로 전송
    $.ajax({
        url: "/customer/commentBUupdate",
        type: "post",
        data: {"cno": cno, "id": id, "ccontent": ccontent},
        // dataType: "json",
        success: function(data){
            alert("댓글이 수정되었습니다.");
            let htmlData = "";
            htmlData += '<li class="name">'+data.id+'
            <span>'+moment(data.cdate).format("YYYY-MM-DD HH:mm:ss")+']</span></li>';
            htmlData += '<li class="txt">'+data.ccontent+'</li>';
            htmlData += '<li class="btn">';
            htmlData += '<a class="rebtn uBtn">수정 </a>&nbsp;';
            htmlData += '<a class="rebtn dBtn">삭제 </a>';
            htmlData += '</li>';
            //현재 위치에 수정 코드 입력
            $("#"+cno).html(htmlData);
        },
        error: function(){
            alert("실패");
        }
    });
    //ajax
}); //updateBtn
```

종아요

15 좋아요 기능 - models.py

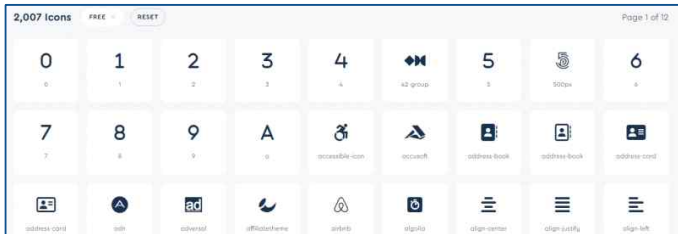
폰트 어썸 6 : <https://fontawesome.com/v6/search?m=free>

[cdn 주소 head에 추가]

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.1.0/css/all.min.css" integrity="sha512-10/jx2EXwxlWqCLX/hHth/wu2KY3jCF70dCQB8TSgNjbCVAC/8vai53GfMDrO2Emgwocf2pJqxd9ehpzG+MTw==" crossorigin="anonymous"
referrerpolicy="no-referrer" />
```

사용예 - 4,5와 클래스 명이 다릅니다. fa-solid 로 되어있습니다.

```
<i class="fa-solid fa-align-right"></i>
```



15 좋아요 아이콘 - 폰트어썸 6 사용법

```
from django.db import models
from member.models import Member
```

models.py

```
class Member(models.Model):
    # 필요한 필드를 추가할 수 있습니다.
    pass

class Board(models.Model):
    title = models.CharField(max_length=100)
    content = models.TextField()
    likes = models.ManyToManyField(Member,
    related_name='likes_member', blank=True)
```

```
def __str__(self):
    return self.title
```

{{ board.likes.all.count }} 명이 이 글을 좋아합니다.

```
from django.contrib import admin
from .models import Board
```

Admin.py

```
class BoardAdmin(admin.ModelAdmin):
    list_display = ('title', 'like_count')
```

```
def like_count(self, obj):
    return obj.likes.count()
```

```
admin.site.register(Board, BoardAdmin)
```

likes = models.ManyToManyField(Member)
하나의 Board에 여러 Member가 좋아요 가능.
하나의 Member도 여러 Board에 좋아요 가능

related_name='likes_member'
Member에서 좋아요 누른 게시글을 조회할 수 있음:
member.liked_boards.all()

15 사용예시

사용 예

```
board = Board.objects.get(bno=1)
member = Member.objects.get(id='aaa')

# 좋아요 추가
board.likes.add(member)

# 좋아요 취소
board.likes.remove(member)

# 좋아요 눌렀는지 확인
board.likes.filter(id=member.id).exists()

# 이 게시글을 좋아요한 멤버 목록
board.likes.all()

# 이 멤버가 좋아요한 게시글 목록
board에 만들어져 있음. 역참조시 related_name 사용
member.liked_boards.all()

# 좋아요 수
board.likes.count()
```

15 좋아요 기능 - models.py

Board > models.py

```
## db구조
from django.db import models
from member.models import Member

#Foreign-key 를 등록하면 Member 객체를 저장시켜야 함.
```

```
class Board(models.Model):
    bno = models.AutoField(primary_key=True)
    id = models.CharField(max_length=100)

    # member = models.ForeignKey(Member, on_delete=models.DO_NOTHING, null=True)
```

```
member = models.ForeignKey(Member, on_delete=models.CASCADE)
likes = models.ManyToManyField(Member, blank=True, related_name='like_member')
```

```
btitle = models.CharField(max_length=1000)
bcontent = models.TextField()
bgroup = models.IntegerField(null=True)
bstep = models.IntegerField(default=0)
bindent = models.IntegerField(default=0)
bhit = models.IntegerField(default=0)
bdate = models.DateTimeField(auto_now=True)
```

```
def __str__(self):
    return self.btitle
```

related_name 을 설정해줘야 한다.
그렇지 않으면 게시글을 작성한 유저
를 나타내는 Member 필드의 매니저
와 충돌하게 된다.

둘다 Member 테이블과 연결된 필드
이기 때문에 manager가 user_set으
로 표현된다. 그렇기 때문에
related_name을 써서 manager 충돌
을 방지

- member.like_member.all()

내가 좋아요한 게시글들

{{ board.like_members.all.count }} 명이 이 글을 좋아합니다.

15 좋아요 기능 : 추가, 삭제 - views.py, urls.py

views.py

```
from django.shortcuts import render
from board.models import Board
from comment.models import Comment
from django.http import JsonResponse, HttpResponse
from member.models import Member
```

```
## 좋아요 : 추가
def likes(request):
    # 데이터 가져오기
    id = request.session['session_id']
    member = Member.objects.get(id=id)
    bno = request.POST.get("bno", 1)
    board = Board.objects.get(bno=bno)

    if board.likes.filter(pk=member.id).exists():
        board.likes.remove(member) # relation_name 사용
    else:
        board.likes.add(member)

    print("개수 : ", board.likes.count())
    # 데이터 저장
    context = {"result": "success", "count": board.likes.count()}
    return JsonResponse(context)
```

urls.py

```
from django.urls import path, include
from . import views
```

```
app_name = "board"
urlpatterns = [
    path('blist/', views.blist, name="blist"),
    path('bview/<int:bno>/', views.bview, name="bview"),
    path('likes/', views.likes, name="likes"),
]
```

models.py

```
class Board(models.Model):
    ...
    member = models.ForeignKey(Member, on_delete=models.CASCADE)
    likes = models.ManyToManyField(Member, default='',
                                   related_name='like_member')
    ...
```

** 의미 정리

board.likes : Board → Member

member.likes_member : Member → Board (역참조)

likes_member 는 Member 쪽에서만 존재

15 좋아요 기능 - html

bview.html - js

```
$(function(){
    let temp = 0
    const csrfToken = $("meta[name='csrf-token']").attr("content");
    $("#likeBtn").click(function(){

        //alert("테스트");
        bno = "{{board.bno}}"
        alert(bno)

        //ajax
        $.ajax({
            url: '/board/likes/',
            type: "post",
            headers: {"X-CSRFToken": csrfToken},
            data: {"bno": bno},
            success: function(data){

                if(data.result == "success"){
                    alert("좋아요 클릭")
                    console.log("개수 : "+data.count)
                    $("#likecount").text(data.count)
                }else{
                    alert("저장하는데 에러가 났습니다. 다시 입력하세요.")
                }
            },
            error: function(){
                alert("실패");
            }
        })//ajax
    })
```

```
if(temp == 0){
    $(this).css({"background": "blue"})
    temp = 1
}else{
    $(this).css({"background": "yellow"})
    temp = 0
}
})//btn
```

bview.html

```
<ul>
<li>{{board.btitle}}
{% if request.session.session_id in board.like_member.all %}
    <button type='button' id="likeBtn" data-bno="{{board.bno}}">좋아요
취소</button>
{% else %}
    <button type='button' id="likeBtn" data-bno="{{board.bno}}">좋아요
</button>
{% endif %}
<br>
    좋아요수 : <span id="likecount"> {{board.like_member.all.count}}
</span>
</li>
</ul>
```

ajax 통신

16 ajax 댓글리스트 1 - get방식으로 전달

html 페이지

html

```
<script src="http://code.jquery.com/jquery-latest.min.js"></script>

<script type="text/javascript">
    $(function(){
$.ajax({
    url: "/board/boardList",
    type: "GET",
    headers: { "X-CSRFToken": '{{csrf_token}}' },
    // CSRF 토큰 포함
    data: { "name": "kim", "age": 30 },
    /* data: $("#frm").serialize(), */
    success: function(data){
        alert("성공");
    },
    error: function(){
        alert("실패");
    }
});
});
```

JsonResponse : Python의 딕셔너리나 리스트를 JSON 형

식

ajax 구조

url : 호출해야 할 url 경로
type : post 또는 get 요청 방식
data : 전달해야 할 데이터 {json String} 보통 json형태로 전달
contentType : 전송할 데이터 타입
datatype : 서버로 부터 리턴 타입을 지정.

beforeSend : function()
{ ajax호출 전 수행해야할 액티비티 로직 처리}
success : function(data) {요청이 완료된 경우 처리}
error : function(data) {실패 했을경우 처리}

@csrf_exempt

views.py에서 csrf 예외처리를 할 필요없음

16 ajax dataset 전송시 list 타입 변경

Views.py

```
from django.http import HttpResponse, JsonResponse
from django.views.decorators.csrf import csrf_exempt
```

```
@csrf_exempt
def chkid(request):
    # 객체타입 데이터를 넘길때, filter 사용
    id = request.POST.get('id', None)
```

```
    qs = list(Member.objects.filter(id=id).values())
```

```
    context = {'id': id, 'member': qs }
```

```
    return JsonResponse(context)
```

objects.get() 검색

```
from django.http import JsonResponse
from django.forms.models import model_to_dict
```

```
comment = Comment.objects.get(cno=cno)
comment_dict = model_to_dict(comment)
```

- * dataset 타입으로 넘길 시 에러발생
- * list 타입으로 변경해서 넘겨야 함.
- * get()이 아닌, filter() 로 검색해야 함.

TypeError: object of type queryset is not JSON serializable 오류발생

확인 :

```
{comment: {...}, result: '성공'}
```

comment:

- board: 105
- ccontent: "test3"
- cno: 30
- cpw: "1111"
- member: "aaa"

[[Prototype]]: Object

result: "성공"

16 ajax 댓글리스트 5-파일업로드 추가

#Ajax 에서 이미지 파일 업로드 방법

html

```
<script src="http://code.jquery.com/jquery-latest.min.js"></script>
```

```
<script type="text/javascript">
    $(function(){
```

//이미지 파일 업로드 방법

```
var data = new FormData($('#mfrm')[0]);
data.append("img_file", $('#img_file')[0].files[0]);
```

```
$.ajax({
    url: "/board/boardList",
    type: "POST",
    processData : false,
    contentType : false,
    data: data,
    success: function(data){
        // 생략~
    }
})
```

* new FormData(\$('#mfrm')[0])
웹페이지의 Form 태그에서 id를 사용하는 모든 값을 나타냄.

* \$('#img_file')[0].files[0]
img_file변수에 첨부파일 정보 나타냄.

id = request.POST.get('id', None)
bfile = request.FILES.get('bfile', "")

bfile = models.ImageField(upload_to="board", null=True)

* processData, contentType : 모두 false 로 해야 파일 전송 됨.

차트그리기

chart.js

17 Chart.js

chart.js는 html의 canvas태그를 통해 차트 생성 : <https://www.chartjs.org/>

1. jsp페이지 <head> 선언

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>  
<script src="https://cdn.jsdelivr.net/npm/chart.js@2.8.0/dist/Chart.min.js"></script>
```

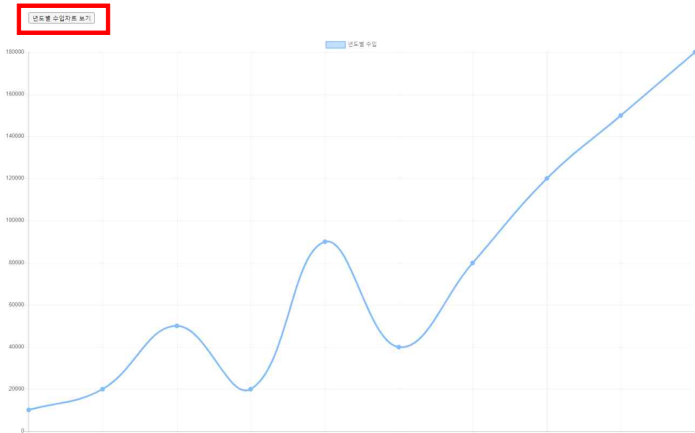
2. html 그려질 위치 선언

```
<div id="graph" style="width: 80%; margin: 30px;">  
  <div>  
    <canvas id="myChart" height="350" width="600"></canvas>  
  </div>  
</div>
```

17 화면

1. 버튼 클릭시 차트 그려짐.

jsp페이지 화면



17 jQuery 선언

1. jQuery 선언

```
$(function() {  
    // 구문 들어감  
});
```

//차트그리기

```
function createChart() {
```

```
    var ctx = document.getElementById("myChart").getContext("2d"); //html id myChart에 그려짐.
```

```
    LineChartDemo = Chart.Line(ctx, {
```

```
        data : shapeChartData, // 차트에 그려질 데이터값이 들어옴.
```

```
        options : {
```

```
            scales : {
```

```
                yAxes : [{
```

```
                    ticks : {
```

```
                        beginAtZero : true
```

```
                    }]
```

```
                }]
```

```
            }]
```

```
        }]
```

```
    });
```

```
}//createChart
```

```
// ajax를 사용할때는 꼭 리셋이 필요함.
```

```
// Chart객체 리셋해야 다시 차트를 그릴수 있음.
```

```
}
```

```
// ----- 그래프 그리기 -----
```

```
const ctx = document.getElementById('myChart');
```

```
let chartStatus = Chart.getChart("myChart");
```

```
if(chartStatus !== undefined){
```

```
    console.log("success");
```

```
    chartStatus.destroy();
```

```
}
```

17 jQuery 선언

1. jQuery 선언

```
$(function(){
```

html

```
//let ch_label = ['1월', '2월', '3월', '4월', '5월', '6월'];  
//let ch_data = [20, 19, 40, 30, 20, 50];  
let ch_label;  
let ch_data;
```

```
$("#btn").click(function(){  
    alert("검색을 시작합니다.");
```

```
//차트 초기화  
let chartStatus = Chart.getChart("myChart"); // <canvas> id  
if (chartStatus != undefined) {  
    chartStatus.destroy();  
}
```

```
ch_label=[];  
ch_data=[];
```

```
$.ajax({  
    url:"/chart_data",  
    type:"post",  
    data:{"data":$("#data").val()},  
    dataType:"json",  
    success:function(data){  
        alert("성공");  
        console.log("data : "+data);
```

```
//데이터 배열에 추가
```

```
data.forEach(function(d){  
  
    if(d.period==null) ch_label.push(d.region);  
    else ch_label.push(d.period);  
    ch_data.push(d.amt);  
});
```

```
//차트생성 시작  
const ctx = document.getElementById("myChart");  
new Chart(ctx, {  
    type: 'bar',  
    data: {  
        labels: ch_label,  
        datasets: [{  
            label: '지역별 매출액',  
            data: ch_data,  
            borderWidth: 1  
        }]  
    },  
    options: {  
        scales: {  
            y: {  
                beginAtZero: true  
            }  
        }  
    }  
});  
//차트생성 끝  
},  
error:function(){  
    alert("실패");  
}  
}); //ajax  
}); //click  
}); //jquery
```

17 jQuery 선언

```
//jQuery id="chartBtn" 버튼을 클릭하면 차트가 그려짐
$('#chartBtn').click(function(){
    chartLabels = [];
    chartData=[];

    //ajax 데이터
    $.ajax({
        url: "./chartIncome",
        data:{
            //ino : ino,
            //imonth : imonth
        },
        dataType: "json",
        type: "POST",
```

```
        backgroundColor: [
            'rgba(255, 99, 132, 0.2)',
            'rgba(54, 162, 235, 0.2)',
            'rgba(255, 206, 86, 0.2)',
            'rgba(75, 192, 192, 0.2)',
            'rgba(153, 102, 255, 0.2)',
            'rgba(255, 159, 64, 0.2)'
        ],
```

// 배열값으로 넣으면 bar 1개마다 색상을 넣을수 있음

```
success: function (data) { // data로 들어온 값을 값을 넣어줌
    $.each(data, function (key, value) {
        chartLabels.push(value.imonth); //chartLabels 배열
        chartData.push(value.iincome); //chartData 배열
    }); //each 배열에 data값을 넣어줌.
    //option부분 들어갈 내용
    shapeChartData = { //위쪽 createChart()함수 data에 들어감
        labels: chartLabels, // x축 데이터 값
        datasets: [{
            label: "년도별 수입",
            backgroundColor: "#bfdaf9",
            borderColor: "#80b6f4",
            pointBorderColor: "#80b6f4",
            pointBackgroundColor: "#80b6f4",
            pointHoverBackgroundColor: "#80b6f4",
            pointHoverBorderColor: "#80b6f4",
            fill: false, //차트하단 색채우기
            borderWidth: 4, //선 두께
            data: chartData //y축 데이터 값
        }]
    };
    createChart();
    //data
    //chartBtn
```

17 Html 페이지 코딩

1. html페이지

```
<body>
  <div class="row">
    <div class="container">
      <div style="margin-top:20px; margin-left:80px">
        <button id="chartBtn">년도별 수입차트 보기</button>
      </div>
    </div>
    <div id="graph" style="width: 80%; margin: 30px;">
      <div>
        <canvas id="canvas" height="350" width="600"></canvas>
      </div>
    </div>
  </div>
</body>
```

버튼 클릭시 이벤트 발생

그래프가 그려지는 위치

17 Html 페이지 코딩

1. views.py

```
# 차트페이지 호출
def chlist(request):
    profit = [19, 20, 21, 22, 23, 24]      # 타입 : List타입
    # profit = [20, 15, 7, 25, 27, 30]      # 타입 : List타입

    qs = TotalSales.objects.filter(year=2025)
    print('qs 기본구문 : ',qs)              # 타입 : QuerySet List타입
    print('list타입 구문 : ',list(qs.values())) # 타입 : List타입
    # 특정컬럼만 List타입으로 변경
    totalSales = list(TotalSales.objects.filter().values_list('totalSales', flat=True))
    print("총매출액 : ",totalSales)
    context = {'profit':profit,'list':list(qs.values()),'totalSales':totalSales}
    print("영업이익 : ",profit)

    return render(request,'chart/chlist.html',context)
```

공공데이터 출력

18 공공데이터 출력 1

Views.py

```
# 공공데이터 라이브러리 import
import requests
import pprint
import json
```

```
### 전역변수
dlist = []
```

```
# 공공데이터 리스트
```

```
def list(request):
    global dlist
    dlist = publicData()
    context = {'list':dlist}
    return render(request,'pboard/list.html',context)
```

```
# 공공데이터 상세보기 - 공공데이터 다시 호출
```

```
def view(request,galleryId):
    global dlist
    print("넘어온 galleryId : ",galleryId)
    print("넘어온 dlist : ",dlist)
    # 공공데이터 호출
    # dlist = publicData()
    for d in dlist:
        if d['galleryId'] == str(galleryId): # 타입확인 할것
            dData = d
            break

    context = {'dData':dData}
    return render(request,'pboard/view.html',context)
```

```
# 공공데이터 가져오기 함수
```

```
def publicData():
    public_key =
'918RE13GA7OY7ZEmUzApgbOeAcQoZ%2FaHsXWcqPAKQ9YNNPj83
KOstRMRIUrCFIAcm9qj2R6b7NFZjp%2FYsYzJLg%3D%3D'
    pageNo = 1
    url =
f'http://apis.data.go.kr/B551011/PhotoGalleryService1/galleryList1?service
Key={public_key}&numOfRows=10&pageNo={pageNo}&MobileOS=ETC&
MobileApp=AppTest&arrange=A&_type=json'
    # 웹스크래핑 requests
    response = requests.get(url)
    #print("공공데이터 : ",response.text) # str타입

    # 문자열 -> json타입으로 변경
    json_data = json.loads(response.text)
    dlist = json_data['response']['body']['items']['item']
    # print('json데이터 : ',json_data['response']['body']['items']['item']) #
json타입
    # print('json데이터 1개 : ',json_data['response']['body']['items']['item'][0])

    return dlist
```

https:// : s를 넣으면 에러 발생 (SSLError ...)

Http:// 해야 함

18 공공데이터 출력 1 – ajax/views.py

Views.py

```
# 공공데이터 라이브러리 import
import requests
import pprint
import json
```

```
### 전역변수
dlist = []
```

```
# 공공데이터 리스트
```

```
def list(request):
```

```
    global dlist
```

```
    json_data = json.loads(response.text)
```

```
    dlist = json_data['response']['body']['items']['item']
```

```
# 데이터 처리 로직
```

```
processed_data = {
```

```
    "received": data
```

```
}
```

```
return JsonResponse(processed_data)
```

18 공공데이터 출력 2

```
{% for d in list %}
```

```
<tr>
  <td class="tnone">{{d.galContentId}}</td>
  <td class="left">
    <a href="/pboard/view/{{d.galContentId}}/">{{d.galTitle}}</a>
  </td>
  <td>{{d.galPhotographyMonth}}</td>
  <td class="tnone right">{{d.galPhotographer}}</td>
</tr>
```

```
{% endfor %}
```

List.html

view.html

```
<h2><strong>공공데이터 상세보기</strong><span>자딩샵 소식을 전해드립니다.</span></h2>
```

```
<div class="viewDivMt">
  <div class="viewHead">
    <div class="subject">
      <ul>
        <li>{{dData.galTitle}}</li>
      </ul>
    </div>
    <div class="day">
      <p class="txt">작성일
      <span>{{dData.galPhotographyMonth}}</span></p>
    </div>

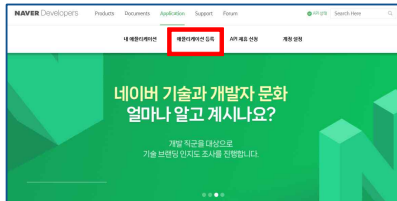
    <div class="viewContents">
      {{dData.galPhotographyLocation}}<br/>
      <br/>
      {{dData.galSearchKeyword}}<br/>
    </div>
    <div class="viewContents">
      <br>
    </div>
  </div>
```

네이버 API

19 네이버 API

개발자 가이드에서 뉴스검색 애플리케이션을 등록

<https://developers.naver.com/main/>



Client ID, Secret 확인하기

애플리케이션 정보

Client ID	
Client Secret	

보기

news

애플리케이션 이름 <O>

- 네이버 로그인할 때 사용자에게 표시되는 이름이므로 서비스 브랜드를 대표할 수 있는 이름으로 가급적 10자 이내로 간결하게 설정해주세요.
- 40자 이내의 영문, 한글, 숫자, 공백문자, 점표(.), "_", "-", " ", 만 입력 가능합니다.

사용 API <O>

선택하세요.

검색

환경 추가

WEB 설정

웹 서비스 URL (최대 10개)

http://127.0.0.1:8000

- 텍스트 중 우측 끝의 '+' 버튼을 누르면 행이 추가되며, '-' 버튼을 누르면 행이 삭제됩니다.
- http와 https는 구분하지 않습니다.
- www는 빼고 입력해 주세요. 예) <http://naver.com>
- 서브 도메인이 있으면 대표 도메인만 입력해 주세요. (예: <http://naver.com>)
- 하이브리드 앱은 location.href 객체 출력 값을 입력하면 됩니다. (예: <file:///로컬URL>)

등록하기

취소

19 네이버 API - views.py : 출력컬럼

4. 출력 결과

요청 변수	값	설명
rss	-	디버그를 쉽게 하고 RSS 리더기만으로 이용할 수 있게 하기 위해 만든 RSS 포맷의 컨테이너이며 그 외의 특별한 의미는 없다.
channel	-	검색 결과를 포함하는 컨테이너이다. 이 안에 있는 title, link, description 등의 항목은 참고용으로 무시해도 무방하다.
lastBuildDate	datetime	검색 결과를 생성한 시간이다.
total	integer	검색 결과 문서의 총 개수를 의미한다.
start	integer	검색 결과 문서 중, 문서의 시작점을 의미한다.
display	integer	검색된 검색 결과의 개수이다.
item/items	-	XML 포맷에서는 item 태그로, JSON 포맷에서는 items 속성으로 표현된다. 개별 검색 결과이며 title, link, description을 포함한다.
title	string	검색 결과 문서의 제목을 나타낸다. 제목에서 검색어와 일치하는 부분은 태그로 감싸져 있다.
link	string	검색 결과 문서의 하이퍼텍스트 link를 나타낸다.
image	string	썸네일 이미지의 URL이다. 이미지가 있는 경우만 나타남니다.
author	string	저자 정보이다.
price	integer	판매가 정보이다. 절판도서 등으로 가격이 없으면 나타나지 않는다.
discount	integer	할인 가격 정보이다. 절판도서 등으로 가격이 없으면 나타나지 않는다.
publisher	string	출판사 정보이다.
isbn	integer	ISBN 넘버이다.
description	string	검색 결과 문서의 내용을 요약한 패시지 정보이다. 문서 전체의 내용은 link를 따라가면 얻을 수 있다. 패시지에서 검색어와 일치하는 부분은 태그로 감싸져 있다.
pubdate	datetime	출간일 정보이다.

개수 지정시 개수만큼 가져옴.
- 100 : 100개 가져옴



19 네이버 API – views.py : requests 방식

views.py

```
import requests
from django.http import JsonResponse
from urllib.parse import quote
```

```
def test(request):
```

```
    client_id = "api Client ID"
    client_secret = "api Client Secret"
```

```
    search_word = "경제"
    encode_type = "json"
    sort = "sim"
    max_display = 100
    start = 1
```

```
    query = quote(search_word) # * 한글 인코딩
    url = (
        f"https://openapi.naver.com/v1/search/book.{encode_type}"
        f"?query={query}&display={max_display}&start={start}&sort={sort}"
    )
```

```
    headers = {
        "X-Naver-Client-Id": client_id,
        "X-Naver-Client-Secret": client_secret,
    }
```

views.py

```
res = requests.get(url, headers=headers)
data = res.json() # JSON 변환
```

```
return JsonResponse(data, safe=False)
```

19 네이버 API - views.py : 네이버방식

네이버 검색 Open API 예제 - 블로그 검색

```
import os
import sys
import urllib.request
import json
```

```
client_id = "YOUR_CLIENT_ID"
client_secret = "YOUR_CLIENT_SECRET"
```

```
encText = urllib.parse.quote("검색할 단어")
url = "https://openapi.naver.com/v1/search/book.json?query=" + encText # JSON 결과
# url = "https://openapi.naver.com/v1/search/book.xml?query=" + encText # XML 결과
request = urllib.request.Request(url)
```

```
request = urllib.request.Request(url)
request.add_header("X-Naver-Client-Id", client_id)
request.add_header("X-Naver-Client-Secret", client_secret)
response = urllib.request.urlopen(request)
rescode = response.getcode()
```

```
if(rescode==200):
    response_body = response.read()
    print(response_body.decode('utf-8'))
else:
    print("Error Code:" + rescode)
```

views.py

```
# JSON -> 딕셔너리 타입변환
data = json.loads(response_body)
```

```
# list 추출
blog_list = data['items']
```

```
context = {"list":blog_list}
return render(request, 'naver1.html', context)
```

views.py

19 네이버 API - html

html

```
<table class="board-table">
  <thead>
    <tr>
      <th>책제 목</th>
      <th>저자 이름</th>
      <th>출판사</th>
      <th>출간일</th>
      <th>이미지</th>
    </tr>
  </thead>
  <tbody>
    {% for item in list %}
    <tr>
      <td class="title"><a href="{{item.link}}">{{item.title}}</a></td>
      <td>{{item.author}}</td>
      <td class="views">{{item.publisher}}</td>
      <td class="views"><img style='width:100px;'
src='{{item.image}}'></td>
      <td class="date">{{item.pubdate}}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
```

오라클 db (model연결)

20 오라클 db변경 : setting.py 환경설정

Django setting.py 오라클 db변경

- Django에서 쓰는 model 명령어 중 oracle11g에는 지원하지 않는 문법이 있기 때문에 oracle11g는 더이상 지원하지 않음.

```
# setting.py

TIME_ZONE = 'Asia/seoul'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.oracle',
        'NAME': 'XE',
        'USER': 'ora_user',
        'PASSWORD': '1111',
        'HOST': 'localhost',
        'PORT': '1521',
    }
}
```

20 Setting.py 설정 - oracle db 변경

1. templates 설정

- ▼ w1128\w01
 - > board
 - > comment
 - > home
 - > media
 - > member
 - > static
 - ▼ templates
 - <> subbase.html
 - ▼ w01
 - > __pycache__
 - 🔗 __init__.py
 - 🔗 asgi.py
 - 🔗 **settings.py**
 - 🔗 urls.py

* 기본 : sqlite3
* Oracle DB 변경 / My-sql 변경

```
# oracle DB 연결
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.oracle',
        'NAME': 'localhost:1521/xe',
        'USER': 'ora_user2',
        'PASSWORD': '1111',
    }
}
```

Setting.py

python manage.py migrate sessions
//세션만 따로 migrate 해야 함.

```
# Mysql 연결
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql', # mysql 엔진 설정
        'NAME': 'mysite', # 데이터베이스 이름
        'USER': 'root', # 데이터베이스 연결시 사용할 유저 이름
        'PASSWORD': 'password', # 유저 패스워드
        'HOST': '127.0.0.1', # 데이터베이스 서버 주소
        'PORT': '3306' # 데이터베이스 서버 포트
    }
}
```

Setting.py

데이터 분석 라이브러리 설치
대체 : `python -m pip install oracledb`
3.10 이전 버전 : `pip install cx_Oracle`

```
import oracledb as ora
```

```
#오라클 connection 연결
```

```
## conn = ora.connect('localhost:1521/x')
```

```
conn = connections()
```

```
print(conn.version)
```

```
# db연결부분을 함수로 정의
```

```
def connections():
```

```
    try:
```

```
        conn = oracledb.connect(user="ora_user", password="1111",  
                                dsn="localhost:1521/x")
```

```
# DB에 연결 (호스트이름 대신 IP주소 가능)
```

```
    except Exception as e:
```

```
        msg="예외발생"
```

```
        print(msg)
```

```
    return conn
```

20 execute 1개 row 저장

Views.py

오라클 연결

import oracledb as ora

def ora_list(request):

오라클 연결

conn = connections()

print(conn.version)

cursor 할당

cursor = conn.cursor()

#SQL문 작성

sql_insert = 'insert into item_code values(:no,:name)'

sql_insert

입력값을 쿼리에 바인딩한 후 전송

cursor.execute(sql_insert,no='0001',name='종목명')

#커서 닫고 commit 해준다.

cursor.close()

conn.commit()

conn.close() # 리소스를 많이 잡혀 열어놓고 사용하기도 함.

return render(request, 'ora_list.html')

db연결부분 함수로 정의

def connections():

try:

conn= ora.connect('c##ora_user1/1111@localhost:1521/xs')

cursor = conn.cursor()

except Exception as e:

msg="예외발생"

print(msg)

return conn

1개의 데이터 입력

sql 구문 , board_seq.nextval : 시퀀스 증가, sysdate : 날짜자동입력

sql_insert = 'insert into board

values(board_seq.nextval, :b_id, :b_title, :b_content, sysdate)'

실행구문 - b_no='1': 직접입력가능

cursor.execute(sql_insert, b_no=b_no, b_title=b_title,

b_content=b_content)

print(qs.query) # sql쿼리문 확인

20 execute 여러 row 저장, 저장타입

Views.py

```
# 오라클 연결 여러개 파일
def ora_list(request):
    ## import cx_Oracle as ora
    ## 오라클 연결
    df =
pd.read_excel('C:\WpyFolder\Wjs_work\score.xlsx', usecols='A:D', dtype={0:str, 1:str, 2:str, 3:str})
df1 = df[['지원번호', '이름', '학교', '키']]
rows = [ tuple(x) for x in df1.to_records(index=False) ]

conn = connections()
print(conn.version) # 18.4.0.0.0
cursor = conn.cursor()
print(cursor) # <cx_Oracle.Cursor on <cx_Oracle.Connection to
c##ora_user1@localhost:1521/xes>

# oracle db 에 저장
cursor.execute('delete from score')
score_query = 'insert into score values(:1,:2,:3,:4)'
cursor.executemany(score_query, rows)
conn.commit()

cursor.execute('select count(*) from score')
cnt = cursor.fetchone()
print('insert 건수 : ', cnt)
cursor.close()

return render(request, 'notice_list.html')
```

pandas 엑셀 데이터 불러오기

usecols='A:D' # 엑셀컬럼 A,B,C,D를 가져옴.

dtype = { 0:str, 1:str, 2:str, 3:str } # 0번째 컬럼 str타입 설정
예) {0: float, 1:int}

1줄 for문은 리턴이 자동으로 LIST가 됨.

rows는 tuple(x)형태의 LIST
data=[
 ('1번', '강나래', '구로고', '197'), ('2번', '강태원', '구로고', '184')
 ...
]

데이터 insert, select시 명령어
cursor.execute(score_insert, stu_no='1번', stu_name='강나래')
cursor.executemany(score_query, rows)

데이터 select시 명령어 -> execute후 fetch사용
cursor.execute(score_select, stu_no='1번')
cursor.fetchall() : 모든 데이터를 가져올 때 사용
cursor.fetchone() : 한번 호출에 1개 Row를 가져올 때 사용
컬럼이 지원번호, 이름, 학교, 키 # no = row[0] # 지원번호 데이터 넘어옴
name = row[1] # 이름 넘어옴...
school = row[2]
cursor.fetchmany(n) : n개 만큼의 데이터를 가져올 때 사용

20 오라클 db연결 - int일때 error

Views.py

```
# 오라클 연결 여러개 파일
def ora_list(request):
    ## import cx_Oracle as ora
    ## 오라클 연결
    df =
    pd.read_excel('C:\WpyFolder\Wjs_work\score.xlsx', usecols='A:D', dtype={0: str, 1: str, 2: str, 3: str})
    df1 = df[['지원번호', '이름', '학교', '키']]
    rows = [ tuple(x) for x in df1.to_records(index=False) ]
    conn = connections()

    ## 이하동일
```

pandas 엑셀 데이터 불러오기

usecols='A:D' # 엑셀컬럼 A,B,C,D를 가져옴.

dtype = { 0:str, 1:str, 2:str, 3:str } # 0번째 컬럼 str타입 설정
예) {0: float, 1:int}

데이터를 tuple()형태의 list타입으로 데이터 저장.

```
# rows = [ tuple(x) for x in df.to_records(index=False) ]
rows=[]
for x in df.to_records(index=False):
    row=[x[0],x[1],x[2],int(x[3])]
    rows.append(row)
```

파이썬 pandas 를 이용하여 str 변수는 문제가 없으나
int인 경우는 변환 에러 발생

(해결) int타입으로 한번 더 변경해 줌.

20 select 데이터 가져오기

```
import cx_Oracle
```

```
# rowfactory 를 통해 데이터 타입 변경
```

```
def makeDictFactory(cursor):  
    columnNames = [d[0] for d in cursor.description]
```

```
    def createRow(*args):  
        return dict(zip(columnNames, args))
```

```
    return createRow
```

```
# 오라클 db연결
```

```
conn = connections()  
cursor = conn.cursor()  
cursor.execute("select * from test_tbl where rownum < 2")
```

```
cursor.rowfactory = makeDictFactory(cursor)
```

```
rows = cursor.fetchall()
```

```
for row in rows :  
    print(row)
```

```
conn.close()
```

다행히 cxOracle은 cursor의 rowfactory 라는 메서드를
오버라이딩하여 리턴받는 데이터의 형태를 바꿀 수 있음.

```
{'name': 'kim', 'age': 27, 'address': '서울시 광진구'}
```

```
name = row['name'] #kim  
age = row['age'] #27  
address = row['address'] #서울시 광진구
```

cxOracle 라이브러리는 SELECT 해오는 쿼리 결과 : tuple형태 list타입

```
('kim', 27, '서울시 광진구')
```

```
name = row[0] #kim  
age = row[1] #27  
address = row[2] #서울시 광진구
```

데이터를 확인

```
name = row['name'] #kim  
age = row['age'] #27  
address = row['address'] #서울시 광진구
```

20 select 검색하려는 데이터 입력

```
import oracledb
conn =
oracledb.connect(user='ora_user',password='1111',dsn='localhost:1521/xe')
cursor = conn.cursor()
```

```
# sql작성, 실행
num = input("숫자를 입력하세요>>(해서 입력하시오.) ")
# 10,20,30
n_list = num.split(",")
```

```
# no=10,20,30 을 검색해서 출력하시오.
# sql = "select * from students where no in(:no1,:no2,:no3)"
# cursor.execute(sql,no1=num,no2=num2,no3=num3)
```

```
# 3. execute 함수 : 리스트 값 전달
# execute뒤에는 dict,list,tuple타입만 가능
sql = "select * from students where no in(:1,:2)"
cursor.execute(sql,n_list)
```

```
# 2. execute 함수 : 변수 key값 전달
# sql = "select * from students where no>:no"
# cursor.execute(sql,no=num)
```

```
# 1. 문자열 함수 f 사용
# sql = f"select * from students where no>={num}"
# cursor.execute(sql)
```

```
# 데이터 가져오기 - fetchone():1개,fetchmany(10):숫자만큼,fetchall():모든것
```

```
rows = cursor.fetchall()
titles = ['번호','이름','국어','영어','수학','합계','평균','등수','등록일']
for title in titles:
    print(title,end="\t")
print()
print("-"*80)
```

```
for row in rows:
    for i,r in enumerate(row):
        if i == 1:
            print(f"{r:10s}",end="\t")
            continue
        if i == 6:
            print(f"{r:.2f}",end="\t")
            continue
        if i == 8:
            # strftime() 함수 : 날짜포맷 함수 %Y : 2024 , %y : 24
            print(r.strftime("%Y-%m-%d"),end="\t")
            continue
        print(r,end="\t")
    print()
```

```
# 종료
conn.close()
```

20 select like

```
import oracledb
conn =
oracledb.connect(user='ora_user',password='1111',dsn='localhost:1521/xs')
cursor = conn.cursor()
```

입력한 값을 가지고 이름이 포함되어 있는 데이터를 출력하시오

```
search = input("이름을 입력하세요.>> ")
search = '%' + search + '%'
sql = "select * from employees where emp_name like :search "
cursor.execute(sql,search=search)
```

파이썬에서 '%'는 외부에서 값을 받아올 때 타입을 지정해주기 위해 사용

```
# search = input("번호검색 >> ")
# sql = "select * from employees where emp_name like '%:search%'"
# 키워드
# sql = "select * from employees where employee_id>=:search"
# cursor.execute(sql,search=search)
# 번호순서
# sql = "select * from employees where employee_id>=:1"
# cursor.execute(sql,[search])
```

20 DataFrame 으로 받기

```
import cx_Oracle
import pandas as pd
from stboard.models import board
```

```
# db연결함수로 정의
def connections():
    try:
        conn= ora.connect('system/1234@localhost:1521/xe')
        # cursor = conn.cursor()
    except Exception as e:
        msg="예외발생"
        print(msg)
    return conn
```

```
# pd. Read_sql_query 구문 사용
def blist(request):
    query ='select * from board'
    df = pd.read_sql_query(query,connections())
    print(df)
```

pandas 라이브러리 read_sql_query사용

sql에서 바로 DataFrame만듬.

메일보내기 (naver메일)

21 네이버 이메일 보내기 설정 1

네이버 설정

NAVER 메일

배달쓰기 | 내게쓰기

970 | ★ | ✎ | TO |
인입률 | 중요 | 첨부 | 받는사람

전체메일 | 관리하기
받은메일함 399 | 수신확인
보낸메일함
임시보관함 | 비공개함
내겨쓰메일함 | 비공개함
새 메일함 (1)
내 메일함 |
Unwanted

프로모션 240
청구결제 48

환경 설정 | 메일로 돌아가기

기본 환경 설정 | 메일함 관리 | 메일 자동 분류 | 서명/백분담장 | 부재 중 설정 | 새 메일 알림 설정
스톱 설정 | 외부 메일 가져오기 | ✓ POP3/IMAP 설정 | 단축키

POP3/SMTP 설정 | IMAP/SMTP 설정

스마트폰에서 네이버 메일을 더욱 잘 관리하고 싶으시다면 IMAP/SMTP를 설정합니다.

jung_ye_jin 님은 현재 IMAP/SMTP를 사용하고 있습니다.

IMAP/SMTP 사용 | ☒ 사용함 | ☐ 사용 안 함 ?

IMAP 동기화 메일 제한 | 1,000통 |
IMAP으로 동기화할 수 있는 최대 메일 통수를 지정할 수 있습니다.
너무 많은 메일을 동기화하는 경우 데이터 과공을 주의해 주세요.

기본 설정으로 | 확인 | 취소

* 환경설정

- 내 메일함 톱니 모양을 클릭

1. POP/IMAP 설정

2. IMAP/SMTP 설정

3. IMAP/SMTP 사용 > 사용함 클릭

4. 저장

* 메일 프로그램 환경 설정 안내

휴대폰, 아웃룩 등 외부 메일 프로그램 환경설정에서 아래와 같이 등록해 주세요.

POP 서버명 : po

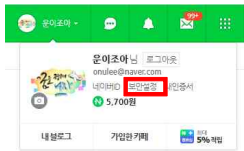
SMTP 포트 : 465

* 하단부분 정보

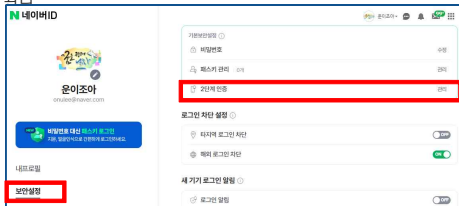
- 메일 프로그램 환경 설정 안내

21 네이버 2단계 인증 및 토큰키 비밀번호 관리

1. 로그인후 닉네임 > 보안설정 클릭



2. 메뉴 위치: 네이버ID > 보안설정 > 기본보안설정 > 2단계 인증 > 관리



3. 비밀번호 생성

애플리케이션 비밀번호 관리 [?]

(IMAP · POP · SMTP 방식의 메일(제어통독), CardDav 방식의 캘린더 등 일부 애플리케이션은 2단계 인증을 지원하지 않습니다.)
이치를 점검에서는 먼저 비밀번호를 설정한 후, 애플리케이션 로그인 화면에서 해당 비밀번호를 입력해 주세요.

애플리케이션 비밀번호 생성

1. 종류선택

☐ 아운트 ☐ 하이픈 ☐ 만드스로드론 ☐ 지어일

☒ 직접 입력

* 편의를 위해 대표적인 종류를 나열한 것으로, 원하는 종류가 없는 경우 '직접 입력'한 후 가입하면 됩니다.

2. 비밀번호 생성

생성하기

3. 비밀번호 확인

* 생성한 비밀번호를 애플리케이션 로그인 시 사용하세요.
* 비밀번호는 12자리이며 알파벳, 대문자와 숫자입니다.

비밀번호는 처음 외에는 볼수 없음.

21 Selenium – 네이버 이메일 보내기 2 (text 내용보내

기)

```
import smtplib
from email.mime.text import MIMEText
```

```
smtpName = "smtp.naver.com"
smtpPort = 포트번호
```

```
sendEmail = "나의 네이버 계정"
password = "나의 네이버 비밀번호"
recvEmail = "받는 사람 이메일"
```

```
title = "파이썬 이메일보내기"
content = "이메일을 보내줍니다."
```

```
msg = MIMEText(content) #msg = MIMEText(text,'html')
msg['From'] = sendEmail
msg['To'] = recvEmail
msg['Subject'] = title
```

* 환경설정 하단 부분에 포트 정보가 있음.

POP 서버명 : pop.naver.com
SMTP 포트 : 465

* 읽어들이는 파일의 텍스트를 MIME 객체화

* html 소스 추가 보내기

```
text = f"""
<html>
<body>
    <h2>{title}</h2>
    <p>메일 html전송</p>
</body>
</html>
"""
```

```
msg = MIMEText(text,'html')
```

21 네이버 이메일 보내기 3

```
# 메일 서버 정보 smtp.naver.com/587
s = smtplib.SMTP(smtpName , smtpPort)
# 메일 서버 접근
s.starttls()
# 메일 서버 로그인
s.login(sendEmail , password)
# 메일 발송
s.sendmail(sendEmail, recvEmail, msg.as_string())
# 메일 닫기
s.close()
```

21 예제

네이버 메일 보내기 > main.py > ...

```
1 import smtplib
2 from email.mime.text import MIMEText
3
4 send_email = "gmlstjq123@naver.com"
5 send_pwd = "i"
6
7 rcv_email = "gmlstjq123@inha.edu"
8
9 smtp_name = "smtp.naver.com"
10 smtp_port = 587
11
12 text = """
13 안녕, 나는 네이버 크롬이야.
14 너는 인하대 크롬이고,
15 만나서 반갑다!
16 """
17
18 msg = MIMEText(text)
19 msg["Subject"] = "나에게 보내는 편지"
20 msg["From"] = send_email
21 msg["To"] = rcv_email
22 print(msg.as_string())
23
24 s = smtplib.SMTP(smtp_name, smtp_port)
25 s.starttls()
26 s.login(send_email, send_pwd)
27 s.sendmail(send_email, rcv_email, msg.as_string())
28 s.quit()
```

* starttls

TLS 보안연결을 시작

TLS는 인터넷 연결을 통해 전송 중인 이메일에 대한 무단 액세스를 방지

* msg.as_string

발신자, 수신자, 제목, 본문 등의 필드를 갖고 있는 MIMEText 객체를 문자열로 변환한다.

21 네이버 이메일 파일첨부 1

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email import encoders

smtpName = "smtp.naver.com"
smtpPort = 587
sendEmail = "onulee@naver.com"
password = "1111"
recvEmail = "onulee@naver.com"

title = "파이썬 파일 첨부"
content = "파이썬 이메일 파일 첨부 소스 코드입니다."
```

* 파일 첨부에 필요한 라이브러리

21 네이버 이메일 파일첨부 2

MIME객체화

```
msg = MIMEMultipart('alternative')
```

내용부분

```
part2 = MIMEText(content)
```

```
msg.attach(part2)
```

```
msg['From'] = sendEmail
```

```
msg['To'] = recvEmail
```

```
msg['Subject'] = title
```

* 파일첨부에 필요한 MIME설정

* MIMEText 설정

21 네이버 이메일 파일첨부3

```
# 파일첨부
part = MIMEBase('application','octet-stream")
# 파일 읽어오기
with open("시가총액_20220216.csv","rb") as f:
    # part에 담기
    part.set_payload(f.read())
# 파일 첨부할수 있는 형태로 인코딩
encoders.encode_base64(part)
# header정보 정의
part.add_header('Content-Disposition','attachment',filename="시가
총액_20220216.csv")

msg.attach(part)
```

* 파일첨부에 필요한 MIME

* Content-Disposition 헤더 추가

- 첨부파일에는 Content-Disposition 헤더를 설정해야
첨부파일로 표현
(gmail은 없어도 첨부목록에 노출)
- text 발송은 설정이 필요없음.

21 예제

9. 네이버 메일 보내기 > 파일 첨부하여 네이버 메일 보내기.py > ...

```
1 import smtplib
2 from email.mime.multipart import MIMEMultipart
3 from email.mime.text import MIMEText
4 from email.mime.application import MIMEApplication
5
6 send_email = "gmlstjq123@naver.com"
7 send_pwd = "!"
8
9 rcv_email = "gmlstjq123@inha.edu"
10
11 smtp_name = "smtp.naver.com"
12 smtp_port = 587
13
14 msg = MIMEMultipart()
15 msg["Subject"] = "파일과 함께 보내는 편지"
16 msg["From"] = send_email
17 msg["To"] = rcv_email
18
19 text = """
20 다시 만나게 되어 반갑다.
21 아까 파일 보내준다는 걸 깜빡했어.
22 """
23 content = MIMEText(text)
24 msg.attach(content)
25
26 file_path = r"9. 네이버 메일 보내기\my_love.txt"
27 with open(file_path, 'rb') as f:
28     attachment = MIMEApplication(f.read())
29     attachment.add_header('Content-Disposition', 'attachment', filename="my_love.txt")
30     msg.attach(attachment)
31
32 s = smtplib.SMTP(smtp_name, smtp_port)
33 s.starttls()
34 s.login(send_email, send_pwd)
35 s.sendmail(send_email, rcv_email, msg.as_string())
36 s.quit()
```

* MIMEMultipart

Multipart 이메일 메시지를 생성하는 데 사용
텍스트와 첨부파일을 함께 보내는 경우에 사용

* MIMEText

텍스트 기반의 이메일 메시지를 생성하는 데 사용

* MIMEApplication

첨부 파일을 다루는 데 사용

* Content-Disposition

웹 서버가 클라이언트(브라우저)에게 전송하는 콘텐츠에 대한 처리 방법을 지정하는 헤더
이 헤더를 사용하면 파일을 다운로드할 때, 파일 이름과 다운로드 방식 등을 지정할 수 있다.

21 대량 메일 발송

```
# 메일 서버 정보 smtp.naver.com/587
s = smtplib.SMTP(smtpName , smtpPort)
# 메일 서버 접근
s.starttls()
# 메일 서버 로그인
s.login(sendEmail , password)
# 메일 발송
s.sendmail(sendEmail, recvEmail, msg.as_string())
# 메일 닫기
s.close()
```

```
#-----
```

```
# 대량 메일 list 이메일 구성
```

```
recvEmails =
['horim159456@naver.com','vawav@naver.com','swaq11@naver.com']
```

```
# for문 사용
```

```
for recvEmail in recvEmails:
```

```
    msg['To'] = recvEmail
```

```
    msg['Subject'] = title
```

* 메일주소를 list로 구성해서
For문으로 작성하면 됨.

카카오로그인

22 카카오 애플리케이션 설정

카카오 개발자사이트 링크 : <https://developers.kakao.com/>

1. 카카오 개발자사이트 접속
(<https://developers.kakao.com/>)

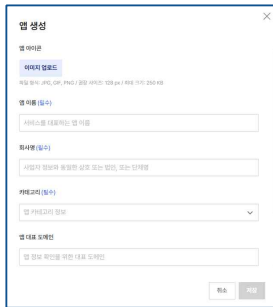
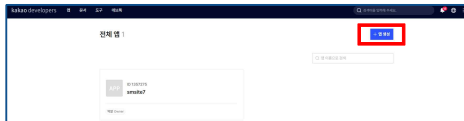
2. 앱 생성

3. 웹 플랫폼 추가 :
앱 이름 - 회사명 - 카테고리 등록

4. 앱 대표 도메인 : 있을 경우 입력
<http://localhost:8000>, <http://127.0.0.1:8000> 안됨.

5. 저장

- 네이티브 앱 키: Android, iOS SDK에서 API를 호출할 때 사용
- JavaScript 키: JavaScript SDK에서 API를 호출할 때 사용
- REST API 키: REST API를 호출할 때 사용
- Admin 키: 모든 권한을 갖고 있는 키다. 노출이 되지 않도록 주의가 필요



22 카카오 로그인 항목설정

카카오 로그인 > 활성화 설정 > 활성화

제품 설정

카카오 로그인

동의 항목

간편가입

카카오톡 채널

개인정보 국제이전

연결 끊기

사용자 프로퍼티

보안

고급

카카오링크

카카오톡 채널

음성

푸시 알림

고급 설정

허용 IP 주소

카카오 로그인 OFF

[동의를 미리보기](#)

활성화 설정

상태

OFF

카카오 로그인 API를 활용하면 사용자들이 번거로운 회원 가입 절차 대신, 카카오톡으로 서비스를 시작할 수 있습니다.
상태가 OFF일 때도 카카오 로그인 설정 항목을 변경하고 서버에 저장할 수 있습니다.
상태가 ON일 때만 실제 서비스에서 카카오 로그인 화면이 연결됩니다.

Redirect URI

Redirect URI 등록

- 카카오 로그인에서 사용할 OAuth Redirect URI를 설정합니다. (최대 10개)
- REST API로 개발하는 경우 필수로 설정해야 합니다.

22 카카오 로그인 항목설정

카카오 로그인 > Redirect URI

1. Redirect URI 설정

- 카카오로그인 후, token키 받을 주소설정

Redirect URI

http://localhost:8000/kakao/oauth

카카오 로그인 ON

[동의창 미리보기](#)

활성화 설정

상태

ON

카카오 로그인 API를 활용하면 사용자들이 번거로운 회원 가입 절차 대신, 카카오톡으로 서비스를 시작할 수 있습니다.

상태가 OFF일 때도 카카오 로그인 설정 항목을 변경하고 서버에 저장할 수 있습니다.

상태가 ON일 때만 실제 서비스에서 카카오 로그인 화면이 연결됩니다.

Redirect URI

Redirect URI 등록

- 카카오 로그인에서 사용할 OAuth Redirect URI를 설정합니다. (최대 10개)
- REST API로 개발하는 경우 필수로 설정해야 합니다.

22 카카오 로그인 항목설정

카카오 로그인 > 고급 > http://localhost:8000/kakao/logout

제품 설정

카카오 로그인

동의항목

간편가입

카카오톡 채널

개인정보 구역이전

연결 끊기

사용자 프로퍼티

보안

보안 이벤트

고급

메시지

카카오톡 채널

푸시 알림

카카오 로그인 ON

Logout Redirect URI

삭제

수정

Logout Redirect URI

http://localhost:8000/kakao/logout

- 카카오 로그인에서 사용할 Logout Redirect URI를 설정합니다. (최대 10개)
- 서비스 로그인 시 카카오 계정과 함께 로그아웃을 위한 스펙으로 선택항목입니다.

사용자 아이디 고정

사용자 아이디(ID)가 고정된 상태입니다.

사용자가 앱 연결, 연결 끊기를 반복해도 사용자 아이디가 동일하게 유지됩니다.

사용자 정보 가져오기에서 받는 사용자는 앱 내에서 유일합니다.

22 카카오 로그인 항목설정

카카오 로그인 > 동의항목

1. 동의항목 선택

- 개인정보에서 필요한 사항 선택
(닉네임, 카카오계정:이메일, 성별 까지 받음)

일반

비즈니스

앱 키

플랫폼

팀 관리

제품 설정

카카오 로그인

동의항목

간판가입

카카오톡 채널

개인정보 국외이전

연결 끊기

사용자 프로파일

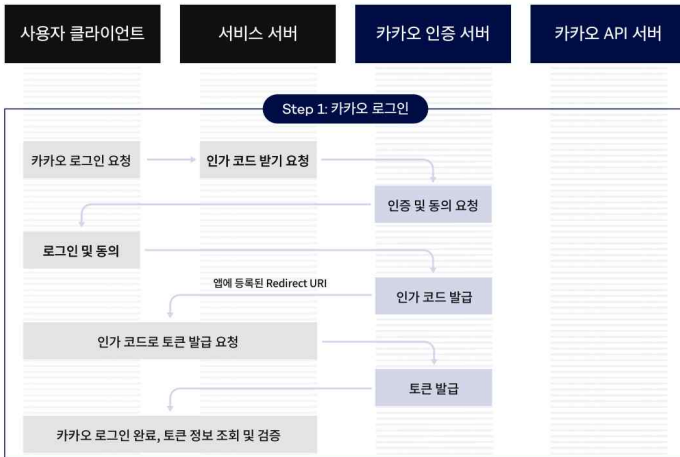
보안

비즈니스 설정 바로가기

개인정보

항목 이름	ID	상태
닉네임	profile_nickname	필수 동의 설정
프로필 사진	profile_image	선택 동의 설정
카카오계정(이메일)	account_email	선택 동의 설정
이름	name	→ 권한 없음
성별	gender	선택 동의 설정
연령대	age_range	사용 안함 설정
생일	birthday	사용 안함 설정

22 카카오 로그인 서비스 과정



22 카카오 로그인 버튼 이미지

카카오 로그인 > 도구 > 리소스 다운로드 > 카카오 로그인

카카오 로그인

카카오싱크

카카오톡

카카오스토리

카카오내비

카카오페이

카카오톡 채널

Karlo

디벨로퍼스

완성형

축약형

버튼 언어

☒ Korean ☐ English

버튼 크기

☒ Middle ☐ Large

버튼 너비

☒ Small ☐ Large

미리보기

카카오 로그인

다운로드(.png)

전체 다운로드(.zip)

22 인가코드 code 키 요청

Code키 값을 요청 : <https://kauth.kakao.com/oauth/authorize> [GET방식 으로 해야 함]
[https://kauth.kakao.com/oauth/authorize?response_type=code&client_id=\\${rest api 키}&redirect_uri=\\${리다이렉트주소}](https://kauth.kakao.com/oauth/authorize?response_type=code&client_id=${rest api 키}&redirect_uri=${리다이렉트주소})

1. GET 방식으로 요청

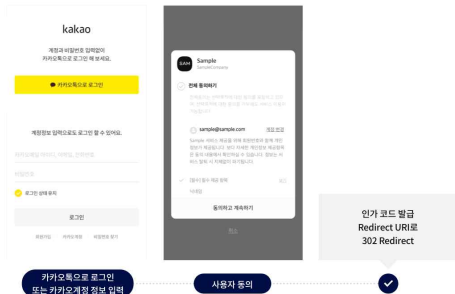
[요청방법]

https://kauth.kakao.com/oauth/authorize?response_type=code&client_id=cca39232502c8598754c9a360e9d0771&redirect_uri=http://localhost:8000/kakao/oauth

쿼리 파라미터 ¶

이름	타입	설명	필수
client_id	String	앱 REST API 키 [내 애플리케이션] > [앱 키]에서 확인 가능	O
redirect_uri	String	인가 코드를 전달받을 서비스 서버의 URI [내 애플리케이션] > [키자오 로그인] > [Redirect URI]에서 등록	O
response_type	String	code로 고정	O

인가 코드 발급 과정



22 인가code 응답

응답은 2가지 성공시 code, 에러시 error

쿼리 파라미터

이름	타입	설명	필수
code	String	토큰 받기 요청에 필요한 인가 코드	X
error	String	인증 실패 시 반환되는 에러 코드	X
error_description	String	인증 실패 시 반환되는 에러 메시지	X
state	String	요청 시 전달한 state값과 동일한 값	X

응답: 사용자가 [동의하고 계속하기] 선택, 로그인 진행

```
HTTP/1.1 302 Found
Content-Length: 0
Location: ${REDIRECT_URI}?code=${AUTHORIZE_CODE}
```

응답: 로그인 취소

```
HTTP/1.1 302 Found
Content-Length: 0
Location: ${REDIRECT_URI}?error=access_denied&error_description=User%20denied%20access
```

22 token키 요청

토큰키 요청 : <https://kauth.kakao.com/oauth/token> [POST방식 으로 해야함]

1. POST 방식으로 요청

2. 요청 URL

- <https://kauth.kakao.com/oauth/token>

3. CODE값 응답 받은 후, 다시 token요청

- 4가지 파라미터 추가해줘야 함.

(
grant_type : authorization_code 고정
client_id : {앱 REST API 키}
redirect_uri : code키 받은 주소
< <http://localhost:8000/kakao/oauth> >
code : 전달받은 code값
)

4. java에서 프로그래밍

- controller.java에 http 전송해야 함.

이름	타입	설명	필수
grant_type	String	authorization_code로 고정	O
client_id	String	앱 REST API 키 [내 애플리케이션] > [앱 키]에서 확인 가능	O
redirect_uri	String	인가 코드가 리다이렉트된 URI	O
code	String	인가 코드 받기 요청으로 얻은 인가 코드	O
client_secret	String	토큰 발급 시, 보안을 강화하기 위해 추가 확인하는 코드 [내 애플리케이션] > [보안]에서 설정 가능 ON 상태인 경우 필수 설정해야 함	X

html

```
<a
href="https://kauth.kakao.com/oauth/authorize?response_type=code&cl
ient_id=d1133ac60217e365da67d276a1313be3&redirect_uri=http://loca
lhost:8000/kakao/oauth
">
  
</a>
```

urls.py

```
from . import views
from django.urls import path,include

app_name = 'kakao'
urlpatterns = [
    path('oauth/',views.oauth,name='oauth' ), # 카카오로그인
]
```

views.py

```
from django.http import HttpResponseRedirect
from django.shortcuts import render

def oauth(request):
    code = request.GET.get("code")
    print("code : ",code)
    return HttpResponseRedirect(f"code : {code}")
```

1.kakao app 추가

w1122\w01

> board

> home

> kakao

> media

> member

> static

> w01

≡ db.sqlite3

🔗 manage.py

22 code 값 받은 후, token 요청

1. Code키를 넣고, token키 요청

```
from django.shortcuts import render
from django.http import HttpResponse
import os
import requests
```

views.py

```
KAKAO_CONFIG = {
    "client_id": "d1133ac60217e365da67d276a1313be3",
    "Redirect_URI": "http://localhost:8000/kakao/oauth/",
}
```

```
kakao_token_uri = "https://kauth.kakao.com/oauth/token" # 토큰키 요청 주소
kakao_profile_uri = "https://kapi.kakao.com/v2/user/me" # 회원정보요청 주소
```

```
def oauth(request):
    code = request.GET.get("code")
    print("리턴받은 코드값 : ",code)
```

```
request_data = { ## 데이터를 묶어서 1개의 형태로 보내어야 함.
    'grant_type': 'authorization_code',
    'client_id': KAKAO_CONFIG['client_id'],
    'Redirect_URI': KAKAO_CONFIG['Redirect_URI'],
    'code': code,
}
```

```
token_headers = {
    'Content-type': 'application/x-www-form-urlencoded;charset=utf-8'
}
```

url링크 : a태그로 연결

https://kauth.kakao.com/oauth/authorize?response_type=code&client_id=d1133ac60217e365da67d276a1313be3&redirect_uri=http://localhost:8000/kakao/oauth

리턴받을 주소 :

[redirect_uri=http://localhost:8000/kakao/oauth](http://localhost:8000/kakao/oauth)
등록을 해야 함.

이름	타입	설명	필수
grant_type	String	authorization_code로 고정	O
client_id	String	앱 REST API 키 [내 애플리케이션] > [앱 키]에서 확인 가능	O
redirect_uri	String	인가 코드가 리다이렉트된 URL	O
code	String	인가 코드 받기 요청으로 얻은 인가 코드	O
client_secret	String	토큰 발급 시, 보안을 강화하기 위해 추가 확인하는 코드 [내 애플리케이션] > [보안]에서 설정 가능 ON 상태인 경우 필수 설정해야 함	X

22 token 받기

2. token키 받음

views.py

```
# post로 token 요청
token_res = requests.post(kakao_token_uri, data=request_data, headers=token_headers)
print(token_res)
token_json = token_res.json() # json()타입으로 변환
print("리턴받은 token값 : ", token_json)
# token 받음
access_token = token_json.get("access_token")

if not access_token:
    return HttpResponse("404에러")
```

뒤로 >

* 토큰키 형태 - json타입

```
{
  "access_token": "qL-
GacZ4QX80x3OBmAilyUM4UaKUmqT3UHleGf6TCj1ymAAAAYlieUIA",
  "token_type": "bearer",
  "refresh_token": "jjY2CAqv0dwwR3U-Z4aZCagj-
se14rSS7T3AD99qCj1ymAAAAYlieUIA",
  "expires_in": 21599,
  "scope": "account_email profile_image gender profile_nickname",
  "refresh_token_expires_in": 5183999
}
```

이름	타입	설명	필수
token_type	String	토큰 타입, bearer로 고정	O
access_token	String	사용자 액세스 토큰 값	O
id_token	String	ID 토큰 값 OpenID Connect 확장 기능을 통해 발급되는 ID 토큰, Base64 인코딩 된 사용자 인증 정보 포함 제공 조건: OpenID Connect가 활성화 된 앱의 토큰 발급 요청인 경우 또는 scope에 openId를 포함한 추가 항목 등의 받기 요청을 거친 토큰 발급 요청인 경우	X
expires_in	Integer	엑세스 토큰과 ID 토큰의 만료 시간(초) 참고: 액세스 토큰과 ID 토큰의 만료 시간은 동일	O
refresh_token	String	사용자 리프레시 토큰 값	O
refresh_token_expires_in	Integer	리프레시 토큰 만료 시간(초)	O
scope	String	인증된 사용자의 정보 조회 권한 범위 범위가 여러 개일 경우, 공백으로 구분 참고: OpenID Connect가 활성화된 앱의 토큰 발급 요청인 경우, ID 토큰이 함께 발급되며 scope 값에 openId 포함	X

22 Kakaoprofile 요청 - token키 사용

Kakaoprofile 요청 : <https://kapi.kakao.com/v2/user/me> [GET/POST 모두가능]

1. GET,POST(모두가능) 방식 요청

2. 요청 URL

- <https://kapi.kakao.com/v2/user/me>

3. 필요한 파라미터 2개를 추가해야 함.

(
Authorization: Bearer
\${ACCESS_TOKEN}
인증 방식, 액세스 토큰으로 인증 요청

Content-type: application/x-www-form-urlencoded;charset=utf-8

)

이름	설명	필수
Authorization	Authorization: Bearer \${ACCESS_TOKEN} 인증 방식, 액세스 토큰으로 인증 요청	O
Content-type	Content-type: application/x-www-form-urlencoded;charset=utf-8 요청 데이터 타입	O

22 token 사용, Kakaoprofile 받기

3. kakaoprofile 요청 – 개인정보 받기

views.py

```
# 회원정보 요청
auth_headers = {
    "Authorization": f"Bearer {access_token}", # 'Bearer ' 마지막 띄어쓰
    "Content-type": "application/x-www-form-urlencoded;charset=utf-8",
}

user_info_res = requests.get(kakao_profile_uri,
headers=auth_headers)

# 회원정보 받음
user_info_json = user_info_res.json()
print(user_info_json) # 회원정보 출력

kakao_account = user_info_json.get('kakao_account')
print("kakao_account : ",kakao_account)
kakao_profile = kakao_account.get('profile')
print("kakao_profile : ",kakao_profile)
kakao_nickname = kakao_profile.get('nickname')
print("kakao_nickname : ",kakao_nickname)

return HttpResponse(f"code : {code}, <br>token_res :
{token_res},<br>token_json : {token_json},<br>access_token :
{access_token},<br>user_info_json : {user_info_json},<br>kakao_profile :
{kakao_profile},<br>kakao_nickname : {kakao_nickname}<br>")
```

json 데이터 가져오기
user_info_json.get("kakao_account") # 이렇게 json데이터 가져옴.

22 kakaoprofile 응답

Kakaoprofile 리턴타입

본문

이름	타입	설명	필수
id	Long	회원번호	O
has_signed_up	Boolean	자동 연결 설정을 비활성화한 경우만 존재 연락처가 호출의 완료 여부 false: 연결 대기(Preregistered) 상태 true: 연결(Registered) 상태	X
connected_at	DateTime	서비스에 연결 완료된 시각, UTC*	X
synced_at	DateTime	카카오싱크 관련 작업을 통해 동기화된 시각, UTC*	X
properties	JSON	사용자 프로퍼티(Property) 사용자 프로퍼티 참고	X
kakao_account	KakaoAccount	카카오계정 정보	X
for_partner	Partner	uid를 추가 정보	X

KakaoAccount

이름	타입	설명	필수
profile_needs_agreement	Boolean	사용자 동의 시 프로필 정보(닉네임/프로필 사진) 제공 가능 필요한 동의 항목: 프로필 정보(닉네임/프로필 사진)	X
profile_nickname_needs_agreement	Boolean	사용자 동의 시 닉네임 제공 가능 필요한 동의 항목: 닉네임	X
profile_image_needs_agreement	Boolean	사용자 동의 시 프로필 사진 제공 가능 필요한 동의 항목: 프로필 사진	X
profile	Profile	프로필 정보 필요한 동의 항목: 프로필 정보(닉네임/프로필 사진), 닉네임, 프로필 사진	X
name_needs_agreement	Boolean	사용자 동의 시 카카오톡 이름 제공 가능 필요한 동의 항목: 이름	X
name	String	카카오톡 이름 필요한 동의 항목: 이름	X

Profile

이름	타입	설명	필수
nickname	String	닉네임 필요한 동의 항목: 프로필 정보(닉네임/프로필 사진) 또는 닉네임	X
thumbnail_image_url	String	프로필 미러보기 이미지 URL 110px * 110px 또는 100px * 100px 필요한 동의 항목: 프로필 정보(닉네임/프로필 사진) 또는 프로필 사진	X
profile_image_url	String	프로필 사진 URL 640px * 640px 또는 480px * 480px 필요한 동의 항목: 프로필 정보(닉네임/프로필 사진) 또는 프로필 사진	X
is_default_image	Boolean	프로필 사진 URL이 기본 프로필 사진 URL인지 여부 사용자가 등록한 프로필 사진이 없을 경우, 기본 프로필 사진 제공 true: 기본 프로필 사진 false: 사용자가 등록한 프로필 사진 필요한 동의 항목: 프로필 정보(닉네임/프로필 사진) 또는 프로필 사진	X

다음지도 API

23 다음지도 api 설정

카카오 개발자사이트 링크 : <https://developers.kakao.com/>

1. 카카오 개발자사이트 접속
(<https://developers.kakao.com/>)

2. 앱 생성

3. 웹 플랫폼 추가 :
앱 이름 - 회사명 - 카테고리 등록

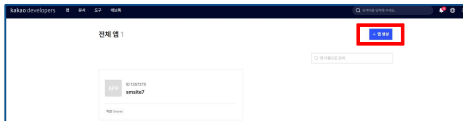
4. 앱 대표 도메인 : 있을 경우 입력
<http://localhost:8000>, <http://127.0.0.1:8000> 안됨.

5. 저장

- 네이티브 앱 키: Android, iOS SDK에서 API를 호출할 때 사용
- JavaScript 키: JavaScript SDK에서 API를 호출할 때 사용
- REST API 키: REST API를 호출할 때 사용
- Admin 키: 모든 권한을 갖고 있는 키다. 노출이 되지 않도록 주의가 필요

아래 코드를 꼭 넣어야 함.

```
<script type="text/javascript" src="//dapi.kakao.com/v2/maps/sdk.js?appkey=발급받은 APP KEY를 사용하세요"></script>
```



23 다음지도 api 설정

카카오 개발자사이트 링크 : <https://developers.kakao.com/console/app/1357275/product/kakao-map>

1. 사용권한 활성화해야 가능

The screenshot shows the 'kakao developers' console interface. On the left, a sidebar menu lists various settings, with '카카오맵' (Kakao Map) highlighted in blue and enclosed in a red rectangle. The main content area displays the '카카오맵' settings for app ID 1357275. Under the '사용 설정' (Usage Settings) section, which is also highlighted with a red rectangle, the '상태' (Status) toggle switch is turned 'ON'. Other sections visible include '카카오 로그인' (Kakao Login) and '쿼터' (Quota).

23 다음지도 api 설정

카카오 개발자사이트 링크 : <https://developers.kakao.com/console/app/1357275/product/kakao-map>

1. 사용권한 활성화해야 가능

The screenshot shows the 'kakao developers' console for app 'smsite7' (ID 1357275). The left sidebar has a red box around '플랫폼 키' (Platform Key) with a red circle and the number '1'. The main content area has two sections: '플랫폼 키' and 'REST API 키'. The 'JavaScript 키' section is highlighted with a red box and a red circle and the number '2'. A modal window titled 'JavaScript 키 수정' (JavaScript Key Edit) is open, showing the 'JavaScript 키' (302e933f7cb0de0fe8d271306dae3d) and '키 이름' (Key Name) as 'Default JS Key'. The 'JavaScript SDK 도메인' (JavaScript SDK Domain) is set to 'http://127.0.0.1:8000'. A red box and a red circle with the number '3' are around the 'JavaScript SDK 도메인' field in the modal.

Annotation 1: 플랫폼 키 (Platform Key)

Annotation 2: JavaScript 키 (JavaScript Key)

Annotation 3: JavaScript SDK 도메인 (JavaScript SDK Domain)

23 Html 페이지 내 다음지도 생성

1. Sample > 지도 생성하기

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>지도 생성하기</title>
```

```
</head>
<body>
```

```
<!-- 지도를 표시할 div 입니다 -->
<div id="map" style="width:100%;height:350px;"></div>
```

```
<script type="text/javascript" src="//dapi.kakao.com/v2/maps/sdk.js?appkey=발급받은 APP KEY를 사
용하세요"></script>
```

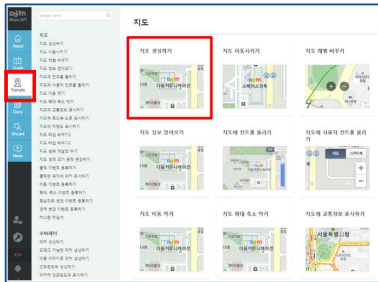
```
<script>
```

// 아래쪽에 있어야 함.

```
var mapContainer = document.getElementById('map'), // 지도를 표시할 div
    mapOption = {
      center: new kakao.maps.LatLng(33.450701, 126.570667), // 지도의 중심좌표
      level: 3 // 지도의 확대 레벨
    };
};
```

// 지도를 표시할 div와 지도 옵션으로 지도를 생성합니다

```
var map = new kakao.maps.Map(mapContainer, mapOption);
</script>
</body>
</html>
```



< 샘플예제 >



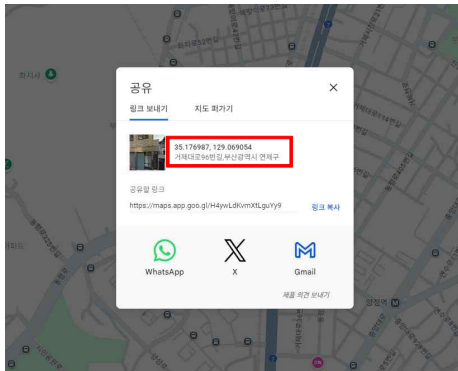
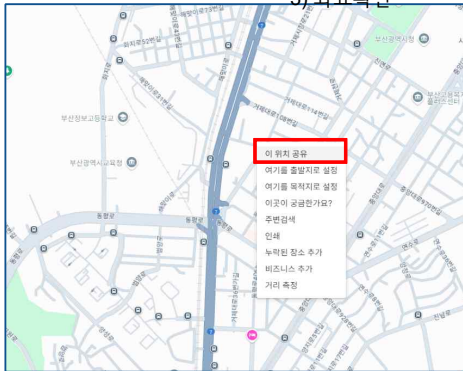
카카오맵 샘플URL : <https://apis.map.kakao.com/web/sample/>

23 위치 경도, 위도 얻기

1. 구글 지도

<https://www.google.co.kr/maps/>

- 1) 구글맵에서 좌표를 얻고 싶은 위치에 마우스 우클릭을 클릭
- 2) '이 위치 공유' 클릭
- 3) 좌표확인



23 마커 생성 - 마커1개 생성하기

1. Sample > 지도 생성하기

```
<script>
var mapContainer = document.getElementById('map'), // 지도를 표시할 div
    mapOption = {
        center: new kakao.maps.LatLng(33.450701, 126.570667), // 지도의 중심좌표
        level: 3 // 지도의 확대 레벨
    };
};
```

// 지도를 표시할 div와 지도 옵션으로 지도를 생성합니다
var map = new kakao.maps.Map(mapContainer, mapOption);

// 마커가 표시될 위치를 변경하면 원하는 지도위치 가능

```
var markerPosition = new kakao.maps.LatLng(33.450701, 126.570667);
```

// 마커를 생성합니다

```
var marker = new kakao.maps.Marker({
    position: markerPosition
});
```

// 마커가 지도 위에 표시되도록 설정합니다

```
marker.setMap(map);
```

// 아래 코드는 지도 위의 마커를 제거하는 코드입니다

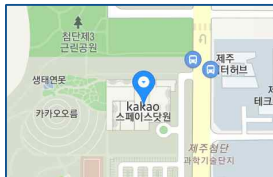
```
// marker.setMap(null);
```

```
</script>
```

```
</body>
```

```
</html>
```

< 샘플예제 >



< 위도경도 찾기 사이트 >

<http://map.esran.com/index.php>



23 마커 생성 -여러개 마커 표시하기

1. Sample > 지도 생성하기

```
<script>
var mapContainer = document.getElementById('map'), // 지도를 표시할 div
    mapOption = {
        center: new kakao.maps.LatLng(33.450701, 126.570667), // 지도의 중심좌표
        level: 3 // 지도의 확대 레벨
    };

var map = new kakao.maps.Map(mapContainer, mapOption); // 지도를 생성합니다
```

```
// 마커를 표시할 위치와 title 객체 배열입니다
var positions = [
    {
        title: '카카오',
        latlng: new kakao.maps.LatLng(33.450705, 126.570677)
    },
    {
        title: '생태연못',
        latlng: new kakao.maps.LatLng(33.450936, 126.569477)
    },
    {
        title: '텃밭',
        latlng: new kakao.maps.LatLng(33.450879, 126.569940)
    },
    {
        title: '근린공원',
        latlng: new kakao.maps.LatLng(33.451393, 126.570738)
    }
];
```

아래 코드를 꼭 넣어야 함.

```
<script type="text/javascript"
src="//dapi.kakao.com/v2/maps/sdk.js?appkey=발급받은 APP KEY
를 사용하세요"></script>
```

```
// 마커 이미지의 이미지 주소입니다
var imageSrc =
    "https://t1.daumcdn.net/localimg/localimages/07/mapapidoc/markerStar.png";

for (var i = 0; i < positions.length; i++) {

    // 마커 이미지의 이미지 크기 입니다
    var imageSize = new kakao.maps.Size(24, 35);

    // 마커 이미지를 생성합니다
    var markerImage = new kakao.maps.MarkerImage(imageSrc, imageSize);

    // 마커를 생성합니다
    var marker = new kakao.maps.Marker({
        map: map, // 마커를 표시할 지도
        position: positions[i].latlng, // 마커를 표시할 위치
        title : positions[i].title, // 마커의 타이틀, 마커에 마우스를 올리면 타이틀이
        image : markerImage // 마커 이미지
    });
}
</script>
</body>
</html>
```

23 주소로 마커 표시하기 - 1개 마커

1. 가져오는 스크립트 확인 - **&libraries=services** 추가 됨.

```
<script type="text/javascript" src="//dapi.kakao.com/v2/maps/sdk.js?appkey=발급받은 APP KEY를 사용하세요  
&libraries=services"></script>
```

```
var mapContainer = document.getElementById('map'), // 지도를  
표시할 div  
mapOption = {  
  center: new kakao.maps.LatLng(33.450701, 126.570667), //  
  지도의 중심좌표  
  level: 3 // 지도의 확대 레벨  
};  
  
// 지도를 생성합니다  
var map = new kakao.maps.Map(mapContainer, mapOption);  
  
// 주소-좌표 변환 객체를 생성합니다  
var geocoder = new kakao.maps.services.Geocoder();
```

```
// 주소로 좌표를 검색합니다  
geocoder.addressSearch('서울특별시 구로구 디지털로34길 27', function(result, status) {
```

```
  // 정상적으로 검색이 완료됐으면  
  if (status === kakao.maps.services.Status.OK) {
```

```
    var coords = new kakao.maps.LatLng(result[0].y, result[0].x);
```

```
    // 결과값으로 받은 위치를 마커로 표시합니다
```

```
    var marker = new kakao.maps.Marker({  
      map: map,  
      position: coords  
    });
```

```
    // 인포윈도우로 장소에 대한 설명을 표시합니다
```

```
    var infowindow = new kakao.maps.InfoWindow({  
      content: '<div style="width:150px;text-align:center;padding:6px 0;">우리회사</div>'  
    });  
    infowindow.open(map, marker);
```

```
    // 지도의 중심을 결과값으로 받은 위치로 이동시킵니다  
    map.setCenter(coords);
```

```
  }  
});  
</script>
```



23 주소로 마커 표시하기 - 여러개 마커표시 + 주소로 좌표 검색

1. 여러 개 마커 표시와 주소로 좌표 검색을 통합

```
<script>
var addressArr = ['서울특별시 구로구 디지털로34길 27','서울특별시 구로구 디지털로34길 43','서울특별시 구로구 디지털로26길 111'];
var positionsData = [];

var aTemp;
for(var i = 0; i<addressArr.length;i++){
  console.log("addressArr : "+addressArr[i]);
  //주소-좌표 변환 객체를 생성합니다
  var geocoder = new kakao.maps.services.Geocoder();
  // 주소로 좌표를 검색합니다
  geocoder.addressSearch(addressArr[i], function(result, status) {
    // 정상적으로 검색이 완료됐으면
    if (status === kakao.maps.services.Status.OK) {
      var coords = new kakao.maps.LatLng(result[0].y, result[0].x);
      console.log("result[0].y : "+result[0].y);
      console.log("result[0].x : "+result[0].x);
      // 주소좌표 추가
      aTemp = { title: addressArr[i], latlng: coords };
      //여러개 마커 부분에 추가
      positionsData.push(aTemp);
      console.log("positionsData : "+positionsData);
    }
  });
  var mapContainer = document.getElementById('map'), // 지도를 표시할 div
  mapOption = {
    center: new kakao.maps.LatLng(37.4851520949372, 126.898809246391),
    // 지도의 중심좌표
    level: 4 // 지도의 확대 레벨
  };
};
```

```
var map = new kakao.maps.Map(mapContainer, mapOption); // 지도를 생성합니다

// 마커를 표시할 위치와 title 객체 배열입니다
var positions = positionsData;
console.log("positions: "+positions);

// 마커 이미지의 이미지 주소입니다
var imageSrc =
"https://t1.daumcdn.net/localimg/localimages/07/mapapidoc/markerStar.png";
for (var i = 0; i < positions.length; i++) {
  // 마커 이미지의 이미지 크기 입니다
  var imageSize = new kakao.maps.Size(24, 35);
  // 마커 이미지를 생성합니다
  var markerImage = new kakao.maps.MarkerImage(imageSrc,
    imageSize);
  // 마커를 생성합니다
  var marker = new kakao.maps.Marker({
    map: map, // 마커를 표시할 지도
    position: positions[i].latlng, // 마커를 표시할 위치
    title : positions[i].title, // 마커의 타이틀, 마커에 마우스를 올리면 타이틀이
    image : markerImage // 마커 이미지
  });
}
//for
//addressSearch
//

</script>
```

23 주소로 마커 표시하기 - 여러개 마커 표시 + 주소로 좌표 검색

1. Ajax으로 데이터를 불러와 addressArr 에 배열로 추가하면 표시가 됨.

```
var addressArr = ['서울특별시 구로구 디지털로34길 27','서울특별시 구로구 디지털로34길 43','서울특별시 구로구 디지털로26길 111'];  
var positionsData = [];
```



23 다른 이미지로 마커 생성

1. Sample > 지도 생성하기

```
<script>
var mapContainer = document.getElementById('map'), // 지도를 표시할 div
    mapOption = {
        center: new kakao.maps.LatLng(37.54699, 127.09598), // 지도의 중심좌표
        level: 4 // 지도의 확대 레벨
    };

var map = new kakao.maps.Map(mapContainer, mapOption); // 지도를 생성합니다

var imageSrc = 'https://t1.daumcdn.net/localimg/localimages/07/mapapidoc/marker_red.png',
    // 마커이미지의 주소입니다
    imageSize = new kakao.maps.Size(64, 69), // 마커이미지의 크기입니다
    imageOption = {offset: new kakao.maps.Point(27, 69)};
    // 마커이미지의 옵션입니다. 마커의 좌표와 일치시킬 이미지 안에서의 좌표를 설정합니다.

// 마커의 이미지정보를 가지고 있는 마커이미지를 생성합니다
var markerImage = new kakao.maps.MarkerImage(imageSrc, imageSize, imageOption),
    markerPosition = new kakao.maps.LatLng(37.54699, 127.09598); // 마커가 표시될 위치입니다

// 마커를 생성합니다
var marker = new kakao.maps.Marker({
    position: markerPosition,
    image: markerImage // 마커이미지 설정
});

// 마커가 지도 위에 표시되도록 설정합니다
marker.setMap(map);

</script>
```

< 샘플예제 >



글쓰기 라이브러리 Summernote

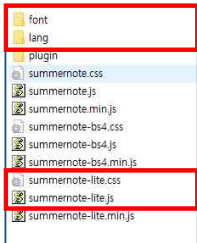
ckeditor, smarteditor2.0

24 Summernote 라이브러리 다운로드/설치

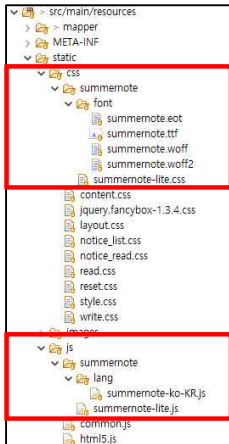
Summernote 링크 : <https://summernote.org/>

1. 파일 압축해제

(부트스트랩 버전이 아닌 것 선택)



2. spring boot 로 복사



24 글쓰기 페이지 내 javascript 및 css 선언

1. jsp페이지 선언 : 파일다운로드 후 연

결

```
<script src="/js/summernote/summernote-lite.js"></script>
<script src="/js/summernote/lang/summernote-ko-KR.js"></script>
<link rel="stylesheet" href="/css/summernote/summernote-lite.css">
```

* 링크 연결 - 아래쪽에 복사 / 링크만 연결만 하면 됨.

```
<!-- include libraries(jQuery, bootstrap) -->
<link href="https://stackpath.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css" rel="stylesheet">
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

<!-- include summernote css/js -->
<link href="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote.min.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote.min.js"></script>
```

2. html form 선언

```
<form action="" name="" method="post">
  <textarea id="summernote" name="bcontent"></textarea>
</form>
```

24 jQuery 선언

1. jQuery 선언

```
$(document).ready(function() {  
    //여기 아래 부분  
    $('#summernote').summernote({  
        height: 300,           // 에디터 높이  
        minHeight: 200,       // 에디터 최소 높이  
        maxHeight: 400,       // 에디터 최대 높이  
        focus: true,          // 에디터 로딩후 포커스를 맞출지 여부  
        lang: "ko-KR",        // 한글 설정  
        placeholder: '최대 2048자까지 쓸 수 있습니다', //placeholder 설정  
    });  
});
```

24 화면

jsp페이지 화면

관리자 글쓰기

이름

제목

sans-serif ▾

16 ▾

B

I

U

S

↺

A ▾

↻

↻

↻

☰

☷

☰ ▾

T1 ▾

☰

oo

☰

×

?

☰ ▾

최대 2048자까지 쓸 수 있습니다

내용

24 상단 메뉴 변경

설명 : <https://summernote.org/deep-dive/#custom-toolbar-popover>

```
$('#summernote').summernote({  
  placeholder: '최대 2048자까지 쓸 수 있습니다' //placeholder 설정  
  // 추가 부분  
  toolbar: [  
    // [groupName, [list of button]]  
    ['fontname', ['fontname']],  
    ['fontsize', ['fontsize']],  
    ['style', ['bold', 'italic', 'underline', 'strikethrough', 'clear']],  
    ['color', ['forecolor', 'color']],  
    ['table', ['table']],  
    ['para', ['ul', 'ol', 'paragraph']],  
    ['height', ['height']],  
    ['insert', ['picture', 'link', 'video']],  
    ['view', ['fullscreen', 'help']]  
  ],  
  fontNames: ['Arial', 'Arial Black', 'Comic Sans MS', 'Courier New', '맑은 고딕',  
    '궁서', '굴림체', '굴림', '돋움체', '바탕체'],  
  fontSizes: ['8', '9', '10', '11', '12', '14', '16', '18', '20', '22', '24', '28', '30', '36', '50', '72']  
});
```



24 View html 설정

```
<div class="viewContents">
  {{board.bcontent|safe}}
</div>
```

View.html

safe : HTML 태그를 이스케이프 하지 않음
capitalize : 문자열의 첫 글자를 대문자로 변환
lower : 문자열을 소문자로 변환
upper : 문자열을 대문자로 변환
title : 각 단어의 첫 글자를 대문자로 변환
trim : 문자열 앞뒤의 공백을 제거
striptags : HTML 태그를 제거
truncate : 문자열을 특정 길이로 줄임
date : datetime 객체를 문자열로 포맷

Django에서
HTML 텍스트를 그대로 불러올 수 없음.

Safe 붙여야
문자가 아닌 html태그로 읽어짐.

```
{{board.bcontent|safe }}
{{board.bcontent|striptags }}
```

```
<script>
$(window).on('beforeunload', function () {
  if (checkUnload) return "이 페이지를 벗어나면 작성된 내용은 저장되지 않습니다.";
});
$("#sbtnMini").on("click", function () {
  checkUnload = false;
  writeFrm.submit();
});
</script>
```

write.html

페이지 벗어나기 경고창 구현

작성 창에서 새로고침, 뒤로가기 등
사용자가 작성중인 페이지를 벗어날
시 경고 alert창 띄움

24 callbacks 설정

```
//이미지 업로드 시 width 적용
// 이미지를 base64형태로 변경
callbacks: {
  onImageUpload: function(image) {
    console.log('in');
    const file = image[0];
    const reader = new FileReader();

    reader.onloadend = function() {
      const image = $('<img>').attr('src', reader.result);
      image.attr('width','100%');
      $('#summernote').summernote("insertNode", image[0]);
    }
    reader.readAsDataURL(file);
  }
}
```

write.html

* 이미지 미리보기 – data base64로 변경

- img File을 js에서 제공하는
FileReader 객체를 생성

- readAsDataURL 이라는 메서드를 통해
img File의 data를 base64로 형태로 인코딩

Base64 인코딩은 아스키코드에서 사용되는 문자64개로 Data를 표현

24 summernote 이미지 업로드설정

```
$(function() {
```

```
    //여기 아래 부분
```

```
    ...
```

```
    fontNames: ['Arial', 'Arial Black', 'Comic Sans MS', 'Courier New', '맑은 고딕', '궁서', '굴림체', '굴림', '돋움체', '바탕체'],
```

```
    fontSizes: ['8', '9', '10', '11', '12', '14', '16', '18', '20', '22', '24', '28', '30', '36', '50', '72'],
```

```
    callbacks: { //여기 부분이 이미지를 첨부하는 부분
```

```
        onImageUpload : function(files) {
```

```
            uploadSummernoteImageFile(files[0], this); //이미지 업로드 호출
```

```
        },
```

```
        onPaste: function (e) {
```

```
            var clipboardData = e.originalEvent.clipboardData;
```

```
            if (clipboardData && clipboardData.items &&
```

```
                clipboardData.items.length) {
```

```
                var item = clipboardData.items[0];
```

```
                if (item.kind === 'file' && item.type.indexOf('image/') !== -1) {
```

```
                    e.preventDefault();
```

```
                }
```

```
            }
```

```
        }
```

```
    } //callbacks
```

```
}); //#summernote
```

```
}); //jquery
```

write.html

summernote기본설정 : base64로 인코딩 후 저장하는 방식
callback을 이용하여 이미지를 특정 경로에 업로드 후, 고유
한 url를 리턴하는 방식으로 구현

스크립트 추가부분

```
/**
```

```
 * 이미지 파일 업로드
```

```
 */
```

```
function uploadSummernoteImageFile (file, this_editor) {
```

```
    var form_data = new FormData();
```

```
    form_data.append('file', file);
```

```
    $.ajax({
```

```
        data: form_data,
```

```
        type: "POST",
```

```
        url: '/profileUpload',
```

```
        cache: false,
```

```
        contentType: false,
```

```
        enctype: 'multipart/form-data',
```

```
        processData: false,
```

```
        success: function(img_name) {
```

```
            $(this_editor).summernote('editor.insertImage', img_name);
```

```
        }
```

```
    });
```

```
}
```

이미지 업로드 추가부분

24 Views.py

```
from django.views.decorators.csrf import csrf_exempt
from django.http import JsonResponse
from django.core.files.storage import default_storage
from django.core.files.base import ContentFile
```

write.html

```
# summernote 파일첨부
@csrf_exempt # AJAX CSRF 처리 (토큰 쓰면 제거 가능)
def profileUpload(request):
    if request.method == 'POST':
        # 브라우저에서 업로드 된 파일
        file = request.FILES.get('file')

        # default_storage → 파일 저장소 : MEDIA 설정 자동 사용
        # 저장 : MEDIA_ROOT, URL : MEDIA_URL
        # file.read() : 파일의 바이너리 데이터를 읽어옴 / ContentFile : 디장고 파일 객체변환
        file_path = f'summernote/{file}'
        saved_path = default_storage.save(file_path, ContentFile(file.read()))

        # MEDIA_URL 기준으로 이미지 URL 생성
        image_url = default_storage.url(saved_path)

    return JsonResponse(image_url, safe=False)
```

summernote 기본설정 : base64로 인코딩 후 저장하는 방식
callback을 이용하여 이미지를 특정 경로에 업로드 후, 고유
한 url을 리턴하는 방식으로 구현

카카오페이

25 카카오페이 api 설정

카카오 페이 개발자 링크 : <https://developers.kakaopay.com/applications>

1. 카카오 개발자사이트 접속

(<https://developers.kakaopay.com/applications>)

2. 애플리케이션 등록 및 기본정보 - Secret key(dev) 발급

가맹점 코드로 "TC0ONETIME"와 "Secret key(dev)"를 통해 결제 호출이 가능

3. 플랫폼 Web 추가 :

앱 선택 - [플랫폼] - [Web 플랫폼 등록] - 사이트 도메인 등록

4. 사이트 도메인 등록: [웹] 플랫폼을 선택하고,
[사이트 도메인] 을 등록 (예: <http://127.0.0.1:8000>)

5. 앱을 실행.

25 카카오 페이 항목설정

애플리케이션 > 등록

pay developers

제품 문서 **</> 애플리케이션** 포럼 지원

애플리케이션 목록

	룸메이크 ROOMMAKE
---	---------------

애플리케이션 등록

앱 아이콘	 파일 선택 JPG, GIF, PNG 권장사이즈 : 128px, 최대 250kb
앱 이름	내 애플리케이션 이름 0/24

취소등록

© Kakao Pay Corp.

서비스약관

25 카카오 페이 항목설정

애플리케이션 > 기본정보 > 발급정보 > Secret key(dev) 발급

pay developers

제품

문서

</>
애플리케이션

포럼

지원

기본 정보

사용 API

카카오페이 로그인

플랫폼

권한 관리

통계

애플리케이션 > 기본 정보

발급정보

Client ID	
Client Secret	재발급
Secret key	재발급
Secret key(dev)	'발급' 버튼 클릭 시 키가 발급됩니다. 발급

Secret key(dev) 키로 개발 확인을 해야 함.

25 카카오 페이 항목설정

애플리케이션 > 플랫폼

pay developers

제품

문서



애플리케이션

포럼

지원

애플리케이션 > 플랫폼

기본 정보

사용 API

카카오페이 로그인

플랫폼

권한 관리

통계

Web

Web 에 꼭 등록을 해야 함.

삭제

수정

사이트도메인

https://localhost:80

http://127.0.0.1:8000

Redirect URL

25 카카오 페이 연결페이지

kakaoView.html

```
<head>
<script src="http://code.jquery.com/jquery-latest.min.js" ></script>
<title>카카오 페이</title>
<script>
  $(function(){
    $("#kpay").click(function(){
      alert("카카오페이 연결")
      location.href="/board/prepare_payment/"
    })
  })
</script>
</head>

<body>
<h2>카카오페이</h2>
<button type="button" id="kpay">카카오결제하기</button>
</body>
```

/board/prepare_payment/ 페이지로 연결

25 카카오페이 urls.py

urls.py

```
from django.urls import path,include
from . import views

app_name= 'payment'
urlpatterns = [
    # 카카오페이 연결 버튼 페이지
    path('kakaopay/', views.kakaopay, name="kakaopay"),

    # 결제화면 QR코드 창
    path('prepare_payment/', views.prepare_payment, name="prepare_payment"),

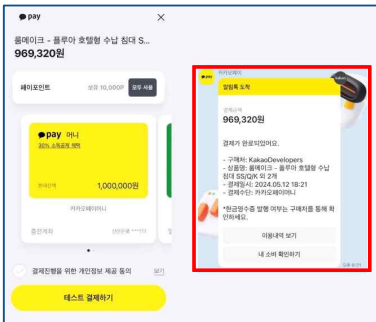
    # 결제승인 창
    path('approve/', views.approve, name='approve'),

    # 결제후 결과창
    path('paySuccess/', views.paySuccess, name='paySuccess'),
    path('payFail/', views.payFail, name="payFail"),
    path('payCancel/', views.payCancel, name="payCancel"),
```

]



Approve 페이지를 연결해야
결제완료창이 개인에게 전달됨.



25 카카오페이 views.py

```
from django.shortcuts import render
from board.models import Board
from member.models import Member
from comment.models import Comment
from django.db.models import F,Q
from django.core.paginator import Paginator
from django.http import JsonResponse
import requests
from django.conf import settings
import json
```

```
KAKAO_API_KEY='DEV09ED661AF03847802D5A5F38EC59AC08698DD'
KAKAOPAY_BASE_URL = 'https://open-api.kakaopay.com/online/v1/payment/ready'
```

1. 카카오페이 연결을 위한 페이지

```
def kakaopay(request):
    return render(request,"kakaopay.html")
```

2. 카카오페이 결제 준비 API 호출

```
def prepare_payment(request):
```

```
    headers = {
        "Authorization":f"SECRET_KEY {KAKAO_API_KEY}",
        "Content-Type":"application/json",
    }
```

```
print("headers : ",headers)
```

Views.py

```
data = {
    "cid": "TC00NETIME", # 테스트용 CID, 상용은 발급받은 CID 사용
    "partner_order_id": "order_id_12345", # 고유 주문 ID
    "partner_user_id": "user_id_67890", # 고유 사용자 ID
    "item_name": "Test Product", # 상품명
    "quantity": "1", # 수량 (0이 아닌 양수)
    "total_amount": "1000", # 총 결제 금액 (0보다 커야 함)
    "vat_amount": "100", # 부가세 금액 (총 금액보다 작거나 같아야 함)
    "tax_free_amount": "0", # 비과세 금액 (생략 가능)
    "approval_url": "http://127.0.0.1:8000/payment/paySuccess",
    "cancel_url": "http://127.0.0.1:8000/payment/payFail",
    "fail_url": "http://127.0.0.1:8000/payment/payCancel",
}
```

```
print("KAKAOPAY_BASE_URL : ",KAKAOPAY_BASE_URL)
print("data : ",data)
print("전 송 데이터 :", json.dumps(data, ensure_ascii=False))
```

```
response = requests.post(KAKAOPAY_BASE_URL, headers=headers,
data=json.dumps(data, ensure_ascii=False))
result = response.json()
```

```
print("결과 : ",result)
```

결과 넘어옴.

```
request.session["tid"] = result["tid"]
```

```
if response.status_code == 200:
```

```
    print("다음 결과 url : ",result["next_redirect_pc_url"])
```

```
    return redirect(result["next_redirect_pc_url"])
```

```
    # return JsonResponse({"next_redirect_pc_url": result["next_redirect_pc_url"]})
```

```
else:
```

```
    return JsonResponse({"error": result}, status=400)
```

25 카카오페이 views.py

```
from django.shortcuts import render
from board.models import Board
from member.models import Member
from comment.models import Comment
from django.db.models import F,Q
from django.core.paginator import Paginator
from django.http import JsonResponse
import requests
from django.conf import settings
import json
```

```
KAKAO_API_KEY='DEV09ED661AF03847802D5A5F38EC59AC08698DD'
KAKAOPAY_BASE_URL = 'https://open-api.kakaopay.com/online/v1/payment/ready'
```

2. 카카오페이 결제승인창

```
def approve(request):
```

```
    # 리턴받은 값
```

```
    pg_token = request.GET.get('pg_token')
```

```
    tid = request.session.get("tid")
```

```
    order_id = request.session.get("order_id")
```

```
    print("리턴받은 값 : ",pg_token,tid,order_id)
```

```
    if not pg_token or not tid:
```

```
        return redirect('/product/fail/')
```

Views.py

```
url = 'https://open-api.kakaopay.com/online/v1/payment/approve'
headers = {
    'Authorization':f'SECRET_KEY {KAKAO_API_KEY}',
    'Content-Type':'application/json',
}
```

```
data = {
    "cid": "TC0ONETIME",
    "tid": tid,
    "partner_order_id": order_id,
    "partner_user_id": "partner_user_id",
    "pg_token": pg_token
}
```

```
response =
requests.post(url,headers=headers,data=json.dumps(data,ensure_ascii=False))
print('리턴받은 결과 : ',response)
result = response.json()
print('결제 날짜 : ',result.get('created_at'))
```

```
if result.get('aid'):
    # db에 저장 - 결제정보
    # Payment.objects.create(          )
```

```
print('결제 성공!!!')
return redirect('/product/success/')
```

```
return redirect('/product/fail/')
```

25 카카오 페이 views.py - 카카오 data 넘기기

Views.py

```
response = requests.post(KAKAOPAY_BASE_URL, headers=headers, data=json.dumps(data, ensure_ascii=False))
result = response.json()
```

```
# ensure_ascii=True (기본값)
print(json.dumps(data, ensure_ascii=True))
# 출력: {"\uba54\uc2dc\uc9c0": "\uc548\ub155\ud558\uc138\uc694", "key2": "value2"}
```

```
# ensure_ascii=False
print(json.dumps(data, ensure_ascii=False))
# 출력: {"메시지": "안녕하세요", "key2": "value2"}
```

```
ensure_ascii=False
```

기본적으로 json.dumps는 모든 문자열을 ASCII 형식으로 변환
이 옵션을 False로 설정하면 비-ASCII 문자(예: 한글, 일본어, 특수 문자 등)가 그대로 출력
Unicode 이스케이프 형태로 출력되지 않도록 하기 위해 사용

```
indent=4
```

JSON 데이터를 보기 좋게 들여쓰는 데 사용
숫자 4는 각 계층에서 들여쓰는 공백(스페이스)의 수를 의미
들여쓰기를 통해 사람이 읽기 쉽게 출력

25 카카오 페이 views.py – 카카오 넘어온 데이터

카카오 페이 넘어온 결과 페이지 : <https://open-api.kakaopay.com/online/v1/payment/ready>

[1차 넘어온 데이터]

Views.py

```
결과 : {  
'tid': 'T75fef41555964df381b',  
'tms_result': False, 'created_at': '2024-12-16T18:13:37',  
'next_redirect_pc_url': 'https://online-payment.kakaopay.com/mockup/bridge/pc/pg/one-time/payment/645a99661623bbfbc1659ad118119ea024552221fa15c9715af86b91af2c70a5',  
'next_redirect_mobile_url': 'https://online-payment.kakaopay.com/mockup/bridge/mobile-web/pg/one-time/payment/645a99661623bbfbc1659ad118119ea024552221fa15c9715af86b91af2c70a5',  
'next_redirect_app_url': 'https://online-payment.kakaopay.com/mockup/bridge/mobile-app/pg/one-time/payment/645a99661623bbfbc1659ad118119ea024552221fa15c9715af86b91af2c70a5',  
'android_app_scheme': 'kakaotalk://kakaopay/pg?url=https://online-pay.kakao.com/pay/mockup/645a99661623bbfbc1659ad118119ea024552221fa15c9715af86b91af2c70a5',  
'ios_app_scheme': 'kakaotalk://kakaopay/pg?url=https://online-pay.kakao.com/pay/mockup/645a99661623bbfbc1659ad118119ea024552221fa15c9715af86b91af2c70a5'}
```

25 카카오 페이 views.py – approve

ready QR결제화면 후 : <https://open-api.kakaopay.com/online/v1/payment/approve>

Views.py

```
# 리턴받은 값
pg_token = request.GET.get('pg_token')
tid = request.session.get("tid")
order_id = request.session.get("order_id")
headers = {
    'Authorization': f'SECRET_KEY {KAKAO_API_KEY}',
    'Content-Type': 'application/json',
}
response = requests.post(url, headers=headers, data=json.dumps(data, ensure_ascii=False))
```

[2차 넘어온 데이터]

result 결과 페이지

리턴받은 값 : 7c72dc6f3478000b5d64, T94a02d6429e53ffc577, partner_order_id

리턴받은 결과 : <Response [200]>

결제 날짜 : 2025-12-23T11:47:51

결제 성공!!!

25 View html 설정

```
<div class="viewContents">
  {{board.bcontent|safe}}
</div>
```

View.html

safe : HTML 태그를 이스케이프 하지 않음
capitalize : 문자열의 첫 글자를 대문자로 변환
lower : 문자열을 소문자로 변환
upper : 문자열을 대문자로 변환
title : 각 단어의 첫 글자를 대문자로 변환
trim : 문자열 앞뒤의 공백을 제거
striptags : HTML 태그를 제거
truncate : 문자열을 특정 길이로 줄임
date : datetime 객체를 문자열로 포맷

Django에서
HTML 텍스트를 그대로 불러올 수 없음.

Safe 붙여야
문자가 아닌 html태그로 읽어짐.

```
{{board.bcontent|safe }}
{{board.bcontent|striptags }}
```

```
<script>
$(window).on('beforeunload', function () {
  if (checkUnload) return "이 페이지를 벗어나면 작성된 내용은 저장되지 않습니다.";
});
$("#sbtnMini").on("click", function () {
  checkUnload = false;
  writeFrm.submit();
});
</script>
```

write.html

페이지 벗어나기 경고창 구현

작성 창에서 새로고침, 뒤로가기 등
사용자가 작성중인 페이지를 벗어날
시 경고 alert창 띄움

25 Approve > models.py 설정

```
from django.db import models
```

Models.py

```
class Payment(models.Model):
    aid = models.CharField(max_length=30)
    tid = models.CharField(max_length=30)
    cid = models.CharField(max_length=20)

    partner_order_id = models.CharField(max_length=100)
    partner_user_id = models.CharField(max_length=100)

    payment_method_type = models.CharField(max_length=20)

    item_name = models.CharField(max_length=100)
    # 0 이상(양수) 정수만 저장 가능한 필드
    quantity = models.PositiveIntegerField()

    created_at = models.DateTimeField()
    approved_at = models.DateTimeField()

    def __str__(self):
        return f"{self.item_name} ({self.tid})"
```

```
class PaymentAmount(models.Model):
    # 1 : 1 관계 - 하나의 Payment 와만 연결
    payment = models.OneToOneField(
        Payment,
        on_delete=models.CASCADE,
        related_name="amount" # 역방향 참조
    )
```

```
total = models.PositiveIntegerField()
tax_free = models.PositiveIntegerField()
vat = models.PositiveIntegerField()
point = models.PositiveIntegerField()
discount = models.PositiveIntegerField()
green_deposit = models.PositiveIntegerField()
```

```
def __str__(self):
    return f"Amount: {self.total}"
```

25 Create 저장

```
payment = Payment.objects.create(  
    aid=result["aid"],  
    tid=data["tid"],  
    cid=data["cid"],  
    partner_order_id=result["partner_order_id"],  
    partner_user_id=result["partner_user_id"],  
    payment_method_type=result["payment_method_type"],  
    item_name=result["item_name"],  
    quantity=result["quantity"],  
    created_at=parse_datetime(result["created_at"]),  
    approved_at=parse_datetime(result["approved_at"]),  
)
```

views.py

```
PaymentAmount.objects.create(  
    payment=payment,  
    total=result["amount"]["total"],  
    tax_free=result["amount"]["tax_free"],  
    vat=result["amount"]["vat"],  
    point=result["amount"]["point"],  
    discount=result["amount"]["discount"],  
    green_deposit=result["amount"]["green_deposit"],  
)
```

채팅 앱

26 카카오페이 api 설정

채팅앱

1. 카카오 개발자사이트 접속

(<https://developers.kakaopay.com/applications>)

2. 애플리케이션 등록 및 기본정보 - Secret key(dev) 발급

가맹점 코드로 "TC0ONETIME"와 "Secret key(dev)"를 통해 결제 호출이 가능

3. 플랫폼 Web 추가 :

앱 선택 - [플랫폼] - [Web 플랫폼 등록] - 사이트 도메인 등록

4. 사이트 도메인 등록: [웹] 플랫폼을 선택하고, [사이트 도메인] 을 등록 (예: <http://127.0.0.1:8000>)

5. 앱을 실행.

THANK YOU